ETAS RTA-OS VRTA

Status: Released

Virtual ECU User Guide

www.etas.com

Copyright

The data in this document may not be altered or amended without special notification from ETAS GmbH. ETAS GmbH undertakes no further obligation in relation to this document. The software described in it can only be used if the customer is in possession of a general license agreement or single license. Using and copying is only allowed in concurrence with the specifications stipulated in the contract.

Under no circumstances may any part of this document be copied, reproduced, transmitted, stored in a retrieval system or translated into another language without the express written permission of ETAS GmbH.

© Copyright 2025 ETAS GmbH, Stuttgart

The names and designations used in this document are trademarks or brands belonging to the respective owners.

ETAS RTA-OS VRTA V3.0.54 – Virtual ECU User Guide EN - 01-2025

Safety Notice

This ETAS product fulfils standard quality management requirements. If requirements of specific safety standards (e.g. IEC 61508, ISO 26262) need to be fulfilled, these requirements must be explicitly defined and ordered by the customer. Before use of the product, customer must verify the compliance with specific safety standards.

Contents

Safety Notice....3

Contents....4

1 Welcome to the RTA-OS Virtual Target....13

1.1 Related Documents....15

1.2 About You....15

1.3 Document Notes....15

1.4 References....15

2 Introduction to the Virtual ECU....16
2.1.1 What do I need?....17
2.1.2 VRTA-win....17
2.1.3 VRTA-ux....18

2.2 What is inside a VECU?....19
2.2.1 Application Manager....20
2.2.2 Device Manager....20
2.2.3 Interrupt Control Unit....21
2.2.4 Embedded GUI (VRTA-win only)....22

2.3 Managing Virtual ECUs....24
2.3.1 vrtaServer.exe: VRTA-win and VRTA-ux (WSL2)....24
2.3.2 vrtaServerCli.exe: VRTA-win and VRTA-ux (WSL2)....24
2.3.3 vrtaServer: VRTA-ux....25
2.3.4 vrtaServerWSL: VRTA-ux....25

2.4 Interacting with a Virtual ECU....26

2.5 Installing VRTA-win....26

2.6 Installing VRTA-ux....27

2.7 Windows Notes....28
2.7.1 Real-Time Behavior....28
2.7.2 Calling the C/C++ Runtime and Windows....28
2.7.3 vrtaVM.dll Location....28

2.8 Linux Notes....29
2.8.1 Real-Time Behavior....29

2.9 Possible Problem Areas....30
2.9.1 General....30
2.9.2 VRTA-win....30
2.9.3 VRTA-ux....30

3 Tutorial....31

3.1 Prerequisites....31

3.2 Creating your first Virtual ECU: Part1....31
3.2.1 Configuring RTA-OS....32
3.2.2 Building RTA-OS....33
3.2.3 Writing Application Code....35
3.2.4 Writing VECU Code....35

3.2.5 Building the Virtual ECU using VRTA-win ... 35
3.2.6 Building the Virtual ECU using VRTA-ux ... 36
3.2.7 Running the VRTA-win Virtual ECU ... 37
3.2.8 Running the VRTA-ux Virtual ECU ... 37

3.3 Embedding a VECU inside a DLL or executable: vrtaInProcessAPI ... 38

3.4 Adding Devices ... 39
3.4.1 Clocks ... 40
3.4.2 Sensors ... 42
3.4.3 Actuators ... 42
3.4.4 IO ... 43
3.4.5 Custom Devices ... 43

3.5 Creating your first Virtual ECU: Part2 ... 44
3.5.1 Devices ... 44
3.5.2 Logger ... 46
3.5.3 Interfacing C code with C++ Devices ... 46
3.5.4 Device Initialization ... 47
3.5.5 The main program ... 48
3.5.6 Trial Run 1 ... 49
3.5.7 Summary so far ... 50
3.5.8 Adding Tasks and ISRs ... 51
3.5.9 Threads ... 54
3.5.10 Trial run 2 ... 56
3.5.11 Linking to Real Hardware (VRTA-win) ... 56
3.5.12 Non-volatile Data ... 57

3.6 Summary ... 59

4 ECUs and Virtual Devices ... 60

4.1 Low-level API ... 61
4.1.1 Registering the device ... 61
4.1.2 Example low-level device implementation ... 63
4.1.3 Handling actions ... 63
4.1.4 Handling state queries ... 64
4.1.5 Raising events ... 64
4.1.6 Raising interrupts ... 64

4.2 Virtual Devices using the VirtualDevice class ... 65
4.2.1 Creating the device ... 65
4.2.2 Handling actions ... 68
4.2.3 Handling state queries ... 69
4.2.4 Raising events ... 71
4.2.5 Raising interrupts ... 72

4.3 Virtual Devices using the vrtaDevice class (deprecated) ... 73
4.3.1 Creating the device ... 73
4.3.2 Example vrtaDevice-based implementation ... 73
4.3.3 Handling actions ... 73
4.3.4 Handling state queries ... 74
4.3.5 Raising events ... 75
4.3.6 Raising interrupts ... 75

4.4 Parent / Child relationships ... 77

4.5 Threads ... 77

5 vrtaServer ... 79

5.1 Multiple instances of a VECU ... 81

5.2 Server status information ... 81
5.3 Security issues ... 82
6 vrtaMonitor GUI ... 83
6.1 Actions ... 84
6.2 Events ... 85
6.2.1 Query ... 85
6.2.2 Monitor ... 86
7 vrta command-line monitor ... 90
7.1 Options ... 90
7.1.1 Common options ... 91
7.2 Load command ... 93
7.3 Start command ... 93
7.4 Run command ... 93
7.5 Pause command ... 94
7.6 Resume command ... 94
7.7 Reload command ... 94
7.8 Kill command ... 94
7.9 List command ... 94
7.10 Describe command ... 95
8 Migration from a VECU to Real Hardware ... 96
8.1 ARXML file ... 96
8.1.1 Target and variant ... 96
8.1.2 Interrupts ... 97
8.1.3 Number of tasks ... 97
8.2 Hardware drivers ... 97
8.3 Initialization ... 98
8.4 Interrupts ... 99
8.5 Building the application ... 99
9 VRTA API Reference ... 100
9.1 General notes ... 100
9.1.1 API Header Files ... 100
9.1.2 Linkage ... 100
9.2 Common Data Types ... 100
9.2.1 vrtaDevID ... 100
9.2.2 vrtaActionID ... 100
9.2.3 vrtaEventID ... 100
9.2.4 vrtaActEvID ... 100
9.2.5 vrtaISRID ... 100
9.2.6 vrtaIntPriority ... 100
9.2.7 vrtaErrType ... 101
9.2.8 vrtamillisecond ... 101
9.2.9 vrtaTimestamp ... 101
9.2.10 vrtaBoolean ... 101

9.2.11 vrtaByte ... 101
9.2.12 vrtaDataLen ... 102
9.2.13 vrtaEmbed ... 102
9.2.14 vrtaAction ... 102
9.2.15 vrtaEvent ... 104
9.2.16 vrtaTextPtr ... 106
9.2.17 vrtaStringlistPtr ... 106
9.2.18 vrtaOptStringlistPtr ... 106

9.3 Data format Strings ... 106
9.3.1 Overview ... 106
9.3.2 Definition ... 107

9.4 API Functions ... 110
9.4.2 InitializeDevices() ... 110
9.4.3 OS_MAIN() ... 111
9.4.4 vrtaEnterUninterruptibleSection() ... 112
9.4.5 vrtaEventRegister() ... 113
9.4.6 vrtaEventUnregister() ... 114

9.5 vrtaGetState() ... 115
9.5.2 vrtaHookEvent() ... 117
9.5.3 vrtaInitialize() ... 118

9.6 vrtaIsAppFinished() ... 120
9.6.2 vrtaIsAppThread() ... 121
9.6.3 vrtaIsIdle() ... 122
9.6.4 vrtaYield() ... 122
9.6.5 vrtaLeaveUninterruptibleSection() ... 123
9.6.6 vrtaLoadVM() ... 124
9.6.7 vrtaNoneUserThread() ... 125
9.6.8 vrtaOSGetIPL() ... 125
9.6.9 vrtaOSSetIPL() ... 126
9.6.10 vrtaRaiseEvent() ... 126
9.6.11 vrtaReadHPTime() ... 127

9.7 vrtaRegisterVirtualDevice() ... 128
9.7.2 vrtaReset() ... 132
9.7.3 vrtaSendAction() ... 134
9.7.4 vrtaSpawnThread() ... 134
9.7.5 vrtaStart() ... 135
9.7.6 vrtaTerminate() ... 136

10 Standard Device Reference ... 138

10.1 vrtaStdDevices.h ... 138

10.2 Action and Event Descriptions ... 138

10.3 Device Manager ... 138
10.3.1 Action: EventRegister ... 139
10.3.2 Action: HookEvents ... 139
10.3.3 Action: ListAll ... 139
10.3.4 Action: GetDeviceActions ... 140
10.3.5 Action: GetDeviceEvents ... 140
10.3.6 Action: GetDeviceInfo ... 140
10.3.7 Event: DeviceList ... 140
10.3.8 Event: DeviceActions ... 140
10.3.9 Event: DeviceEvents ... 140
10.3.10 Event: DeviceInfo ... 141

10.4 Interrupt Control Unit ... 141
10.4.1 Action: Raise ... 142
10.4.2 Action: Clear ... 142
10.4.3 Action: Mask ... 142
10.4.4 Action: Unmask ... 142
10.4.5 Action: GetPending ... 143
10.4.6 Action: GetIPL ... 143
10.4.7 Action: SetIPL ... 143
10.4.8 Event: Pending ... 143
10.4.9 Event: Start ... 143
10.4.10 Event: Stop ... 144
10.4.11 Event: IPL ... 144
10.4.12 Event: EnabledVecs ... 144

10.5 Application Manager ... 144
10.5.1 Action: Start ... 144
10.5.2 Action: Terminate ... 145
10.5.3 Action: Pause ... 145
10.5.4 Action: Restart ... 145
10.5.5 Action: Reset ... 145
10.5.6 Action: GetInfo ... 145
10.5.7 Action: TestOption ... 146
10.5.8 Action: ReadOption ... 146
10.5.9 Action: ReadParam ... 146
10.5.10 Event: Started ... 146
10.5.11 Event: Paused ... 146
10.5.12 Event: Restarted ... 146
10.5.13 Event: Reset ... 147
10.5.14 Event: Terminated ... 147
10.5.15 Event: Info ... 147
10.5.16 Event: Option ... 147
10.5.17 Event: OptionText ... 147
10.5.18 Event: ParamText ... 147
10.5.19 Event: State ... 148

11 Sample Device Reference ... 149

11.1 Summary ... 149

11.2 Device Descriptions ... 150
11.2.1 Methods ... 150
11.2.2 Actions and Events ... 150

11.3 vrtaClock ... 151
11.3.1 Method: vrtaClock() ... 151
11.3.2 Method: SetInterval() ... 152
11.3.3 Method: SetScale() ... 152
11.3.4 Method: Start() ... 153
11.3.5 Method: Stop() ... 153
11.3.6 Action: Interval ... 153
11.3.7 Action: Scale ... 154
11.3.8 Action: Start ... 154
11.3.9 Action: Stop ... 154
11.3.10 Event: Interval ... 154
11.3.11 Event: Scale ... 154
11.3.12 Event: Running ... 155

11.4 vrtaUpCounter ... 155
11.4.1 Method: vrtaUpCounter() ... 155

11.4.2 Method: Min() 156
11.4.3 Method: Max() 156
11.4.4 Method: Value() 157
11.4.5 Method: SetMin() 157
11.4.6 Method: SetMax() 158
11.4.7 Method: SetVal() 158
11.4.8 Method: Start() 159
11.4.9 Method: Stop() 159
11.4.10 Action: Minimum 160
11.4.11 Action: Maximum 160
11.4.12 Action: Set 160
11.4.13 Action: Start 160
11.4.14 Action: Stop 160
11.4.15 Action: Report 161
11.4.16 Event: Set 161

11.5 vrtaDownCounter 161
11.5.1 Method: vrtaDownCounter() 161
11.5.2 Method: Min() 162
11.5.3 Method: Max() 162
11.5.4 Method: Value() 163
11.5.5 Method: SetMin() 163
11.5.6 Method: SetMax() 164
11.5.7 Method: SetVal() 164
11.5.8 Method: Start() 165
11.5.9 Method: Stop() 165
11.5.10 Action: Minimum 166
11.5.11 Action: Maximum 166
11.5.12 Action: Set 166
11.5.13 Action: Start 166
11.5.14 Action: Stop 167
11.5.15 Action: Report 167
11.5.16 Event: Set 167

11.6 vrtaSensor 167
11.6.1 Method: vrtaSensor() 167
11.6.2 Method: GetMax() 168
11.6.3 Method: Value() 168
11.6.4 Method: SetMax() 169
11.6.5 Method: SetVal() 169
11.6.6 Action: Value 170
11.6.7 Action: Maximum 170
11.6.8 Event: Value 170
11.6.9 Event: Maximum 171

11.7 vrtaSensorToggleSwitch 171
11.7.1 Method: vrtaSensorToggleSwitch() 171
11.7.2 Method: Value() 171
11.7.3 Method: SetVal() 172
11.7.4 Action: Position 172
11.7.5 Event: Position 173

11.8 vrtaSensorMultiwaySwitch 173
11.8.1 Method: vrtaSensorMultiwaySwitch() 173
11.8.2 Method: GetMax() 173
11.8.3 Method: Value() 174
11.8.4 Method: SetMax() 174
11.8.5 Method: SetVal() 175
11.8.6 Action: Value 176

11.8.7 Action: Maximum....176
11.8.8 Event: Value....176
11.8.9 Event: Maximum....176

11.9 vrtaActuator....176
11.9.1 Method: vrtaActuator()....177
11.9.2 Method: GetMax()....177
11.9.3 Method: Value()....178
11.9.4 Method: SetMax()....178
11.9.5 Method: SetVal()....179
11.9.6 Action: Value....179
11.9.7 Action: Maximum....179
11.9.8 Event: Value....179
11.9.9 Event: Maximum....180

11.10 vrtaActuatorLight....180
11.10.1 Method: vrtaActuatorLight()....180
11.10.2 Method: Value()....180
11.10.3 Method: SetVal()....181
11.10.4 Action: Value....181
11.10.5 Event: Value....182

11.11 vrtaActuatorDimmableLight....182
11.11.1 Method: vrtaActuatorDimmableLight()....182
11.11.2 Method: GetMax()....183
11.11.3 Method: Value()....183
11.11.4 Method: SetMax()....183
11.11.5 Method: SetVal()....184
11.11.6 Action: Value....185
11.11.7 Action: Maximum....185
11.11.8 Event: Value....185
11.11.9 Event: Maximum....185

11.12 vrtaActuatorMultiColorLight....185
11.12.1 Method: vrtaActuatorMultiColorLight()....185
11.12.2 Method: GetMax()....186
11.12.3 Method: Value()....186
11.12.4 Method: SetMax()....187
11.12.5 Method: SetVal()....187
11.12.6 Action: Value....188
11.12.7 Action: Maximum....188
11.12.8 Event: Value....188
11.12.9 Event: Maximum....189

11.13 vrtaCompare....189
11.13.1 Method: vrtaCompare()....189
11.13.2 Method: GetMatch()....190
11.13.3 Method: SetMatch()....190
11.13.4 Method: IncrementMatch()....191
11.13.5 Method: SetVector()....191
11.13.6 Action: Match....192
11.13.7 Action: Vector....192
11.13.8 Event: Match....192

11.14 vrtaIO....192
11.14.1 Method: vrtaIO()....193
11.14.2 Method: SetValue()....193
11.14.3 Method: SetValues()....194
11.14.4 Method: GetValue()....194
11.14.5 Method: GetValues()....195

11.14.6 Action: Value ... 195
11.14.7 Action: Values ... 195
11.14.8 Action: GetValue ... 196
11.14.9 Action: GetValues ... 196
11.14.10 Event: Value ... 196
11.14.11 Event: Values ... 196

12 VRTA Communications Libraries ... 197

12.1 Languages Supported ... 198
12.1.1 .NET ... 198
12.1.2 Java ... 198
12.1.3 Ruby ... 198
12.1.4 Ruby_in_Bake ... 198
12.1.5 jRuby ... 199
12.1.6 Python ... 199
12.1.7 C++ ... 199

12.2 Controlling the VECU lifecycle ... 201
12.2.1 Aliases ... 201
12.2.2 Talking to vrtaServer ... 202
12.2.3 Starting a new VECU ... 204
12.2.4 Connecting to an existing VECU ... 210
12.2.5 Detaching from a VECU ... 213
12.2.6 Terminating a VECU ... 214
12.2.7 Finding existing instances of a VECU ... 215
12.2.8 Finding all loaded VECUs ... 216
12.2.9 Searching for VECUs on disk ... 218
12.2.10 Find files ... 220
12.2.11 Read file ... 222

12.3 Communication with VECUs ... 223
12.3.1 Get / change VECU state ... 223
12.3.2 Get device information ... 225
12.3.3 Get device instances ... 225

12.4 Communication with Devices ... 227
12.4.1 Device information ... 227
12.4.2 Actions ... 228
12.4.3 Events ... 229

12.5 Communication with Actions ... 231
12.5.1 Sending the action ... 231
12.5.2 Action parameters ... 232

12.6 Communication with Events ... 234
12.6.1 Polling the event ... 234
12.6.2 Event parameters ... 235

12.7 Hooking Events ... 237

13 Command Line Option Reference ... 240

13.1 Virtual ECU Executables ... 240
13.1.1 -alias=name (override the default alias) ... 240
13.1.2 -priority=<n> (set the priority) ... 240
13.1.3 -core=<n> (set the physical core) ... 240
13.1.4 -silent (select silent or GUI mode) ... 241
13.1.5 -noserver (do not contact vrtaServer) ... 241
13.1.6 -slave (select slave or autostart mode) ... 241

13.2 vrtaServer.exe 241
13.2.1 -install (install as a service) 241
13.2.2 -p<n> (specify the TCP port) 242
13.2.3 -silent (silent install or uninstall) 242
13.2.4 -standalone (run in standalone mode) 242
13.2.5 -start (start the vrtaServer service) 242
13.2.6 -stop (stop the vrtaServer service) 242
13.2.7 -uninstall (un-install as a service) 242

13.3 vrtaMonitor 242
13.3.1 Command Files 242

13.4 vrtaMonitor Global Options 243
13.4.1 -f<filename> (close when <filename> appears) 243
13.4.2 -k (terminate with specific error level) 243
13.4.3 -log=<file> (write to a log file) 243
13.4.4 -scripter=<name> (select a scripting engine) 243
13.4.5 -t<n> (close after <n> seconds) 243

13.5 vrtaMonitor Sequential Options 243
13.5.1 Virtual ECU Executable Name (auto-load) 243
13.5.2 -alias=<name> (connect to VECU) 243
13.5.3 -d (load but not start a Virtual ECU) 243
13.5.4 -g (load with a GUI) 244
13.5.5 -host=<hostname> (select a remote PC) 244
13.5.6 -mon=<dev>.<event> (monitor event) 244
13.5.7 –n (load without a GUI) 244
13.5.8 -p<n> (select TCP port) 244
13.5.9 -quit (terminate) 244
13.5.10 –r (load and start a VECU) 244
13.5.11 -script=<file> 244
13.5.12 -send=<dev>.<act> (send action) 244
13.5.13 -send=<dev>.<act>(<str>) (send action) 244
13.5.14 -start (start a VECU) 245
13.5.15 -wait=<n> (wait) 245

14 Glossary 246

15 Contact Information 253

1 Welcome to the RTA-OS Virtual Target

RTA-OS plus the Virtual Target (VRTA) is a complete environment for developing AUTOSAR OS applications that can run on either Windows or Linux machines.

Mostly you'll use it to prototype a new application before migrating it on to production hardware, but you will also find that it is a good tool for learning how to use RTA-OS to develop applications for embedded targets.

You needn't stop there though. Because RTA-OS is a complete and fast implementation of AUTOSAR, you can add inter-ECU communication using a standard networking solution such as CAN. You can write applications that sit on your CAN network as test or simulation units. You can remotely monitor the state and progress of your applications using the supplied vrtaMonitor program, or you can write your own monitoring applications that connect directly to the VECU's virtual devices. And of course the development turnaround time is tiny - just recompile and run. No downloading of hex files to an emulator. No programming Flash.

This document covers both the Windows and Linux flavors of VRTA. Both operate very similarly, but where there are differences between the two we use **VRTA-win** to mean the Windows-hosted version and **VRTA-ux** to mean the Linux version.

Because of the automotive heritage of RTA-OS, we use the term **VECU** to refer to the actual executable (e.g. the .exe file) that runs on the host PC and contains the RTA-OS, VRTA and application code.

The guide is structured as follows:

Chapter 1 introduces you to the Virtual ECU, covering what tools are provided, which standards are supported by the kernel and gives a brief overview of kernel features.

Chapter 2 Introduces Virtual ECUs.

Chapter 3 takes you through a tutorial that explains how to build, run and monitor a Virtual ECU.

Chapters 4 to 8 provide detailed information about how to work with a Virtual ECU, and its support programs including how to migrate from a Virtual ECU to an embedded target.

Chapters 9 to 11 provide a technical reference for creating a Virtual ECU and interacting with it from your application code. The sample devices that are automatically provided for you are also documented here.

Chapter 12 describes the communications library code that you can use to communicate with a VECU from an external tool.

Chapter 13 describes the command-line options that you might use.

Chapter 14 explains commonly used terms.

1.1 Related Documents

This guide does not tell you how to configure and use RTA-OS. That information is provided in the **RTA-OS User Guide**. A complete technical reference to RTA-OS can be found in the **RTA-OS Reference Guide**. Both of these documents can be found in the Documents folder of your RTA-OS installation. For a default installation, this will be C:\ETAS\RTA-OS\Documents.

Specific technical details about the implementation of RTA-OS in a virtual environment are contained in the associated VRTA Port Guide.

1.2 About You

You are a trained software engineer who wants to build real-time applications using a pre-emptive operating system. You should have knowledge of the C and C++ programming languages, including the compilation and linking of C and C++ code for PC-based applications with a PC-hosted development environment. Advanced features of RTA-OS on a Virtual ECU will require knowledge about Windows COM Component Object Model) applications.

You should also be familiar with common use of the Windows operating system, and Linux for the VRTA-ux flavor of VRTA. This includes installing software, selecting menu items, clicking buttons, navigating files and directories.

1.3 Document Notes

Please note that, due to ETAS' policy of continual product improvement, some of the screenshots reproduced in this manual may not exactly match the onscreen appearance of the GUI tool. GUI appearance may also be affected by your local Windows setup.

1.4 References

AUTOSAR (AUTomotive Open System ARchitecture) is an open and standardized automotive software architecture, jointly developed by automobile manufacturers, suppliers and tool developers. For details of the AUTOSAR standards, please refer to:

http://www.autosar.org.

The tools that you use to create, configure and build VECUs are designed to run on Microsoft Windows.

2 Introduction to the Virtual ECU

RTA-OS is commonly used in automotive environments where the term ECU (Electronic Control Unit) is used to refer to the target hardware on which the application runs. The ECU can be considered as a black box with inputs and outputs that perform a specific set of functions.

In a typical modern car, ECUs will be found in the engine compartment, the doors, the body etc. Many if not all will be running AUTOSAR applications.

The VRTA port does not need any real hardware, other than the host PC that you run it on. Instead, you create a Virtual ECU (VECU) in software that simulates the real-life devices such as switches or sensors that will be present in your physical ECU.

The VRTA code in the VECU provides services such as the interrupt controller, application control and device communication links as shown in Figure 1 Virtual ECU Architecture.

VRTA is able to simulate both single and multicore microcontrollers. You can choose how many cores VRTA simulates.

VRTA ships with programs called **vrtaMonitor** and (confusingly) **vrta**.

vrtaMonitor is a GUI application and **vrta** is a command-line tool. Both can be used to monitor and manage VECU on local or remote host PCs.

We provide software to help you to build your own applications that can interact with VECUs, including:

- C++
- C#
- Ruby
- Java

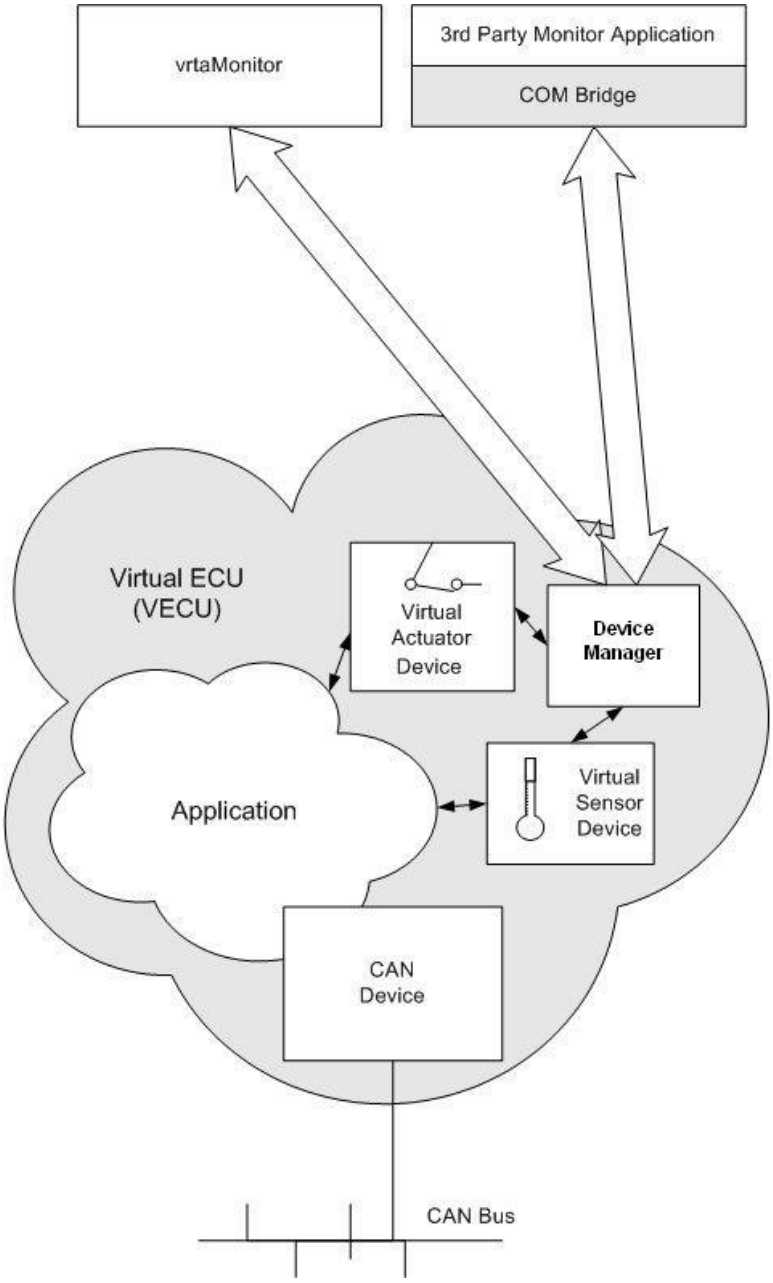


Figure 1 Virtual ECU Architecture

2.1.1 What do I need?

The tools that you use to create, configure and build VECUs are designed to run on Microsoft Windows.

2.1.2 VRTA-win

VECUs built using VRTA-win run under Microsoft Windows.

The actual performance of a VECU will clearly be dependent on the power of the processor, but you will find that a modern PC is capable of running a typical AUTOSAR OS application many times faster than a typical embedded target.

You will need a Windows C++ Compiler to generate Virtual ECUs (and optionally the associated debugger to debug it). VECUs can be built using the MinGW or Visual Studio compilers. You choose which one you want to use by selecting the appropriate compiler name as the target variant in the project configuration.

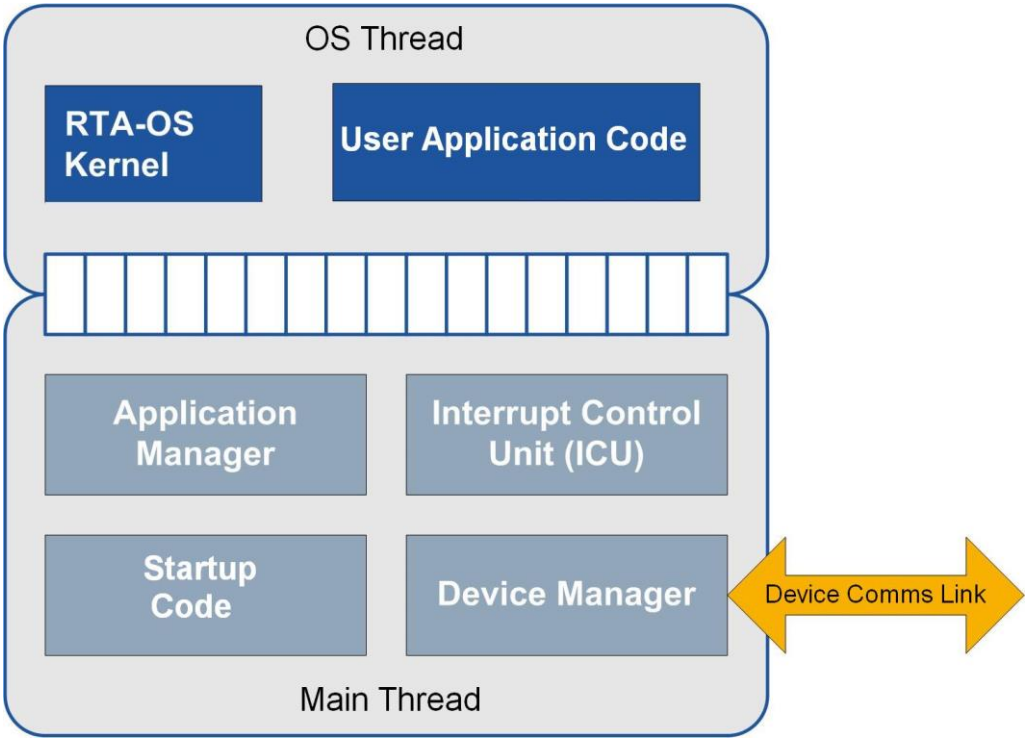
2.1.3 VRTA-ux

VECUs built using VRTA-ux run on a Linux host machine. The RTA-OS library build process runs on a Windows PC as normal, but the generated VECUs run on an appropriate Linux distribution.

VRTA-ux supports version 2 of Microsoft's Windows Subsystem for Linux (WSL2) running on Windows 10. RTA-OS uses the gcc toolchain in the WSL2 instance to perform the build steps, so you may need to install this first.

VECUs built using WSL2 are generally portable to other x86 or x64 Linux distributions, such as Ubuntu, simply by copying the executable file.

2.2 What is inside a VECU?



A VECU contains your application code (tasks, ISRs etc), virtual devices and communication code.

For **VRTA-win** much of the VRTA code is supplied in a DLL file called vrtaVM.dll. A VECU gets dynamically linked to this DLL during initialization.

> VRTA-win applications must be able to find vrtaVM.dll on the Windows DLL search path. It is highly recommended that you add <installdir>\Bin to your Windows PATH environment variable to ensure that every Virtual ECU application can find the Virtual Machine.

For **VRTA-ux**, no supporting DLL (shared object file) is needed – all of the code gets built into the RTAOS library file that gets linked into your executable file.

The VECU has a number of components as explained in the following sub-sections.

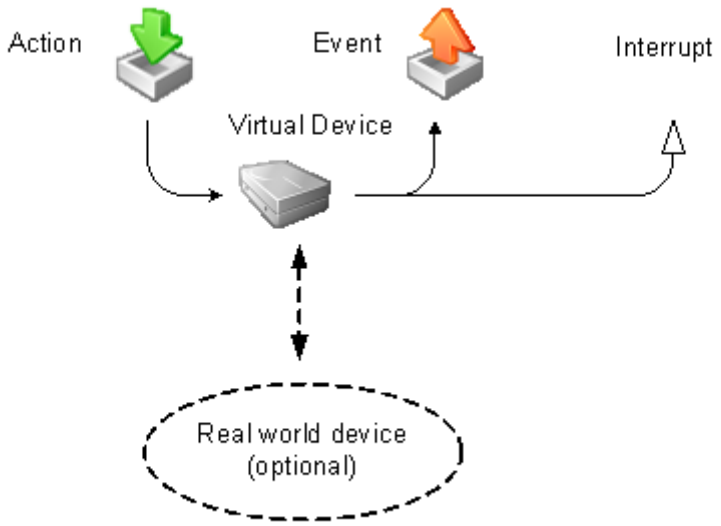
2.2.1 Application Manager

The Application Manager is a device within the VECU that manages the state of the overall application. It is responsible for controlling the thread(s) in which your application code runs[1]. On VRTA-ux, it can also control the physical core on which the OS thread runs.

The Application Manager can be controlled via the Device Communication Link to cause the OS thread to be started, paused, resumed, reset or terminated. It can also be interrogated to find out the state of the VECU.

2.2.2 Device Manager

VECUs use "virtual devices" to get things done. Virtual Devices have actions and events. You can tell a device to perform a particular action, and it can inform you of some change in state by raising an event or interrupt.



A virtual device can represent something simple like a switch or LED, or it can represent a complex component such as a USB-based CAN controller.

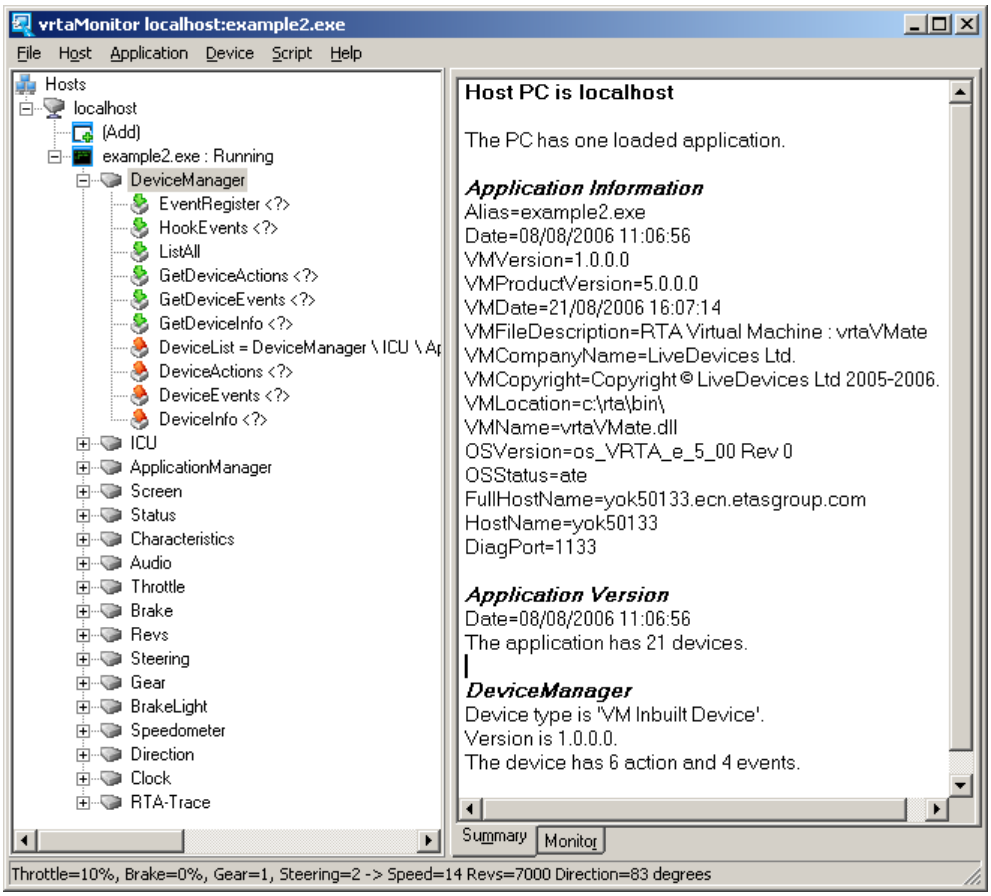
The device can either pretend to be a physical device, or it can behave as a controller/wrapper around a real physical piece of hardware.

[1] Application code runs in its own thread, not the main OS thread. Sometimes we call this the 'OS' thread.

The VECU contains the three standard devices: the DeviceManager[2], the Interrupt Control Unit[3] and the ApplicationManager.

The DeviceManager coordinates all devices in the VECU. Each device registers with the DeviceManager during initialization. The DeviceManager can then be queried to find what devices exist, what actions/events they support and the data that can be passed to and from the device.

Devices are not only available within the VECU: the DeviceManager includes a Device Communication Interface that allows external applications, such as `vrtaMonitor` (see Chapter 6) to inspect the state of the VECU's devices.



2.2.3 Interrupt Control Unit

The Interrupt Control Unit (ICU) is a device within the VECU that simulates multi-level interrupts in the Virtual ECU.

---

[2] Yes, the Device Manager is itself also a device. You query it as device zero to find out what other devices are present.

[3] On a multicore VECU there will be one Interrupt Control Unit per virtual core.

The **VRTA-win** ICU supports 32 different interrupt vectors (1 to 32) and 33 interrupt priorities (0[4] to 32). Each interrupt vector can be assigned a priority 1 to 32.

The **VRTA-ux** ICU supports 30 different interrupt vectors (1 to 30) and 31 interrupt priorities (0 to 31). Each interrupt vector can be assigned a priority 1 to 31.

The ICU maintains a current Interrupt Priority Level (IPL). An interrupt that is ready but has a priority < = the current IPL remains pending until the IPL drops below the assigned priority, at which point it can run.

When an interrupt is handled, the IPL is raised to match the interrupt's assigned priority. When the interrupt handler completes, the IPL is taken back to the value that was in effect when the interrupt was taken. This is the same behavior that would be expected by a traditional embedded ECU.

In a multi-core VECU each core has its own independent ICU. This allows each core to have independent interrupt priorities and masks. All ICUs share the same interrupt vector table, and when you raise an interrupt you can configure which core gets notified.

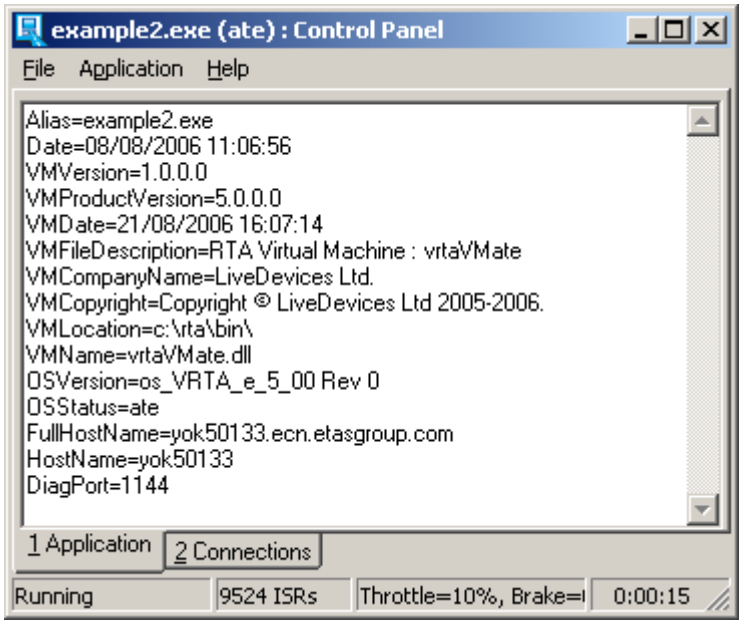
On **VRTA-win**, each interrupt vector can be masked. A masked interrupt can still become pending, but its interrupt handler will not run unless the vector is unmasked.

After reset, all interrupts are masked.

2.2.4 Embedded GUI (VRTA-win only)

When it runs, your VECU probably doesn't show much other than a boring black empty console window. The VECU can optionally display a simple GUI window that will give you a bit of confidence that the application is actually alive.

[4] Interrupt priority zero means 'no interrupt'.



You can perform some simple operations such as pause/resume/reset from the menu of the GUI. For more complex options, just select menu option Application/Monitor to launch `vrtaMonitor`.

2.3 Managing Virtual ECUs

You can run several VECUs on the same host PC at the same time. This causes something of a problem for external monitor programs, because they need a way to find out what VECUs are running, and how to connect to them.

VRTA is supplied with a program called **vrtaServer** that is designed to be started when the host PC boots, and sits quietly in the background providing a match-making service between VECUs and monitor programs.

Whenever a VECU starts running, it registers with vrtaServer and tells it what communication port its Device Comms Link is listening on. Monitor programs can then talk to vrtaServer to find out what VECUs exist, and how to communicate with them. **vrtaServer** can also be asked by a monitor program to load and start a VECU from its disk image.

> ***vrtaServer** must be running in order for a monitor program to be able to connect to a VECU.*

One useful benefit of having such a server is that monitor programs can also attach to servers and VECUs that are on remote PCs. The monitor has all of the same features that are available when used on a local machine, including the ability to reset, terminate and load VECUs.

2.3.1 vrtaServer.exe: VRTA-win and VRTA-ux (WSL2)

The `vrtaServer.exe` program can be found in the <installdir>\bin directory.

You can start this manually using the following command:

```
vrtaServer.exe
```

If you add the option `-standalone` then it will show a small status GUI, otherwise you will only see a small entry in the Windows system tray.

It can be used to manage both Windows VECUs and WSL VECUs running on the host PC.

2.3.2 vrtaServerCli.exe: VRTA-win and VRTA-ux (WSL2)

The `vrtaServerCli.exe` program can be found in the <installdir>\bin directory.

You can start this manually using the following command:

```
vrtaServerCli.exe
```

This is an equivalent program to `vrtaServer.exe`, but runs in a command window and shows a bit more diagnostic output.

It can be used to manage both Windows VECUs and WSL VECUs running on the host PC.

2.3.3 vrtaServer: VRTA-ux

For VRTA-ux on Linux, the `vrtaServer` executable has to be installed on the remote Linux host PC and either started manually or set to auto start. It should run with supervisor privileges because it needs to be able to launch VECUs and control their run-time characteristics.

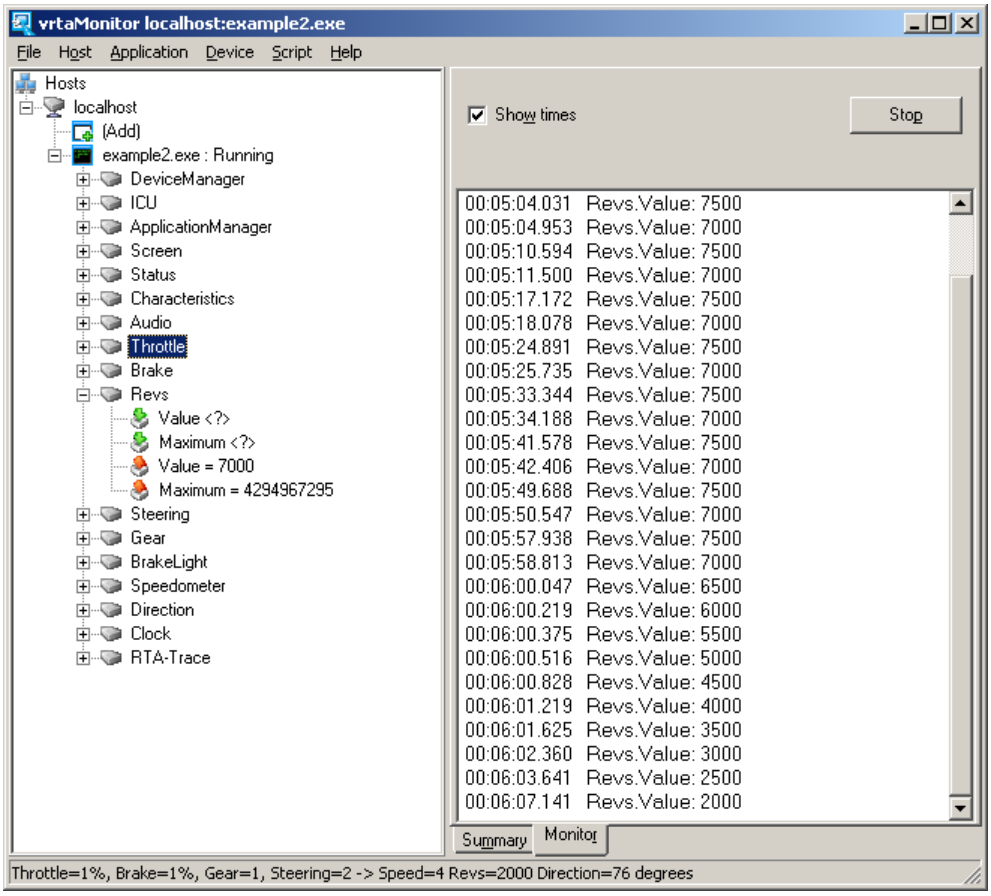
2.3.4 vrtaServerWSL: VRTA-ux

This is similar to vrtaServer on Linux but `vrtaServerWSL` is better optimized for running in WSL because there are some communication quirks that exist between Windows and WSL – particularly with a VPN.

Note that if you use this rather than `vrtaServer.exe` or `vrtaServerCli.exe` then you will not be able to run VECU-win VECUs at the same time as VRTA-ux VECUs.

2.4 Interacting with a Virtual ECU

The `vrtaMonitor` program is the quickest way to interact with a Virtual ECU. All VECUs have a Device Communication Link to which the monitor can connect, so no special action is needed when building your application. The monitor allows you to send actions and view events on local and remote PCs. It is discussed in detail in Chapter 6.



The command-line `vrta` program can do many of the same tasks as `vrtaMonitor`, and is intended to be used for automated testing tasks. It is described in Chapter 7.

You can also use the debugger that comes with your compiler toolchain to debug a VECU at a line-by-line level. Simply ensure that your compile and link options are set correctly, and then load the VECU into the debugger.

2.5 Installing VRTA-win

If you have installed VRTA-win on the default path then all VRTA-win applications such as vrtaMonitor.exe, vrta.exe and vrtaServer.exe can be found at C:\ETAS\RTA-OS\Bin.

vrtaServer must be running on the PC in order for other applications **vrtaMonitor** or **vrta** to communicate with VECUs.

2.6 Installing VRTA-ux

If you have installed VRTA-ux on the default path then vrtaMonitor.exe and vrta.exe can be found at C:\ETAS\RTA-OS\Bin.

The other Linux-hosted VRTA-ux applications such as vrtaServer are found at C:\ETAS\RTA-OS\Targets\VRTA-ux_x.x.x\linux. These applications have to be installed on to the Linux PC.

You may wish to arrange that vrtaServer starts automatically when your Linux host PC reboots. There are different ways to do this, so we leave the choice to you.

2.7 Windows Notes

Although *VRTA* tries very hard to simulate the behavior of a real ECU, ultimately Virtual ECUs are running under Windows alongside other applications. This section contains notes about Windows related behavior that developers may find useful.

2.7.1 Real-Time Behavior

When an embedded application runs on a real ECU the application is the only code using the ECU's processor. As a result the real-time behavior is predictable. However an application running in a Virtual ECU has to share the processor with other applications and Windows itself. As a result it is not possible to completely predict the real-time behavior of applications running in Virtual ECUs.

Despite this our experience has shown that applications running in Virtual ECUs exhibit very close to real-time behavior. This is due to the very fast processor speeds of Windows PCs. For example, consider an embedded application that needs to read and then process input from a sensor every 5 milliseconds. On a real ECU it might take almost 5 milliseconds to carry out this activity. However on a Windows PC it may only take 0.5 milliseconds. Thus even if Windows assigns the processor to another application for 3 milliseconds the Virtual ECU application can still carry out the necessary processing in the 5 milliseconds allowed.

If your application is not behaving as you expect for timing reasons you can try the following:

- Shutdown other Windows applications so that more of the processor's time can be dedicated to the Virtual ECU.
- Increase the process priority of the Virtual ECU – see the `-priority=<n>` command line options.

2.7.2 Calling the C/C++ Runtime and Windows

In order to simulate interrupts in a Virtual ECU, VRTA has to asynchronously manipulate the stack of the OS thread (the thread that calls `OS_MAIN()`). Few C/C++ runtime functions or Windows API functions can cope with the stack being changed asynchronously. Therefore if the OS thread needs to call a C/C++ runtime function (including `printf()`) or a Windows API function it must make the call in an uninterruptible section. See the `vrtaEnterUninterruptibleSection()` and `vrtaLeaveUninterruptibleSection()` calls.

2.7.3 vrtaVM.dll Location

When a Virtual ECU is started it tries to load the VRTA DLL (`vrtaVM.dll`). The VECU first tries to load the VRTA DLL using the normal DLL search rules. That is, it searches the following locations in the specified order:

1. The directory containing the VECU.
2. The current directory.
3. The 32-bit Windows system directory.
4. The Windows directory.
5. The directories listed in the `PATH` environment variable.

If the VECU fails to find the vrtaVM.dll then the application will fail to start. You can fix this by making sure that vrtaVM.dll can be found on the DLL search path.

> *It is highly recommended that you add the installation directory for RTAOS executables (by default C:\ETAS\RTA-OS\Bin to your Windows PATH environment variable to ensure that vrtaVM.dll can always be found.*

2.8 Linux Notes

Although the Virtual ECU tries very hard to simulate the behavior of a real ECU, ultimately a Virtual ECU is running under Linux alongside other applications. This section contains notes about Linux related behavior that may be useful.

2.8.1 Real-Time Behavior

When an embedded application runs on a real ECU the application is the only code using the ECU's processor. As a result the real-time behavior is predictable. However an application running in a Virtual ECU has to share the processor with other applications and Linux itself. Consequently it is not possible to completely predict the real-time behavior of applications running in Virtual ECUs.

Despite this our experience has shown that applications running in Virtual ECUs exhibit very close to real-time behavior.

If your application is not behaving as well as you expect you can try the following:

- Kill other Linux applications so that more of the processor's time can be dedicated to the Virtual ECU.
- Increase the process priority of the Virtual ECU – see the `-priority=<n>` command line option.
- Force all OS threads to run on a specific physical core – see the `-core=<n>` command line options.

VECUs are tuned by default to be as responsive as possible to external stimuli. This means that even when in the AUTOSAR idle callback, the VECU will be spinning waiting for work. For individual VECUs this is probably what you want. However if you are running several VECUs simultaneously or are running the VECU as part of a larger simulation system then you may want to reduce the CPU utilization when idling. You can use the `-fair` option when starting the VECU to do this.

2.9 Possible Problem Areas

The following is a short list of problems that you might encounter when you start developing VECU code:

2.9.1 General

- You cannot connect to a VECU using vrtaMonitor. Make sure that vrtaServer is running.
- Your application seems to lock up. Provide `StackFaultHook()` and `ShutdownHook()` handlers and print an error to the screen if they occur. (You can use `printf()` here.)

2.9.2 VRTA-win

- You must not make any non-VRTA or non-OS API calls from your OS thread if virtual interrupts could occur. This includes 'quick hacks' such as using `printf()` to display the content of some piece of data. All non-VRTA and non-OS API calls must be protected by an uninterruptible section (see section 9.4.4). See section 2.7.2 for more details.

- Nothing appears to be happening? Check that the vrtaVM.dll is on the Windows DLL search path

2.9.3 VRTA-ux

- You get timeouts or similar failures when building your VECU. Ensure that the WSL instance is still running. It may be necessary to shut it down (`wsl --shutdown`) and restart it.

3 Tutorial

The best way to obtain an understanding of RTA-OS and VRTA is to get something running, so let's set aside a couple of hours and see what we can come up with.

3.1 Prerequisites

For this tutorial we will need a copy of RTA-OS and VRTA installed and licensed on your computer. We also need a C++ compiler for VRTA-win.

3.1.1.1 RTA-OS

If you have followed the instructions in the *Getting Started Guide* then you will have:

- Installed RTA-OS and VRTA;
- Obtained and installed your license; and
- Built and run the sample applications.

In this tutorial we will assume that you have installed RTA-OS to `C:\ETAS\RTA-OS`, so the path to the executables is `C:\ETAS\RTA-OS\Bin`.

The files that are specific to VRTA-win are found at `C:\ETAS\RTA-OS\Targets\VRTA_n.n.n`.

The files that are specific to VRTA-ux are found at C`:\ETAS\RTA-OS\Targets\VRTA-ux_n.n.n`.

3.1.1.2 Compiler

This tutorial will use the MinGW compiler for VRTA-win, and the Linux PC's gcc compiler for VRTA-ux.

For VRTA-win, the MinGW compiler tools must be on the system PATH, otherwise the RTA-OS tools will not be able to find it.

For VRTA-ux, you must have a WSL2 available and the gcc tools must be present.

3.2 Creating your first Virtual ECU: Part1

Let's go for the traditional 'Hello World' starter application - with an AUTOSAR OS twist. The following steps are required:

1. Configure RTA-OS
2. Build the RTA-OS library
3. Write the application code
4. Write the VECU code
5. Build the Virtual ECU

3.2.1 Configuring RTA-OS

Start **RTAOSCfg**, the RTA-OS[5] GUI-based configuration tool.

You are now looking at an empty gray window which we can brighten up a bit by selecting menu option **File → New Project**. You can accept the default project values.

3.2.1.1 Basic Configuration

We need to add enough basic RTA-OS to generate an OS kernel.

In **OS Configuration → General** on the **General** tab:

Item	Setting
Scalability Class	SC1
Status	STANDARD
Enable Stack Monitoring	FALSE
Number of OS Cores	1

In **OS Configuration → General** on the **Hooks** tab click **Clear All**.

In **OS Configuration → General** on the **Error Hook** tab click **Clear All**.

In **OS Configuration → General** on the **Timing** tab set **Enable Time Monitoring** to **FALSE**.

In **OS Configuration → General Target**:

Item	Setting
Target → Name	VRTA or VRTA-ux
Target → Variant	MinGW / WSL_64bit
Clock → Instruction rate	1000000 (1MHz)
Clock → Stopwatch	1000 (1kHz)

This has configured the target-specific aspects we need for RTA-OS this tutorial.

[5] For a default installation of RTA-OS is located in C:\ETAS\RTA-OS\Bin.

In **OS Configuration → General Optimizations** on the **AUTOSAR** tab set **Use RES_SCHEDULER** to FALSE.

In **OS Configuration → Application modes** create a mode called OSDEFAULTAPPMODE.

These settings create a basic, empty, application for VRTA. Now give the project a name and save it.

Select menu **File → Save As** and save the project as something like `C:\Play\Tutorial1\Tutorial1.rtaos`.

If you click on **Check Now** it should report no errors. If it doesn't then you'll need to check that you've done the preceding steps correctly.

3.2.1.2 Hooks

We'll use AUTOSAR OS's `StartupHook()` and `ShutdownHook()` to say 'Hello World! ' and 'Goodbye World! ' so these need to be enabled in the RTA-OS configuration.

Go back to **OS Configuration → General** and the **Hooks** tab and make the following changes:

Item	Setting
Call Startup Hook	TRUE
Call Shutdown Hook	TRUE

3.2.2 Building RTA-OS

Select the **Builder → Setup** workspace and the **Samples** tab and make the following changes.

Item	Setting
Sample Header Files	All checked

You can now build the RTA-OS kernel library. Go to **Builder → Build** and click **Build Now**. Building will create the following RTA-OS files in the same directory as your RTA-OS file:

File name	Contents
Os.h	The main include file for the OS.
Os_Cfg.h	Declarations of the objects you have configured. This is included by Os.h.
Os_MemMap.h	AUTOSAR memory mapping configuration used by RTA-OS to merge with the system-wide MemMap.h file.
RTAOS.<lib>	The RTA-OS library for your application. The extension `<lib>` depends on your target.
RTAOS.<lib>.sig	A signature file for the library for your application. This is used by RTA-OS to work out which parts of the kernel library need to be rebuilt if the configuration has changed.
<projectname>.log	A log file that contains a copy of the text that the tool and compiler sent to the screen during the build process.

You will also see a set of files whose names are prefixed vrta that have been created to support VRTA.

3.2.3 Writing Application Code

You now need to implement your application. This example shows the code you need.

```
#include <Os.h>
OS_MAIN(){
  StartOS(OSDEFAULTAPPMODE);
}
FUNC(boolean,OS_CALLOUT_CODE) Os_Cbk_Idle(void){
  ShutdownOS(E_OK);
  return TRUE; /* Never reached */
}
FUNC(void,OS_CALLOUT_CODE) StartupHook(void){
  vrtaEnterUninterruptibleSection();
  printf("Hello World!\n");
  vrtaLeaveUninterruptibleSection();
}
FUNC(void,OS_CALLOUT_CODE) ShutdownHook(StatusType s){
  vrtaEnterUninterruptibleSection();
  printf("Goodbye World!\n");
  vrtaLeaveUninterruptibleSection();
  vrtaTerminate();
}
```

Save this as a file called `Application.c` in the same directory as `Tutorial1.rtaos`.

3.2.4 Writing VECU Code

Any AUTOSAR OS-based VECU that you build will need to initialize your virtual devices. VRTA doesn't know how to initialize your devices, but it knows *when* it needs to do this, so it makes a callback to your code. The callback is named `InitializeDevices()` and it is a C++ function.

InitializeDevices() must always be present in any VECU application.

This is an example 'dummy' implementation.

```
#include "vrtaCore.h"
void InitializeDevices(void){
  // Placeholder
}
```

Save this as a file called `VirtualDevices.cpp` in the same directory as `Tutorial1.rtaos`.

All that remains is build the Virtual ECU:

3.2.5 Building the Virtual ECU using VRTA-win

The steps that are needed are:

- Compile the `Application.c` file and the `VirtualDevices.cpp` file. Make sure that the working directory (.) and `Samples\Includes` are on the include path.

- Link two object files with `RTAOS.lib` and your compiler's Windows and User32 libraries to create an executable called `RTAOS.exe`.

The following batch file shows how this can be done with the MinGW compiler:

```
set CC=gcc.exe
set CPP=g++.exe
set LNK=g++.exe

rem Set options
set COPTS=-c -g -I. -ISamples\Includes
set CPPOPTS==%COPTS%

rem Compile application and virtual device code
%CC% %COPTS% Application.c
%CPP% %CPPOPTS% VirtualDevices.cpp

rem Link the application
%LNK% *.o RTAOS.lib -lwinmm -lws2_32 -o" Tutorial1.exe"
```

3.2.6 Building the Virtual ECU using VRTA-ux

The steps that are needed are:

- Compile the `Application.c` file, the `VirtualDevices.cpp` file. Make sure that the working directory (`.`) and `Samples/Includes` are on the include path.
- Link two object files with `RTAOS.a` and your compiler's Linux and libraries to create an executable called `Tutorial1.vrta_vecu`.

The following batch file shows how this can be done:

```
set CC=wsl.exe gcc
set CPP=wsl.exe gcc
set LNK=wsl.exe g++
set AR=wsl.exe ar

rem Set Options
set COPTS=-c -g -pthread -O2 -I. -I./Samples/Includes -m64
set CPPOPTS=%COPTS%
PATH=%windir%\Sysnative;%PATH%

rem Compile application and virtual device code
%CC% %COPTS% Application.c
%CPP% %CPPOPTS% VirtualDevices.cpp

rem Link the application
@echo -o Tutorial1.vrta_ecu > linkeropts
for %%f in (*.o) do echo "%%f" >> linkeropts
for %%f in (*.a) do echo "%%f" >> linkeropts
@echo -Wl,-Map -Wl,Tutorial1.map >> linkeropts
%LNK% @linkeropts -lrt -lpthread -m64
```

3.2.7 Running the VRTA-win Virtual ECU

If everything has compiled and linked correctly then you can run your first Virtual ECU!

Simply open a Windows command prompt ("DOS box") and run the program `Tutorial1.exe`. You will see it run and print "Hello World" followed by "Goodbye World".

If you are having problems, then you can find a working version of this part of the tutorial by generating the sample applications for VRTA-win using the command line:

```
RTAOSGen --samples:[Applications] --target:VRTA
```

The working version will then be found in the directory `Samples\Applications\Tutorial_Part1`. See the *Getting Started Guide for* further information about generating sample applications.

3.2.8 Running the VRTA-ux Virtual ECU

If everything has compiled and linked correctly then you can run your first Virtual ECU!

Simply open a Windows command prompt ("DOS box") at the directory that contains your project. Enter the line below.

```
wsl ./Tutorial1.vrta_ecu
```

You will see it run and print "Hello World" followed by "Goodbye World".

As an alternative you can also switch into the WSL shell and execute `./Tutorial1.vrta_ecu` there.

If you are having problems, then you can find a working version of this part of the tutorial by generating the sample applications for VRTA-win using the command line:

```
RTAOSGen --samples:[Applications] --target:VRTA-ux
```

The working version will then be found in the directory `Samples\Applications\Tutorial_Part1`. See the *Getting Started Guide for* further information about generating sample applications.

3.3 Embedding a VECU inside a DLL or executable: vrtaInProcessAPI .

In normal operation, a VECU is created as a stand-alone executable that runs independently and is controlled, if necessary, from external applications using its diagnostic interface.

VRTA support this via the **vrtaInProcessAPI** interface.

The file `vrtaInProcessAPI.h` gets created as part of the OS library generation. This contains the declaration of the APIs that can be used if you want to embed a VECU into your own program or DLL.

As an example, this code can be compiled and linked with the OS library to create a new-standalone executable that will operate as a VECU but can also contain your own code in **main**.

```
//////////////////////////////////////////////////////////////////////
///////////////// In-process VECU demo
//////////////////////////////////////////////////////////////////////

#include "vrtaInProcessAPI.h"
#include "stdio.h"
#include "conio.h"

int main(int argc, char *argv[]) {
// Modes:
//
//  VECU_RUN_IN_THREAD:
//    Load and run the VECU in a separate thread.
//    VRTALoad() will return once the VECU is loaded.
//    Otherwise VRTALoad() returns only when the VECU terminates
//
//  VECU_SHOW_GUI:
//    Show the VECU's internal GUI. Ignored when running in a thread.
//
//  VECU_WAIT_FOR_START:
//    Load but do not start the VECU.
//    The VECU can be started externally or by calling VRTAStart()
//
//  VECU_NO_SERVER:
//    The VECU will not attempt to contact vrtaServer.
//
//  VECU_DIRECT_TICKED:
//    The clock will not start. VRTATick() is called to inject ticks.
//

  printf("Starting VECU\n");

  VRTALoad(argv[0], VECU_WAIT_FOR_START |
                    VECU_RUN_IN_THREAD |
                    VECU_DIRECT_TICKED);

  printf("VECU Loaded\n");
  VRTAStart();
  printf("VECU Started\n");

  // e.g. VRTAAutoTick();    // Switch to free-running clock
  // e.g. VRTADirectTick();  // Switch to manually ticked clock

  while(!VRTAIsAppFinished()) {
```

```
    if (kbhit()) {
      printf("\nStopping...\n");
      VRTAStop();
      while(!VRTAIsAppFinished()) {
        printf("."); VRTASleep(125);
      }
      break;
    }

    VRTATick();
    printf("\r\\"); VRTASleep(125);
    printf("\r|");  VRTASleep(125);
    printf("\r/");  VRTASleep(125);
    printf("\r-");  VRTASleep(125);
  }

  printf("\nDone\n");
  return 0;
}
```

Before we look at making the Virtual ECU more interesting, it is useful to look at how virtual devices are provided.

Some note:

- **VRTALoad** must be called first. Pass it the name of the application and then one or more of the startup flags described above.
- **VRTAStart()** starts the VECU running. It is only needed if the load uses `VECU_WAIT_FOR_START`. If it is not called, then external programs such as **vrtaMonitor()** can start the VECU via the diagnostic interface.
- **VRTAStop()** can be used to stop the VECU. It cannot be restarted.
- **VRTAIsAppFinished()** can be used to determine if the VECU has stopped running – either because it has completed or because it has been terminated by an external program.
- **VRTAAutoTick()** can be used to tell the VECU's clock to tick automatically – usually every millisecond.
- **VRTADirectTick()** can be used to tell the VECU's clock to stop ticking automatically. Your code can call **VRTATick()** to inject single ticks.
- **VRTASleep(ms)** can be used to pause for a specified number of milliseconds.
- **VRTAFireISR(vec, core)** can be called to inject an interrupt on a named vector to a specific OS core.

3.4 Adding Devices

Before we look at making the Virtual ECU more interesting, it is useful to look at how virtual devices are provided.

In the same way that a real ECU contains hardware such as counters, comparators, switches and the like, a Virtual ECU will also need some devices to work with. Virtual devices are software functions that can declare their name, type, actions and events to the VRTA Device Manager, and then respond to action requests, or event queries.

Virtual devices must be declared statically so that they can become known to the Device Manager before entering the main program. Do not attempt to create devices dynamically[6].

All of the repetitious plumbing code needed to create a device can be handled for you in a C++ class called `vrtaDevice` that is defined in the file `vrtaDevice.h`. This is generated when you build the RTA-OS library. You can use this as a base class from which to create your own devices.

When the RTA-OS is built, a number of sample devices are automatically generated for you and placed in the library. You can access these devices though the header file `vrtaSampleDevices.h`. These sample devices are automatically included into your VECU if you use them and are documented in detail in Chapter 11.

The following sections show you how you might use these devices in application code.

3.4.1 Clocks

The `vrtaClock` device represents a clock source in a real ECU. It can be thought of as the oscillator.

It ticks independently of the rest of the application at up to 1000 times per second. The code example below shows you how to declare a clock named “Clock” that ticks every 5ms:

```
#define MSECS_PER_TICK (5)
vrtaClock Clock("Clock", MSECS_PER_TICK);
```

NOTE: All clocks derive from the SystemClock device. This device has actions that can be used to stop/start/scale all the clocks at the same time. It normally ticks every 1ms, but can be set into IdleTicked or DirectedTick modes for enhanced test control.

[6] You wouldn’t expect a hardware counter to suddenly come into existence on a real ECU either!

On its own, the clock source is not of much use. You can start it, stop it and change its interval, but it does not maintain a value or raise any interrupts. We need to connect a counter to this clock source. Two types of counter are provided:

1. up-counting
2. down-counting

These lines of code declare two such counters attached to the clock source:

```
vrtaUpCounter CountUp("CountUp", Clock);
vrtaDownCounter CountDown("CountDown", Clock);
```

By default, a counter increments or decrements its current value within the range zero to $2^{32}-1$. You can reduce the range of the count using the `SetMin()` and `SetMax()` methods. The normal place to do this is in the `InitializeDevices()` function as shown below. `InitializeDevices()` is also a good place to enable the counter.

```
void InitializeDevices(void){

  /* CountUp goes 0...999 then back to 0 */
  CountUp.SetMax(999);
  CountUp.Start();

  /* CountDown goes 1999...1000 then back to 1999 */
  CountDown.SetValue(1999);
  CountDown.SetMin(1000);
  CountDown.SetMax(1999);
  CountDown.Start();
}
```

You can read the current value of a counter using `counter.GetValue()`. But what we'd really like to do is to raise an interrupt when the counter reaches a certain value. The `vrtaCompare` virtual device is provided to do exactly this job.

When you declare a compare device you can attach it to a counter and specify an interrupt that fires when the value of the counter reaches a specific value. The next code example shows two comparators connected to the same counter. The first one raises interrupt 1 when the counter reaches value 5. The second raises interrupt 2 when the counter value reaches 15.

```
vrtaCompare Compare5("Compare5",CountUp,5,1);
vrtaCompare Compare15("Compare15",CountUp,15,2);
```

Hopefully it is clear from this that you can very easily construct a chain of timing elements of arbitrary complexity by simple combinations of clock, counter and compare devices.

> *Note that `vrtaClock` sources run independently of the application. If you use the `ApplicationManager` to pause the application thread, the clocks continue to run and any counter/compare devices that are attached can continue to raise interrupts that will be serviced once the application thread resumes. You can pause the SystemClock itself if that is necessary to give you the behavior you need.*

3.4.2 Sensors

Sample sensor devices are provided which are intended to represent inputs to an ECU. Sensors in the real world could include switches, thermocouples, pressure monitors and so on.

Sensors have a minimum value, a maximum value and a current value in the same way as a counter. A basic `vrtaSensor` device has values range from zero to $2^{32}-1$, but you can change the upper limit for this using the `SetMax()` method.

Two derivatives of `vrtaSensor` are:

- `vrtaSensorToggleSwitch` which can have a position value zero or one.
- `vrtaSensorMultiwaySwitch` which has multiple positions, with an upper limit specified in its declaration.

This code example shows how these three types of sensor are declared.

```
vrtaSensor Throttle("Throttle");
vrtaSensorToggleSwitch EjectSwitch("EjectSwitch");
vrtaSensorMultiwaySwitch Gear("Gear", 5); //0 to 5
```

The values of a sensor can be changed remotely using vrtaMonitor. You can read the value of the sensor directly from your application using the `GetValue()` method. Alternatively you can attach a `vrtaCompare` device to the sensor and raise an event when a certain value is set as shown in the next code example which raises interrupt 3 when the EjectSwitch position is set to 1.

```
vrtaCompare Eject("Eject",EjectSwitch,1,3);
```

3.4.3 Actuators

Sample actuators represent outputs from the Virtual ECU. The base `vrtaActuator` device can be set to a value from zero to $2^{32}-1$. Each actuator raises an event when

its value changes, so external programs such as vrtaMonitor can detect changes in the VECU's output.

As with sensors, you can attach a `vrtaCompare` device to an actuator so that you can raise an interrupt when a set value is reached[7].

Three derivatives of vrtaActuator are:

- `vrtaActuatorLight` with values zero and one.
- `vrtaActuatorDimmableLight` with levels zero to n.
- `vrtaActuatorMultiColorLight` ('colors' zero to n).

The code example below shows how these three types of sensor are declared.

```
vrtaActuator Speedometer("Speedometer");
vrtaActuatorLight BrakeLight("BrakeLight");
vrtaActuatorDimmableLight InteriorLight("InteriorLight",20);
vrtaActuatorMultiColorLight FuelIndicator("FuelIndicator",4);
```

3.4.4 IO

The `vrtaIO` virtual device is a more general purpose component. It simulates a block of 32-bit values in an ECU's IO or memory space. You (or an external monitor) can set or inspect values in the individual elements. An event is raised when a value is changed, but there is no ability to generate an interrupt based on the value.

3.4.5 Custom Devices

The sample devices which are generated automatically by RTA-OS are a good starting point when creating a VECU, but you will want to create your own devices to reflect your own environment. Thanks to C++ inheritance this is a straightforward process that is shown in Chapter 4.

[7] By our definition of an actuator, it is an output device so you wouldn't expect it to be able to raise an interrupt: that is really the role of a sensor attached to it. Nevertheless, you may find this capability useful in certain cases.

3.5 Creating your first Virtual ECU: Part2

Reopen the Tutorial1.rtaos project then use **File → Save As** to make a copy in a new directory as `C:\Play\Tutorial2\Tutorial2.rtaos`.

Copy `Application.c` and `VirtualDevices.cpp` from the `Tutorial1` directory to `Tutorial2`.

We are going to create a very artificial VECU to demonstrate the use of virtual devices:

- There will be 4 input switches: `Accelerate`, `Brake`, `Left` and `Right`.
- There will be a speed setting actuator.
- There will be a direction indicating actuator.
- The `Accelerate` and `Brake` switches cause the speed to increase / decrease by one unit on each zero to one transition under interrupt control.
- The `Left` and `Right` switches are polled from an AUTOSAR task every 100ms and cause a one degree change in direction each time they are sampled at '1'.
- The speed starts at zero and limits at 100. The speed is preserved if the program resets, but not if the program restarts.
- The direction is from zero to 359 degrees. The direction is preserved over reset AND program restart.
- There will be on-screen feedback of the current speed and direction.

3.5.1 Devices

Edit `VirtualDevices.cpp` so that it contains the code below

```
#include "vrtaCore.h"
#include "vrtaSampleDevices.h"
#include "vrtaLoggerDevice.h"

#define MSECS_PER_POLL (100)
#define ACCEL_ISR (1)
#define BRAKE_ISR (2)
#define POLL_ISR (3)

// Switches
vrtaSensorToggleSwitch Accelerate("Accelerate");
vrtaSensorToggleSwitch Brake("Brake");
vrtaSensorToggleSwitch Left("Left");
vrtaSensorToggleSwitch Right("Right");

// Actuators
vrtaActuator Speed("Speed");
vrtaActuator Direction("Direction");

// Comparators
vrtaCompare AccelDetect("AccelDetect",Accelerate,1,ACCEL_ISR);
vrtaCompare BrakeDetect("BrakeDetect",Brake,1,BRAKE_ISR);

// Clock
vrtaClock ClockSource("ClockSource",MSECS_PER_POLL);
vrtaUpCounter PollCounter("PollCounter", ClockSource);
vrtaCompare PollCompare("PollCompare", PollCounter, 0,
POLL_ISR);

// Data
#define DATA_SIZE (1)
#define DIR_DATA (0)
vrtaIO PersistentData("PersistentData",DATA_SIZE);

// Status
Logger Status("Status");

//----------------------------------------
int status_printf(const vrtaTextPtr format, ...){
  va_list argptr;
  va_start(argptr, format);
  int ret = Status.printf(format, argptr);
  va_end(argptr);
  return ret;
}

//----------------------------------------
void InitializeDevices(void){
  Speed.SetMax(100);
  Direction.SetMax(359);
  PollCounter.SetMin(0);
  PollCounter.SetMax(0);
  PollCounter.Start();
  ClockSource.Start();
  Speed.PersistThroughReset(true);
  PersistentData.PersistThroughReset(true);
  Direction.SetValue(PersistentData.GetValue(DIR_DATA));
}
```

Hopefully the declarations for the switches, actuators, clock devices and IO will be clear.

3.5.2 Logger

The `Logger` device is not included as part of the standard set of sample devices, but you will find it very useful if you want to output diagnostic text from an application. The `Logger` device is provided by the header file `vrtaLoggerDevice.h`.

You can make `printf()` style calls to a `Logger` device and it can output the text to the console window and/or a file. If you create a `Logger` device with the special name 'Status', then the VRTA-win embedded GUI will display its output in its status bar. vrtaMonitor will do the same.

> *Use a `Logger` device to output text to the VECU's console. Do not use direct printf() calls. The `Logger` device has the interrupt protection that is needed when making non-VRTA API calls from your OS thread.*

Now we come to an interesting issue. Your logger device is a C++ object with a nice set of methods for outputting text and saying where to place the text. You can make calls such as `Status.printf("Boo")` so that you can indicate to the outside world what is happening.

But your RTA-OS application is written in C not C++, so it does not understand the `Status` object. How can it make use of it?

3.5.3 Interfacing C code with C++ Devices

There are two answers to this.

3.5.3.1 Using vrtaSendAction

Your C code can use the API call `vrtaSendAction` to send a string to the device's 'Print' action.

An action is a structure that needs to be populated with details of the action and its associated characteristics. Chapter 9 tells you more about actions, but the code would be similar to that shown here[8].

[8] This won't work if the string length is less than 16 bytes because the string has to be copied into the action's embedded data area. This is discussed later.

```
vrtaAction act;
char *pText_to_send = "Boo";
act.devID = status_device_id;
act.devAction = 1;
act.devActionLen = strlen(pText_to_send);
act.tiono_ = pText_to_send;
vrtaSendAction(act.devID, &act);
```

Note: if you allocate memory for the sent data, then you need to decide whether to free the memory in the receiver or the sender. Otherwise you may end up leaking memory.

3.5.3.2 Using C/C++ Interfacing

The alternative, and simpler, model is to write a C / C++ interface function. This is what is done below. The function `status_printf` is a simple wrapper function that is intended to be callable from C code so your code becomes:

```
status_printf(``Boo'');
```

You just have to provide a prototype for this function that can be seen by both the C and C++ source code that declares `status_printf` to be a C rather than C++ function. For this reason we need to add the file `VirtualDevices.h` which will export the C definitions for use in your C program. `VirtualDevices.h` needs to contain the code shown here.

```
/* Interface between C and C++ */
#ifdef __cplusplus
extern "C" {
#endif

extern int status_printf(const char* format, ...);

#ifdef __cplusplus
}
#endif
```

The file `VirtualDevices.h` must be #included into C or C++ files that define or reference `status_printf`. For this tutorial application this means you need to add `#include "VirtualDevices.h"` in `Application.c` and `VirtualDevices.cpp`.

3.5.4 Device Initialization

The initialization code in our `InitializeDevices()` function sets both the min and max values for the `PollCounter` to zero. You will note that `PollCompare` is also set to match on zero. This has the effect of raising an interrupt every clock tick, because each time the `ClockSource` ticks the `PollCounter` is incremented. Incrementing past its maximum value (0) causes the count to reset to the minimum value

(0). Each time the `PollCounter` gets incremented the new value is passed to the `PollCompare` - which matches on zero every time.

The calls to `PersistThroughReset()` tell the `Speed` and `PersistentData` devices to preserve their data if the program resets. Note that a reset is not the same as killing the program and restarting it manually. We do have a requirement that the direction is preserved over a restart that is not satisfied by this - but we will sort that out a bit later.

3.5.5 The main program

We need to modify `Application.c` so that the program does not simply start and stop by modifying what happens in `Os_Cbk_Idle`. We can also use the new `status_printf` function to display our output. The code now looks like this:

```
#include <Os.h>
#include "VirtualDevices.h"

OS_MAIN(){
  StartOS(OSDEFAULTAPPMODE);
}
FUNC(boolean,OS_CALLOUT_CODE) Os_Cbk_Idle(void){
  while(!vrtaIsAppFinished()){
    vrtaIsIdle(5);
  }
  ShutdownOS(E_OK);
  return TRUE; /* Never reached */
}

FUNC(void,OS_CALLOUT_CODE) StartupHook(void){
  status_printf("Hello World!\n");
}

FUNC(void,OS_CALLOUT_CODE) ShutdownHook(StatusType s){
  status_printf("Goodbye World!\n");
  vrtaTerminate();
}
```

You will notice the use of `status_printf()`.

`vrtaIsAppFinished()` returns false normally or true if the VECU should terminate.

`vrtaIsIdle()` waits for the specified number of milliseconds. During this time the processor is assigned to a different thread. Virtual interrupts can still occur during a call to `vrtaIsIdle()`. Note that `vrtaIsIdle()` should only be called within Os_Cbk_Idle() because it contains hooks to code that must be run when the OS is idle. You can use `vrtaYield()` the other cases.

3.5.6 Trial Run 1

Let's check whether all that was entered correctly. Save your project and perform a build. Hopefully the build will be successful. If not just go back and check that you have entered everything correctly.

You can run the VECU again and this time will get an empty console screen.

On VRTA-win you will also get a GUI window for the VECU. This shows a few details about your VECU and gives you the ability to suspend, resume, reset and terminate the VECU. You should also see "Hello World" on the status bar.

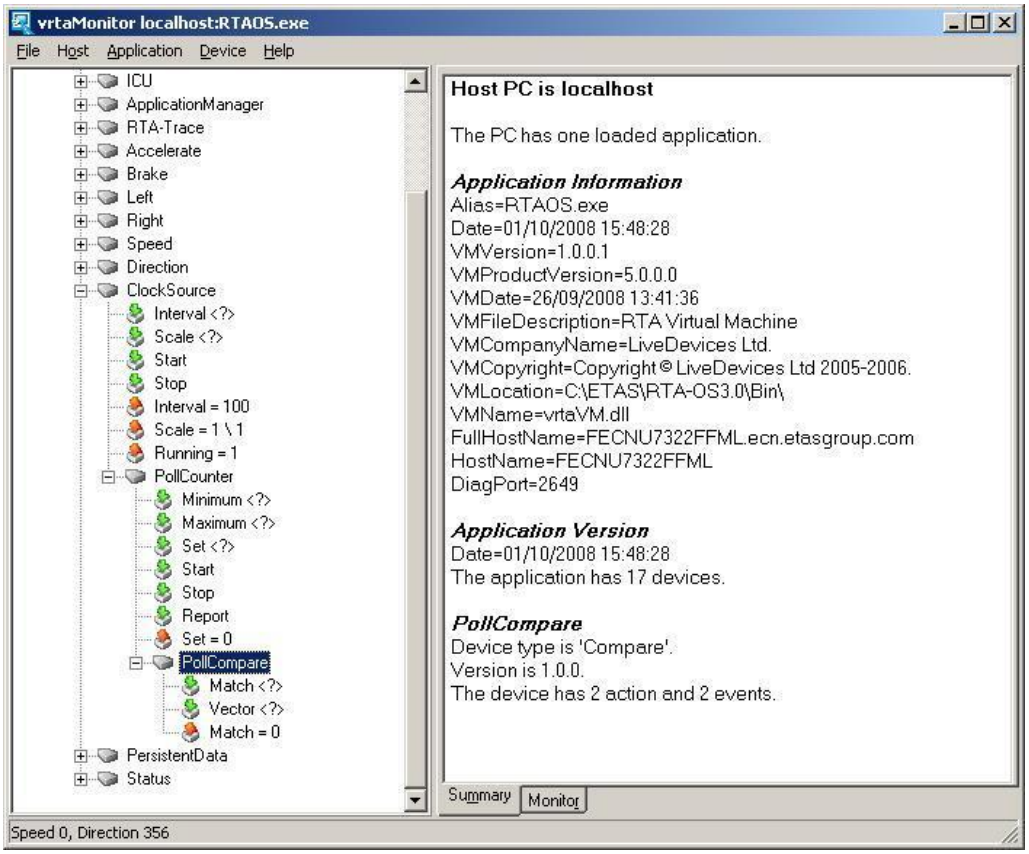
Now launch vrtaMonitor (in `C:\ETAS\RTA-OS\Bin`) and connect it to the vrtaServer running on the host PC. For VRTA-ux this means the version of vrtaServer on the Linux PC, so you might need to add it as a Host name or IP address.

Next click to 'Add' the VECU. You will be given a dialog that allows you to navigate to your VECU.

For VRTA-win this will be `C:\Play\Tutorial2\Tutorial2.exe`.

For VRTA-ux this will be something like `/home/user/Play/Tutorial2/Tutorial2.vrta_ecu`.

The end result should look a bit like this:



Take some time to explore the devices in the monitor. You should be able to work out most of the features by checking out the main menu, clicking (and especially right-clicking) on elements in the left hand navigation pane and looking at the tabs on the right hand side. Chapter 6 describes vrtaMonitor in more detail.

One thing that is worth pointing out is that the monitor has worked out that the `PollCompare` device 'belongs to' the `PollCounter`, which in turn 'belongs to' the `ClockSource`. It has therefore arranged these devices in a hierarchy. The same applies to the Accelerate and Brake compare devices.

3.5.7 Summary so far

We haven't really written much code yet, but a large part of the framework is in place. In fact, we can even see that the clock chain is working. Drag the `Match` Event of the `PollCompare` device from the left hand side over to the right-hand side. This causes the monitor to hook all match events from the device and display them in the Monitor tab. If you tick the Show Times checkbox you will also see the time that each event was raised in the VECU.

To stop monitoring an event, drag the event onto the Stop button. When you are ready, close the monitor and VECU.

3.5.8 Adding Tasks and ISRs

We now create the OS elements that implement the required functionality. Use **RTAOSCfg** to add the following:

- A Task called `DirectionPoll` with
  - Priority 1
  - Activations 1
  - Preemptability FULL
- A Category 2 ISR named `isrAccelerate` with
  - Priority 1
  - Vector 1.
- A Category 2 ISR named `isrBrake` with
  - Priority 1
  - Vector 2.
- A Category 2 ISR named `isrPoll` with
  - Priority 2
  - Vector 3.
- A Counter named `AlarmCounter` with:
  - Minimum Cycle of 1
  - Maximum Value of 65535
  - Ticks Per Base of 1
  - Seconds Per Tick of 0.1 (it is 100ms)
  - Type of SOFTWARE
- An Alarm named `Poller` attached to `AlarmCounter` with
  - Action/Activate Task `DirectionPoll`,
  - Auto start in `OSDEFAULTAPPMODE` with:
    - Type of RELATIVE
    - Alarm Time of 1
    - Cycle Time of 1

We now need to implement the tasks and ISRs. Edit `Application.c` to add the three Category 2 ISRs and the task as shown next.

```
ISR(isrPoll){
  IncrementCounter(AlarmCounter);
}

ISR(isrBrake){
  change_speed(-1);
}

ISR(isrAccelerate){
  change_speed(+1);
}

TASK(DirectionPoll){
  if (left_pressed()) {
    change_direction(-1);
  }
  if (right_pressed()) {
    change_direction(+1);
  }
  TerminateTask();
}
```

We also need to get access to the VECUs devices. Add the lines shown in next to `VirtualDevices.h`. These provide the C/C++ interface function prototypes that allow your VECU to get access to the devices:

```
#ifdef __cplusplus
extern "C" {
#endif

// The existing logger device
extern int status_printf(const char* format, ...);

// The new interfacing functions
extern void show_status(void);
extern int left_pressed(void);
extern int right_pressed(void);
extern void change_direction(int amount);
extern void change_speed(int amount);

#ifdef __cplusplus
}
#endif
```

We now add the code shown below to `VirtualDevices.cpp` to provide the C/C++ functions themselves. They are clearly quite simple wrappers to the devices.

```
int left_pressed(void){
  return Left.Value();
}

int right_pressed(void){
  return Right.Value();
}

void show_status(void){
  Status.printf("Speed %d, Direction %d",
    Speed.Value(),
    Direction.Value());
}

void change_direction(int amount){
  int newvalue = Direction.Value() + amount;
  while (newvalue < 0) {
    newvalue += 360;
  }
  while (newvalue > 359) {
    newvalue -= 360;
  }
  PersistentData.SetValue(DIR_DATA,newvalue);
  Direction.SetValue(newvalue);
  show_status();
}

void change_speed(int amount){
  int newvalue = Speed.Value() + amount;
  if ((newvalue >=0) && (newvalue <=100)) {
    Speed.SetValue(newvalue);
    show_status();
  }
}
```

If you try to run this now, you'll find that nothing much appears to respond to the inputs. This is because we need to enable (or unmask) the 3 interrupts that we are using. We can do this by sending an action to the interrupt control unit (ICU). The action is a structure, so it needs to be created by the application before being sent to the Virtual ECU

This code gets added to `InitializeDevices()`. It sends the Unmask action to the ICU device, passing in each interrupt number in turn.

```
vrtaAction action;

action.devID = ICU_DEVICE_ID;
action.devAction = ICU_ACTION_ID_Unmask;
action.devActionLen = sizeof(action.devEmbeddedData.uVal);
action.devActionData = NULL;

// Unmask the accelerator interrupt
action.devEmbeddedData.uVal = ACCEL_ISR;
vrtaSendAction(ICU_DEVICE_ID,&action);

// Unmask the brake interrupt
action.devEmbeddedData.uVal = BRAKE_ISR;
vrtaSendAction(ICU_DEVICE_ID,&action);

// Unmask the poll interrupt
action.devEmbeddedData.uVal = POLL_ISR;
vrtaSendAction(ICU_DEVICE_ID,&action);
```

Nearly there!

3.5.9 Threads

For no other reason than to show you some interesting stuff, we are now going to add a 'spring' to the Accelerate and Brake switches so that they flip back to zero after being pushed, but we will do this in a separate thread.

You can create any number of threads of execution that run *independently* of your OS thread. These threads are native Windows/Linux threads but a small amount of protection built in. You can make Windows / Linux API calls from within a thread without having to protect them from RTA-OS interrupts. You can access the VECU's devices and even raise interrupts from within a thread, which makes them an excellent choice for interfacing to real hardware.

The basic shape of a thread is usually something like this:

```
void AsyncThread(void) {
  while (!vrtaIsAppFinished()) {
    vrtaYield(100); // Sleep 100ms
  }
}
```

You should check `vrtaIsAppFinished()` regularly within a thread so that the VECU can perform an orderly tidy up when asked to terminate. You should use vrtaYield `()` to yield control to other threads if you have no work to do.

In our thread, we want to hook event changes in the `AccelDetect` and `BrakeDetect` compare devices. Whenever the Match event fires for one of these, we know that the associated switch has been pressed. We then reset the switch

value to zero. Note that hooking the `Accelerate` and `Brake` Position event will not work because events are raised before the compare devices are informed. If you reset the switch in the event hook, the compare devices only ever see the zero value.

To see a thread at work we need to add the thread and hook code below to `VirtualDevices.cpp` *before* `InitializeDevices()`:

```
static vrtaErrType ListenCallback(
  const void *instance,
  const vrtaEvent *event){

  // Has 'Accelerate' become 1?
  if ((event->devID == AccelDetect.GetID())
    && (event->devEmbeddedData.uVal == 1)) {
    // Set directly
    Accelerate.SetValue(0);
  }

  // Has 'Brake' become 1?
  if ((event->devID == BrakeDetect.GetID())
    && (event->devEmbeddedData.uVal == 1)){
    // Set via an action
    vrtaAction act;
    act.devAction = 1;
    act.devActionLen = sizeof(unsigned);
    act.devActionData = NULL;
    act.devID = Brake.GetID();
    act.devEmbeddedData.uVal = 0;
    vrtaSendAction(act.devID, &act);
  }
  return RTVECUErr_NONE;
}

void AsyncThread(void) {

// Create a listener and associate its callback
vrtaEventListener tListener =
  vrtaEventRegister(ListenCallback,0);

// Hook event 1 of AccelDetect into listener
vrtaHookEvent(tListener, AccelDetect.GetID(), 1, true);

// Hook event 1 of BrakeDetect into listener
vrtaHookEvent(tListener,BrakeDetect.GetID(), 1, true);

// Nothing else to do while app is running
while (!vrtaIsAppFinished()) {
  vrtaYield(100);
}

// Tidy up hooks on exit from thread
vrtaHookEvent(tListener,AccelDetect.GetID(),1,false);
vrtaHookEvent(tListener,BrakeDetect.GetID(),1,false);
}
```

We now need to make the thread run, so at the start of `InitializeDevices()` add the line:

```
vrtaSpawnThread(AsyncThread);
```

This tells VRTA to spawn the thread called `AsyncThread`.

3.5.10 Trial run 2

Save your project and perform a build.

You can either rebuild the RTAOS kernel part using the RTA-OS GUI as before, or you can switch to using the command-line RTA-OS tool **RTAOSGen** and add this to your build script. (The full command-line that you need to add is shown in the GUI build window…)

You will now have a (nearly) fully working VECU, so let's run it.

Launch vrtaMonitor and connect it to the VECU.

Expand the `Accelerate` device and drag the `Position` event to the right hand side. Double click the Position action (not the event) and enter '1' as the data value. Press OK. You will see the position value change to one and then zero in the monitor window. The status bar will show that the speed has increased. Double-click on the Position action a few times and you will see the speed go up further.

Do the same for the `Brake` device and notice the speed decrease.

Now expand the `Left` device and send its Position action value '1'. The direction value will count down and wrap at 0 degrees until you change it back to zero.

The same applies to the `Right` device, though obviously that will make the value count up and wrap at 359 degrees.

3.5.11 Linking to Real Hardware (VRTA-win)

Just to prove that you can link to real hardware, you might like to change the implementation of the `AsyncThread` as shown below. It just peeks at the state of the arrow keys on your keyboard and makes calls to your switches.

The below code works only for VRTA-win as Windows API calls have been used for getting the key values from the keyboard.

```
void AsyncThread(void) {

// Create a listener and associate its callback
vrtaEventListener tListener =
  vrtaEventRegister(ListenCallback,0);

// Hook event 1 of AccelDetect into listener
vrtaHookEvent(tListener,AccelDetect.GetID(),1,true);

// Hook event 1 of BrakeDetect into listener
vrtaHookEvent(tListener,BrakeDetect.GetID(),1,true);

// Peek at keys to control Accelerator, Brake, Direction
while (!vrtaIsAppFinished()) {

  if (GetAsyncKeyState(VK_LEFT) < 0) {
    if (Left.GetValue() == 0) {
      Left.SetValue(1);
    }
  } else {
    if (Left.GetValue() == 1) {
      Left.SetValue(0);
    }
  }

  if (GetAsyncKeyState(VK_RIGHT) < 0) {
    if (Right.GetValue() == 0) {
      Right.SetValue(1);
    }
  } else {
    if (Right.GetValue() == 1) {
      Right.SetValue(0);
    }
  }

  if (GetAsyncKeyState(VK_UP) < 0) {
    Accelerate.SetValue(1);
  }

  if (GetAsyncKeyState(VK_DOWN) < 0) {
    Brake.SetValue(1);
  }

  vrtaYield(100);
}

// Tidy up hooks on exit from thread
vrtaHookEvent(tListener,AccelDetect.GetID(),1,false);
vrtaHookEvent(tListener,BrakeDetect.GetID(),1,false);
}
```

3.5.12 Non-volatile Data

If you were to reset the VECU now (for example using the Application menu in vrta-Monitor), the VECU console and GUI will flash off and then return. The speed and direction will have persisted across the reset.

We have seen that the speed and direction values persist over a VECU reset. But we don't yet have a way to keep data between completely different runs of the VECU.

We'd really like to have something that looks like Flash memory in a real ECU. There is no direct support for nonvolatile memory so we will have to make our own version.

We will do this by creating a `vrtaFlash` device that inherits from `vrtaIO`. We can then change the type of `PersistentData` to `vrtaFlash` and the job is done. The code to do this is shown below. Just replace the current declaration for `PersistentData` in `VirtualDevices.cpp`:

```
vrtaIO PersistentData("PersistentData",DATA_SIZE);
```

with these lines

```
#include <stdio.h>
class vrtaFlash : public vrtaIO {

protected:
  void Starting(void) {
    vrtaIO::Starting();
    FILE *f = fopen("VirtualECU.flash","rb");
    if (f) {
      char buffer[100];
      if (GetPersistentDataSize() ==
        fread(buffer,1,GetPersistentDataSize(), f)) {
        memcpy(
          GetPersistentData(),
          buffer,
          GetPersistentDataSize());
      }
    fclose(f);
  }
}

  void Stopping(void) {
    FILE *f = fopen("VirtualECU.flash","wb");
    fwrite(
      GetPersistentData(),
      1,
      GetPersistentDataSize(),
      f);
    fclose(f);
    vrtaIO::Stopping();
  }

public:
  // Constructor
  vrtaFlash(const vrtaTextPtr name, unsigned elements):
    vrtaIO(name, elements) {};
};
vrtaFlash PersistentData("PersistentData",DATA_SIZE);
```

You will now find that if you rebuild your VECU that it will remember the value of the direction it was last pointing in when it restarts. The data is stored in the file called `VirtualECU.flash` which is read when the device starts and written when the device stops. In a real-life application you might want to move away from a hard-coded file name, to avoid conflicts where two applications try to access the same file.

3.6 Summary

If you have worked through this tutorial then you should have a good understanding of how to work with a Virtual ECU. In particular you should now have an idea of how you can:

- Create virtual devices and initialize them.
- Access virtual devices from your C (RTA-OS) application.
- Print debugging data to the embedded GUI and to the status bar of vrtaMonitor.
- Interact with your virtual devices using
    - vrtaMonitor
    - an asynchronous thread to simulate some behavior
    - real hardware

4 ECUs and Virtual Devices

At its most basic level, a virtual device is simply a software component that has a name and provides functions that can be called to send it a command (action) or query its status (event).

The action callback function gets passed information through a `vrtaAction` structure.

The event callback function gets passed information through a `vrtaEvent` structure.

Virtual devices can be written from scratch using C code, but we recommend that they are implemented as C++ classes that derive from either the `vrtaDevice` class that is defined in `vrtaDevice.h` or the `VirtualDevice` class that is defined in `virtualDevice.hpp`. The `vrtaDevice` class is the original implementation of a wrapper for a virtual device. The `VirtualDevice` class is a more recent implementation of the same thing that is preferred simply because it has a more refined user interface. These are described shortly.

This chapter covers some of the issues that you should understand when writing your own devices.

4.1 Low-level API

4.1.1 Registering the device

You have to tell the VRTA that your device exists by calling the `vrtaRegisterVirtualDevice()` API. This must be done before `vrtaStart()` is called. It is done automatically for you if you are using a class that derives from `vrtaDevice` or `VirtualDevice`.

When registering a device, you supply the following information:

- **name**
  This is the name that external monitor programs will see when accessing the device. Each device in a VECU must have a different name.
  e.g. “LeftWindowSwitch”, “EjectorSeatTrigger”

- **info**
  This is a string containing information about the device in the form “<tag1>=<value1>\n<tag2>=< value 2>…”. As a minimum the string should contain Type and Version tags. This information is used by external monitor programs.
  e.g. “Type=Thruster\nVersion=1.2.3\n”

  By convention, you should provide info strings for each event and action that the device supports. This information can be used to guide the user how to use the device. e.g.

  ```
  Action Start=Enable counting
  Action Stop=Disable counting
  Action Report=Causes the Set event to be raised
  (Raises Set Event)
  Event Set=The value of the count when Report action
  is raised (Raise,Poll)
  ```

- **list of events**
  This is a list of \n separated name-value pairs as above. It lists the name and data format for each event in the device. The tags are the event names and the values are the data format descriptions. These are explained in detail in section 9.3.
  e.g. “Value=%u,%u(%u)\nValues=%a\n”

  You can (and should) give a name and description for each data element. This is done by extending the description of the data element.
  e.g.

```
Scale=%u Multiplier{Multiplies the interval},%u Di-
vider{Divides the interval}

Running=%u:;0;1 State{1 when the clock is running}
```

- **list of actions**
  This is the list of device actions, using the same form as for events. e.g. “Value=%u,%u\nValues=%a\nGetValue=%u\nGetValues\n”

  Again, data elements can be given a name and description.

  ```
  Interval=%u ms{Set tick interval in milliseconds}
  ```

- **action callback function**
  This is a reference to the C function that will be called when an action is sent to the device. See the section below on how to handling actions.

- **state callback function**
  This is a reference to the C function that will be called when a state query is sent to the device. See the section below on handling state queries.

4.1.2 Example low-level device implementation

```
vrtaRegisterVirtualDevice(
 "MyName",
 "Type=Demo device\nVersion=1.0.0",
 "MyEvent1=%x(%u)\nMyEvent2=%u",
 "MyAction1=%a\nMyAction2=%d",
 MyActionCallback,
 MyStateCallback,
 MyTag);

vrtaErrType
MyActionCallback(void *tag, const vrtaAction *action)
{
  if (tag != MyTag) {
    return RTVECUErr_Dev;
  }
  if (action->devAction == 1) { // first action
    ...
    return RTVECUErr_NONE;
  } else if (action->devAction == 2) {
    ...
    return RTVECUErr_NONE;
  } else {
    return RTVECUErr_ID;
  }
}

vrtaErrType
MyStateCallback(void *tag, vrtaEvent *event)
{
  if (tag != MyTag) {
    return RTVECUErr_Dev;
  }
  if (event->devEvent == 1) { /* first event */
    ...
    return RTVECUErr_NONE;
  } else if (event->devEvent == 2) {
    ...
    return RTVECUErr_NONE;
  } else {
    return RTVECUErr_ID;
  }
}
```

4.1.3 Handling actions

The action callback function that you register in `vrtaRegisterVirtualDevice()` gets called when code in the VECU calls `vrtaSendAction()`, or when an external monitor sends data via the Device Communication Link.

The callback can be invoked from any thread so the callback must take care of any reentrancy issues.

Often you will raise an event as a result of receiving an action.

4.1.4 Handling state queries

The state callback function that you register in `vrtaRegisterVirtualDevice()` gets called when code in the VECU calls `vrtaGetState()`, or when an external monitor queries the device via the Device Communication Link.

The callback can be invoked from any thread so the callback must take care of any reentrancy issues.

4.1.5 Raising events

The `vrtaRaiseEvent()` API is used to raise an event. e.g.

```
vrtaEvent event;
event.devEvent = event_id;
event.devEventLen = sizeof(event.devEmbeddedData.uVal);
event.devEmbeddedData.uVal = 123U;
event.devEventData = NULL;
vrtaRaiseEvent(device_id, &event);
```

4.1.6 Raising interrupts

Interrupts get raised by sending action ICU_ACTION_ID_Raise to the ICU device using code like this:

```
void RaiseInterrupt(unsigned vector) {
  vrtaAction action;
  action.devID                = ICU_DEVICE_ID;
  action.devAction            = ICU_ACTION_ID_Raise;
  action.devActionLen         =
    sizeof(action.devEmbeddedData.uVal);
  action.devEmbeddedData.uVal = vector;
  action.devActionData        = NULL;
  vrtaSendAction(ICU_DEVICE_ID, action);
}
```

4.2 Virtual Devices using the VirtualDevice class

This is the preferred way to create virtual devices. It provides a wrapper around the low-level C API. It simplifies the task of creating devices and dealing with actions and events.

You do not need to call `vrtaRegisterVirtualDevice()` for devices that are created as instances of `VirtualDevice` – this gets done via the `RegisterAs()` method.

4.2.1 Creating the device

You create a new device based on `VirtualDevice` by subclassing it and supplying the registration information through three callback functions. These get called by the `RegisterAs()` method which takes the information that they supply and makes the appropriate call to `vrtaRegisterVirtualDevice()`.

4.2.1.1 GetDeviceInfo(OptList &info)

`GetDeviceInfo` is used to declare the **info** descriptions that are needed when declaring a device. Rather than writing these as one long string, you effectively just declare name-value pairs.

e.g.

```
void GetDeviceInfo(OptList &info) {
  info.Add("Type", "Widget");
  info.Add("Description", "Example device.");
  info.Add("Version", "1.2.3");
}
```

Note that you don't need to provide any information about actions and events in here. That information comes from the other two callbacks.

4.2.1.2 GetDeviceActions(ActionList &actions)

`GetDeviceActions` is used to declare the action-related information for the device. It is passed an ActionList type for you to add the information to.

For each action, you specify:

1) Its action handler callback
2) Its name
3) Its description
4) Usage information

5) Its parameters

The action handler callback is declared using the `ADD_ACTION_HANDLER` macro. You pass it the full name of the callback, including the name of the device class.

The name, description and usage information are declared as the 3 argument to `actions.add()` as shown below.

If the action has parameters, you should declare these by chaining `In()` calls on to the `actions.add()` as shown below. The `In()` call is passed the parameter name, description and format string.

```
class Widget : public VirtualDevice {
...

void GetDeviceActions(ActionList &actions) {

  ADD_ACTION_HANDLER(Widget::ThisAction);
  actions.Add("This", "Do this", "Raises This Event");

  ADD_ACTION_HANDLER(Widget::ThatAction);
  actions.Add("That", "Do that", "Raises That Event").
    In("Depth", "Depth in metres", "%d);

  ADD_ACTION_HANDLER(Widget::OtherAction);
  actions.Add("Other", "Do something else", "").
    In("Angle", "New angle", "%u).
    In("Speed", "Rate of change degree/s", "%u");
}
...

}
```

NOTE: Where the action receives data that is bigger than `sizeof(action.devEmbeddedData)` you should free the memory that was allocated for the sent data. Us code like this:

```
if (action.devActionLen > sizeof (action.devEmbeddedData)) {
  delete [] action.devActionData;
}
```

GetDeviceEvents(EventList &events)

`GetDeviceEvents` is used is used to declare the event-related information for the device. It is passed an EventList type for you to add the information to.

For each action, you specify:

1) Its event handler callback
2) Its name
3) Its description
4) Usage information
5) Its parameters

The event handler callback is declared using the `ADD_EVENT_HANDLER` macro. You pass it the full name of the callback, including the name of the device class.

The name, description and usage information are declared as the 3 argument to `events.add()` as shown below.

If the action has parameters, you should declare these by chaining `In()` or `Out()` calls on to the `events.add()` as shown below. The `In/Out` calls are passed the parameter name, description and format string.

`In()` parameters are those passed from the outside world in to the VECU.

`Out()` parameters are those passed to the outside world from the VECU.

```
class Widget : public VirtualDevice {
...

void GetDeviceEvents(EventList &events) {

  ADD_EVENT_HANDLER(Widget::ThisEvent);
  events.Add("This", "This ocurred", "Hook");

  ADD_EVENT_HANDLER(Widget::ThatEvent);
  events.Add("That", "That value", "Poll,Hook").
    Out("Depth", "Depth in metres", "%d);

  ADD_EVENT_HANDLER(Widget::QueryEvent);
  events.Add("Other", "Do something else", "Poll").
    In("Channel", "Channel number", "%u).
    Out("Channel", "Channel number", "%u).
    Out("Pressure", "Pressure on the channel", "%f");
}
...

}
```

4.2.2 Handling actions

In the `VirtualDevice` class, each action has its own handler that gets called when the action is invoked. It can be invoked from any thread so must take care of any reentrancy issues.

Example action handlers are shown below.

```
class Widget : public VirtualDevice {
...

// ----------------------------------------
vrtaErrType ThisAction(const vrtaAction &action) {
  RunThis();
  Raise(EVENT_HANDLER(Widget::ThisEvent));
  return RTVECUErr_NONE;
}

// ----------------------------------------
vrtaErrType ThatAction(const vrtaAction &action) {
  uint32 depth = UnsignedValueFor(action);
  RunThat(depth);
  Raise(EVENT_HANDLER(Widget::ThatEvent));
  return RTVECUErr_NONE;
}

struct OtherData { vrtaUInt angle; vrtaUInt rate; };
// ----------------------------------------
vrtaErrType OtherAction(const vrtaAction &action) {
  struct OtherData data;
  UnpackData(action, data);
  RunOther(data.angle, data.rate);
  return RTVECUErr_NONE;
}

}
```

The action handler gets passed a `vrtaAction` instance that contains information about the action, and specifically any parameter data. You can extract the data from the instance directly, or you can take advantage of these helper functions:

- `vrtaInt SignedValueFor(action)` returns the parameter value for an action with a single %d parameter.
- `vrtaUInt UnsignedValueFor(action)` returns the parameter value for an action with a single %u parameter.
- `vrtaDouble DoubleValueFor(action)` returns the parameter value for an action with a single %f parameter.
- `template<typename T> void UnpackData(action, dest)` copies the data to the referenced type (`T &`) `dest`.

- `void UnpackData(action, dest, size_of)` copies `size_of` bytes of data to the address in the (`void *`) pointer `dest`.

4.2.3 Handling state queries

In the `VirtualDevice` class, each event has its own handler that gets called when the event is invoked. The callback gets passed a `vrtaEvent` instance that contains information about the event, and specifically any incoming parameter data. The same `vrtaEvent` instance then has to be populated with the outgoing data.

Example event handlers are shown below.

```
class Widget : public VirtualDevice {
...
// ----------------------------------------
vrtaErrType ThisEvent(vrtaEvent &event) {
  NoData(event);
  return RTVECUErr_NONE;
}

// ----------------------------------------
vrtaErrType ThatEvent(vrtaEvent &event) {
  SetValue(ReadDepth());
  return RTVECUErr_NONE;
}

// ----------------------------------------
vrtaErrType QueryEventOption1(vrtaEvent &event) {
  struct QueryData data;
  uint32 channel = UnsignedValueFor(event);
  SetValue(event, channel, ReadPressure(channel));
  return RTVECUErr_NONE;
}

struct QueryData { vrtaUInt chan; vrtaFloat pressure; };
// ----------------------------------------
vrtaErrType QueryEventOption2(vrtaEvent &event) {
  struct QueryData data;
  uint32 channel = UnsignedValueFor(event);
  data.chan = channel;
  data.pressure = ReadPressure(channel);
  PackData(event, data);
  return RTVECUErr_NONE;
}

}
```

For **incoming** parameters, you can extract the data from the instance directly, or you can take advantage of these helper functions:

- `vrtaInt SignedValueFor(action)` returns the parameter value for an action with a single %d parameter.
- `vrtaUInt UnsignedValueFor(action)` returns the parameter value for an action with a single %u parameter.
- `vrtaDouble DoubleValueFor(action)` returns the parameter value for an action with a single %f parameter.
- `template<typename T> void UnpackData(action, dest)` copies the data to the referenced type (`T &`) `dest`.
- `void UnpackData(action, dest, size_of)` copies `size_of` bytes of data to the address in the (`void *`) pointer `dest`.

For **outgoing** parameters, you can put the data in the instance directly, or you can use these helpers.

- `NoData(event)` should be used when no data has to be returned.
- `SetValue(event, value)` is used to pass one value (any type).
- `SetValue(event, value1, value2)` is used to pass two values (any type).
- `SetValue(event, value1, value2, value3)` is used to pass three values (any type).
- `PackData(event, value, sz)` is used to pass a type value of size sz bytes.
- `PackData(event, src, sz)` is used to pass data at pointer (void *) value with size sz bytes.

By convention, `AsyncGetState` returns the value of the most recent `RaiseEvent` for the event in question so that the state of a device can be tracked by either 'hooking' the events or polling them.

4.2.4 Raising events

Code within a class that derives from `VirtualDevice` ancestor can use the `Raise()` method to raise an event.

It causes the specified event callback to get called to fill in an event structure and publish its value to any listeners.

e.g.

```
//--------------------------------------------------void
MyDev::NewValue(unsigned val)
//-------------------------------------------------
{
  m_Val = val; /* Value that event 1 returns */

  Raise(EVENT_HANDLER(MyDev::MyEvent));
}
```

4.2.5 Raising interrupts

Interrupts get raised by sending action `ICU_ACTION_ID_Raise` to the ICU device using code like this:

```
//-----------------------------------------------------
void RaiseInterrupt(unsigned vector)
{
  vrtaAction action;

  action.devID                  = ICU_DEVICE_ID;
  action.devAction              = ICU_ACTION_ID_Raise;
  action.devActionLen =       sizeof(action.devEmbed-
dedData.uVal);
  action.devEmbeddedData.uVal = vector;
  action.devActionData          = NULL;

  SendAction(ICU_DEVICE_ID, action);
}
```

Note the use of `SendAction` to send the action.

Alternatively the `VirtualDevice` class already has its own version of `RaiseInterrupt()` that can be used directly.

e.g.

```
//-----------------------------------------------------void
MyDev::NewValue(unsigned val)
//-----------------------------------------------------
{
  if (val == m_Match) {
    RaiseInterrupt(m_Vector);
    RaiseEvent(MyDev::NewEvent);
  }
}
```

4.3 Virtual Devices using the vrtaDevice class (deprecated)

This is the old way of creating virtual devices and is described here for completeness. New software will find it easier to use the `VirtualDevice` class.

You do not need to call `vrtaRegisterVirtualDevice()` for devices that are created as instances of `vrtaDevice` – this gets done automatically in the constructor.

4.3.1 Creating the device

You create a new device based on `vrtaDevice` by subclassing it and passing the registration information through the constructor.

The registration information is the same that is passed to `vrtaRegisterVirtualDevice()` – just omitting the callbacks and tag.

4.3.2 Example vrtaDevice-based implementation

```
const vrtaTextPtr LoggerVer = (vrtaTextPtr)"Type=Logger\n"
"Version=1.0.0\n"
"Action Print=Sends a string to the logger (Raises Read and
Lines Events)\n"
"Action OutputFile=Sets the name of the primary logger output
file\n"
"Action Duplicate=Sets the name of the duplicate logger output
file\n"
"Event Read=Content of logger text buffer (Raise,Poll)\n"
"Event Lines=Number of lines in logger text buffer
(Raise,Poll)\n"
"";

const vrtaOptStringlistPtr LoggerEv = (vrtaTextPtr)
"Read=%s Text{Content of logger text buffer}\n"
"Lines=%u Count{Number of lines in logger text buffer}\n"
"_Parent=%u";

const vrtaOptStringlistPtr LoggerAct = (vrtaTextPtr)
"Print=%s Text{Text to write}\n"
"OutputFile=%s File{Name of primary output file}\n"
"Duplicate=%s File{Name of secondary output file}\n";

Logger::Logger(const vrtaTextPtr name) : vrtaDevice(name, Log-
gerVer, LoggerEv, LoggerAct)
{
  ...
}
```

4.3.3 Handling actions

When device inherits from `vrtaDevice`, the action callback is translated into a call to its `OnAction` method.

This gets called when code in the VECU calls `vrtaSendAction()`, or when an external monitor sends data via the Device Communication Link. It can be invoked from any thread so must take care of any reentrancy issues.

Often you will raise an event as a result of receiving an action.

The basic form of the `OnAction` method is shown below.

```
//----------------------------------------------------
vrtaErrType mydev::OnAction(const vrtaAction *action)
//----------------------------------------------------
{
  switch (action->devAction) {
    case 1:
      /* respond to action 1 */
      RaiseEvent(...);
      break;

    default:
      return ErrorAction(action);
  }
  return OKAction(action);
}
```

4.3.4 Handling state queries

When a device inherits from `vrtaDevice`, the event callback is translated into a call to its `AsyncGetState` method.

This get called when code in the VECU calls `vrtaGetState()`, or when an external monitor queries the device via the Device Communication Link. It can be invoked from any thread so must take care of any reentrancy issues.

The basic form of the `AsyncGetState` method is shown below.

```
//----------------------------------------------------
vrtaErrType mydev::AsyncGetState(vrtaEvent *event)
//----------------------------------------------------
{
  switch (event->devEvent) {
    case 1:
      /* Update *event */
      break;

    default:
      return ErrorState(event);
  }
  return OKState(event);
}
```

By convention, `AsyncGetState` returns the value of the most recent `RaiseEvent` for the event in question so that the state of a device can be tracked by either 'hooking' the events or polling them.

4.3.5 Raising events

Code within a `vrtaDevice` ancestor can use the `ReadState()` and `RaiseEvent()` methods.

`ReadState()` causes the `AsyncGetState` callback to get used to fill in the event structure, then `RaiseEvent()` is used to do the raise.

e.g.

```
//-----------------------------------------------void
mydev::NewValue(unsigned val)
//-----------------------------------------------
{
  m_Val = val; /* Value that event 1 returns */

  vrtaEvent event;
  ReadState(&event,1);
  RaiseEvent(event);
}
```

4.3.6 Raising interrupts

Interrupts get raised by sending action `ICU_ACTION_ID_Raise` to the ICU device using code like this:

```
//-----------------------------------------------
void RaiseInterrupt(unsigned vector)
{
  vrtaAction      action;
  action.devID                = ICU_DEVICE_ID;
  action.devAction            = ICU_ACTION_ID_Raise;
  action.devActionLen         =
     sizeof(action.devEmbeddedData.uVal);
  action.devEmbeddedData.uVal = vector;
  action.devActionData        = NULL;
  SendAction(ICU_DEVICE_ID, action);
}
```

Note the use of `SendAction` to send the action.

Alternatively the `vrtaDevice` class has a `RaiseInterrupt()` method that does this all more directly.

e.g.

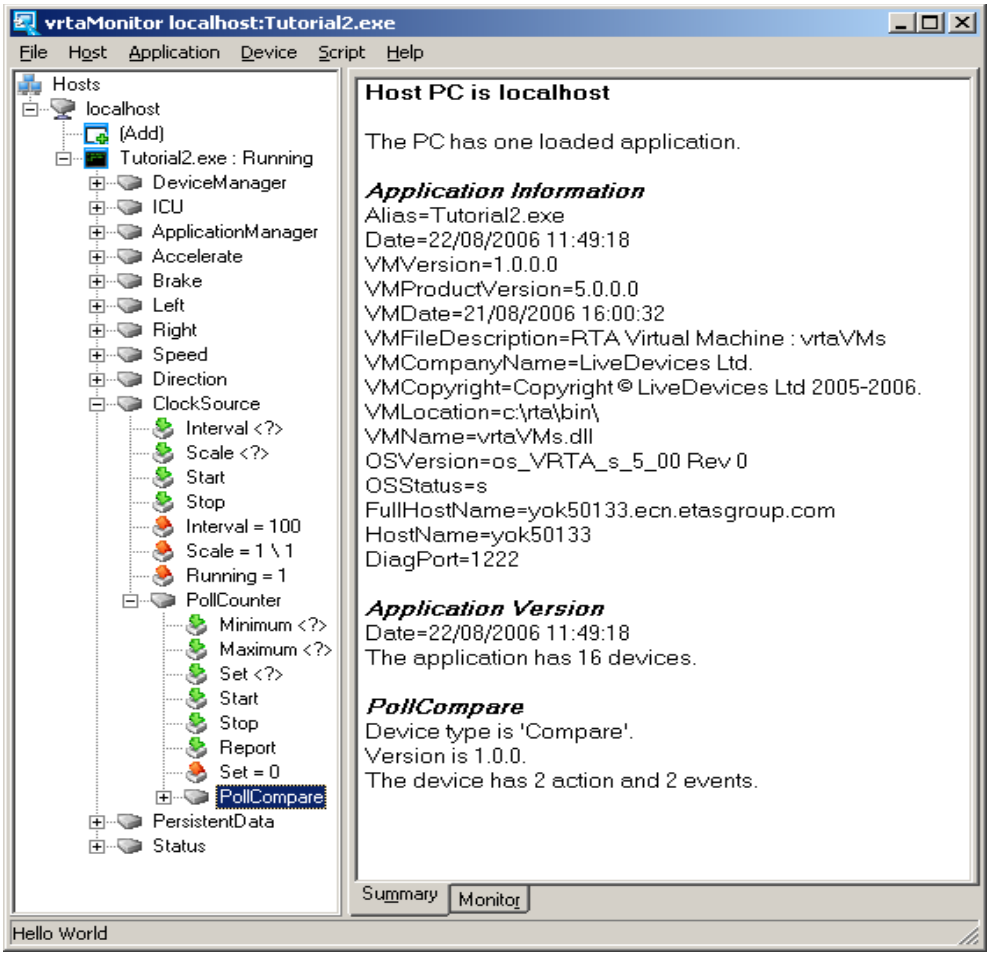
```
//--------------------------------------------------void
mydev::NewValue(unsigned val)
//-------------------------------------------------
{
  m_Val = val;
  if (val == m_Match) {
    vrtaEvent event;
    ReadState(&event,1);
    if (m_Vector) {
      RaiseInterrupt(m_Vector);
    }
    RaiseEvent(event);
  }
}
```

4.4 Parent / Child relationships

Sometimes you want to create a device that somehow 'belongs to' another device.

An example is the `vrtaCounter` device that 'belongs' to a `vrtaClock`.

You can tell external programs such as `vrtaMonitor` about this relationship by implementing an event called "_Parent"[9] which returns the device ID of the device that it belongs to. The program can then represent this relationship visually and will normally hide the _Parent event from view. This feature is demonstrated in the screenshot below – where `PollCompare`'s _Parent is set to `PollCounter` and `PollCounter`'s _Parent is set to `ClockSource`.



4.5 Threads

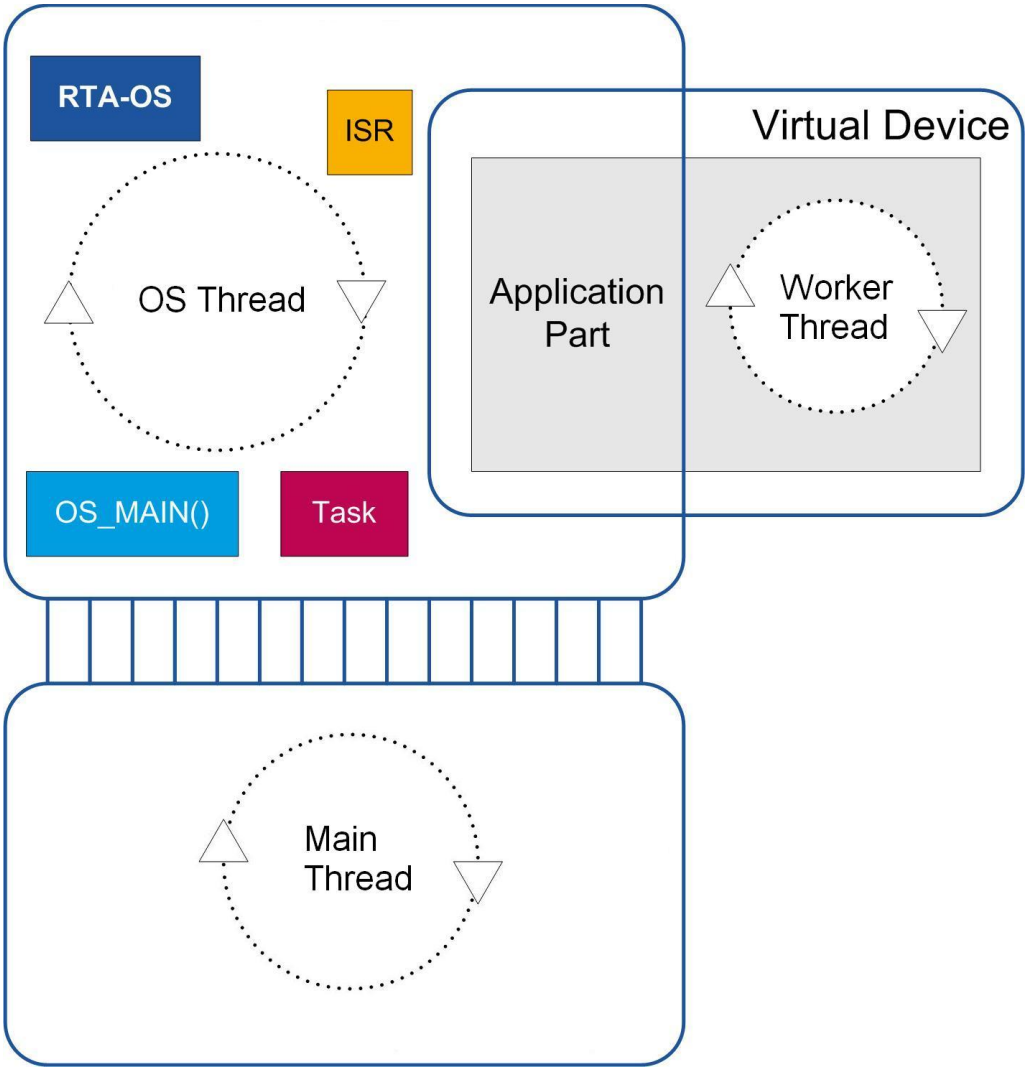
A virtual device can spawn a VRTA worker thread to perform operations asynchronously from the main or OS threads. Such threads can, with appropriate interlocks, access the device data and methods. They can cause events and interrupts to be raised.

---

[9] By convention this is the highest numbered event in the device.

The `vrtaDevice` class provides the `SpawnThread()` method that can be used by classes that inherit from it.

The diagram below illustrates how threads are used within a VECU. The OS thread is the thread that runs the VECU application code, including ISRs and RTA-OS tasks. This is the thread that calls `OS_MAIN()`. The main thread is the thread created by when the VECU was loaded. This is the thread that executes `main()`. Virtual device drivers may be called by the OS thread but may also contain private threads.

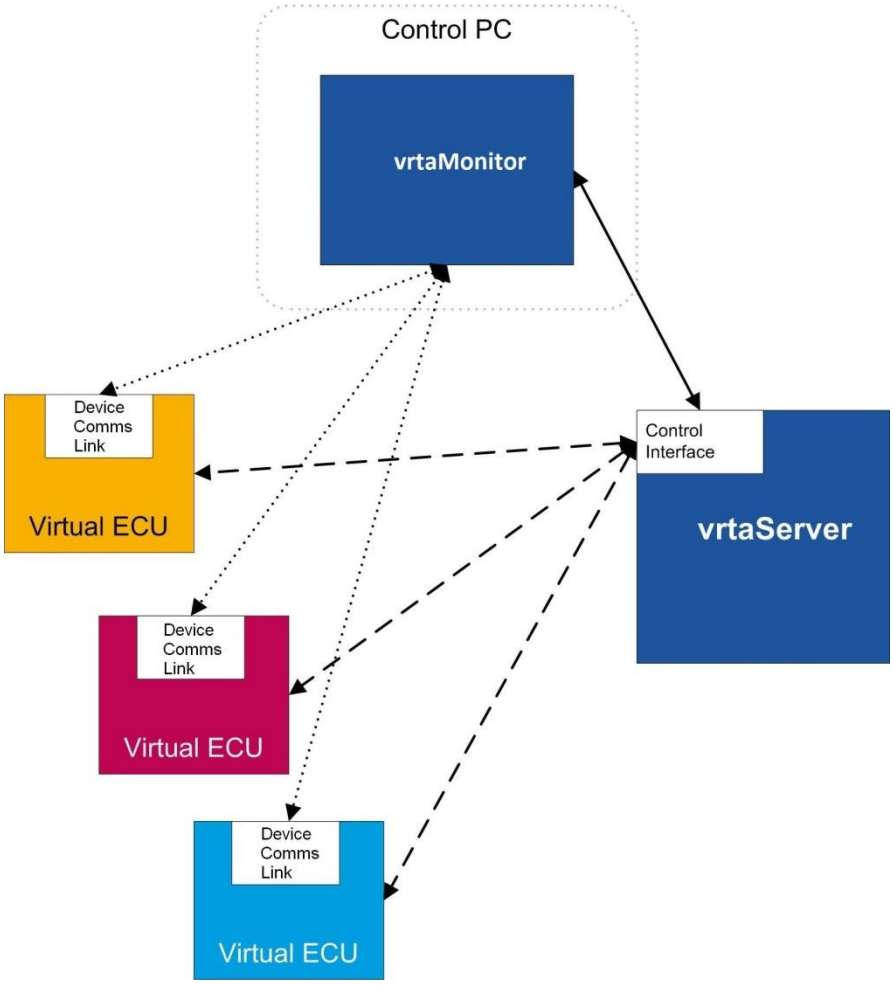


5 vrtaServer

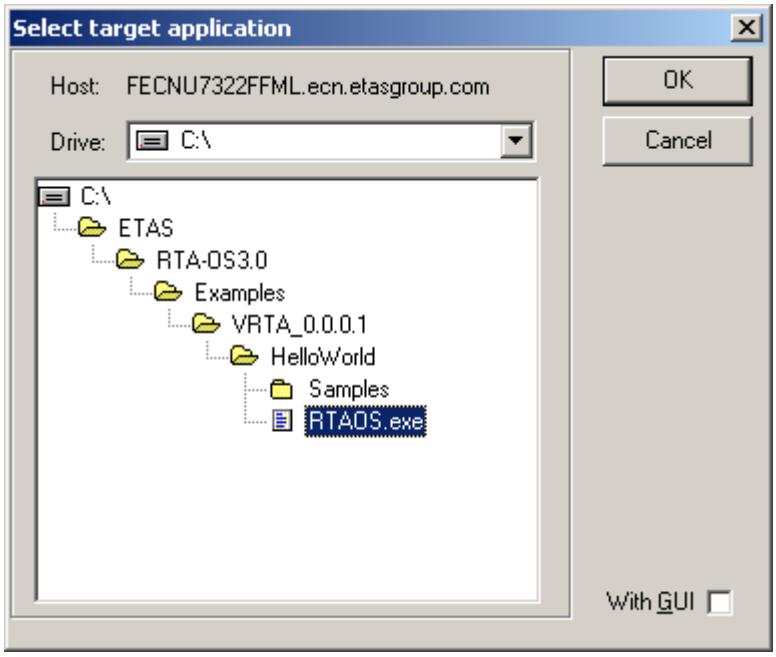
`vrtaServer.exe` is a small program that runs unobtrusively on the PC hosting your VECU which coordinates the loading and locating of your Virtual ECUs. (As discussed in section 2.3, there are several different versions of vrtaServer that you can choose from.)

A VECU informs `vrtaServer` when it starts or terminates. External programs such as `vrtaMonitor` can then ask `vrtaServer` what VECUs are loaded, and attach to a VECU via its device communications link. An important point to note is that `vrtaMonitor` can be on a different PC to the server and its ECUs, so you can perform remote monitoring and control of a bank of test PCs.

The figure below shows a `vrtaMonitor` attached to three VECUs. When the VECUs load, they register with `vrtaServer` (dashed lines). `vrtaMonitor` then queries `vrtaServer` to find out what VECUs are loaded on the local machine and the TCP port numbers of their Device Communication Link (solid line). `vrtaMonitor` then communicates with the VECUs via these links (dotted lines).



Monitor programs can use `vrtaServer` to locate and load VECUs. This is necessary because the monitor may be running on a remote PC without access to files on the host PC. In the example load dialog below, `vrtaMonitor` is accessing a remote PC, so the directory structure that you see reflects that on the remote PC.



5.1 Multiple instances of a VECU

Each VECU that loads is assigned a name (or alias) by `vrtaServer`. Normally this is just the file name of the VECU with the path information stripped off. This 'user-friendly' alias is the name that gets shown in a monitor program.

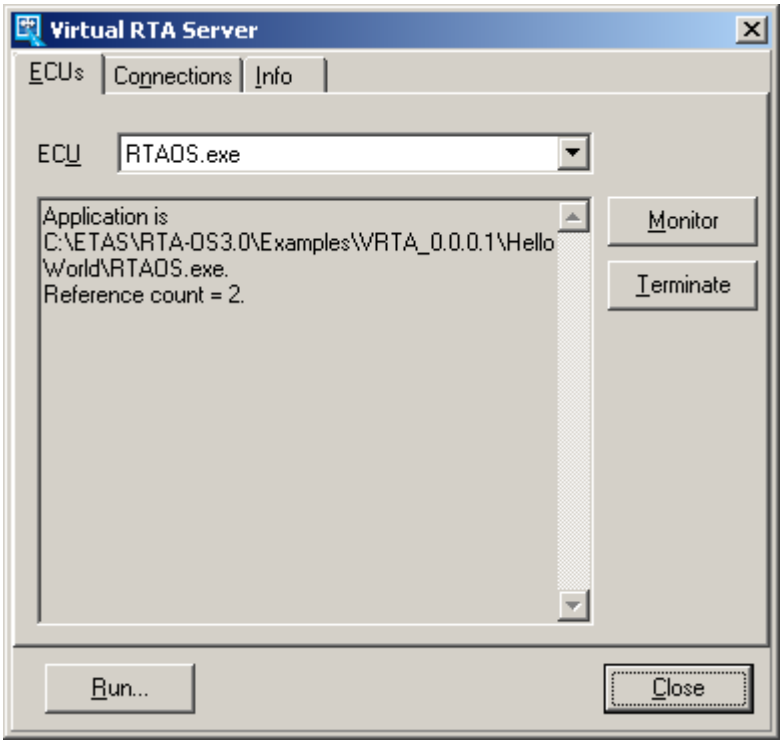
If the same VECU is loaded twice, or if an ECU with the same name but in a different directory is loaded, `vrtaServer` has to generate a different alias. Typically it will do this by adding _2, _3 etc. to the default alias.

`vrtaServer` keeps a count of the ECUs and monitors that know about the different aliases. It frees an alias when no programs are using it.

5.2 Server status information

You normally run `vrtaServer` as a Windows system-tray application. Run '`vrtaServer -standalone`'. You will see a tabbed dialog-style window appear along with a new icon in your system tray.

This dialog will give you information about the VECUs and monitors that are connected to `vrtaServer`.



For **VRTA-ux,** `vrtaServer` normally runs as a terminal application, so its console output can be used to track status.

5.3 Security issues

`vrtaServer` allows remote monitor programs to launch VECUs on its host PC. This means that there are security issues of which you need to be aware.

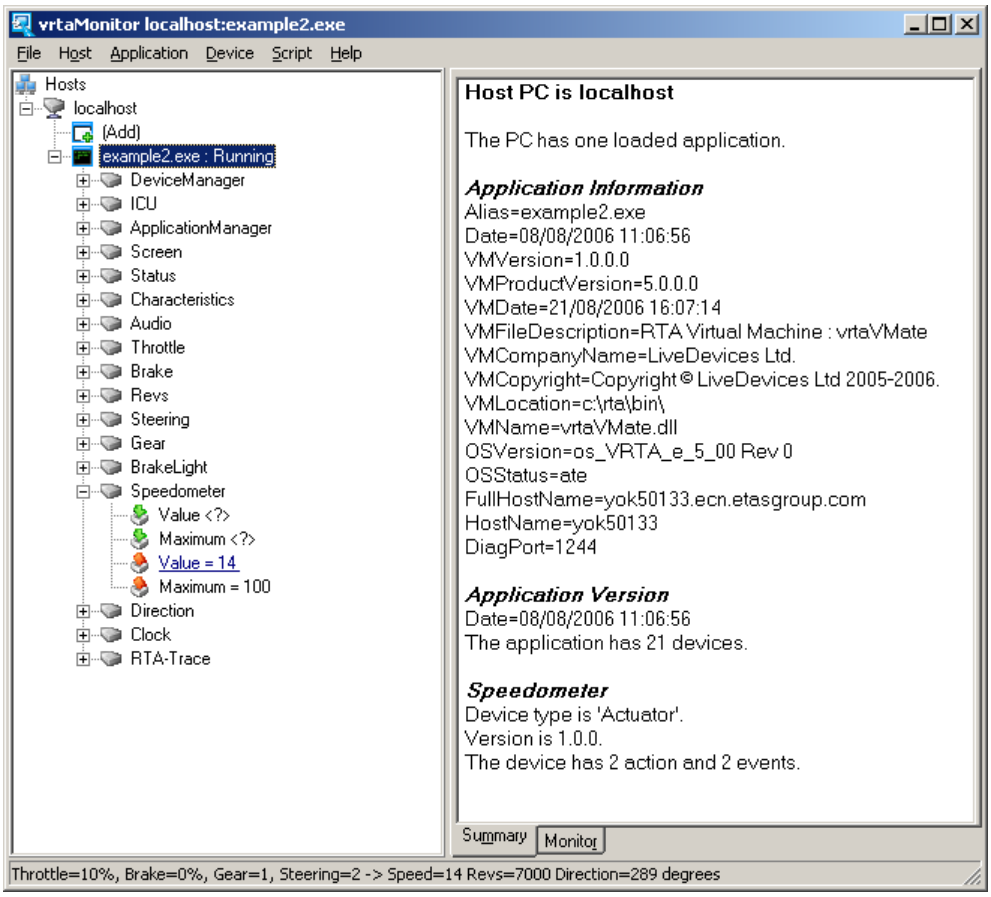
`vrtaServer` does not allow a monitor program to copy any programs or data to the host PC or allow a remote user to modify files via the load dialog.

In a controlled test environment, on a secure network, vrtaServer should not pose any serious security threat. However if you are worried about malicious abuse of this feature, you should configure your firewall to block external access to vrtaServer and ensure that vrtaServer and any VECUs run on the same host PC. The default TCP ports used by `vrtaServer` are 26000, 31765 and 17185.

You can alternatively use the `-p<n>` command-line option to force `vrtaServer` to listen on a non-standard TCP port number. A casual visitor would find it difficult to guess the port, though a port-scan would find the connection easily.

6 vrtaMonitor GUI

The program `vrtaMonitor.exe` can be used to inspect and control virtual ECUs on local and remote PCs.



vrtaMonitor can connect to multiple PCs (Hosts), and multiple VECUs within each host.

You can interact with `vrtaMonitor` in a number or ways including:

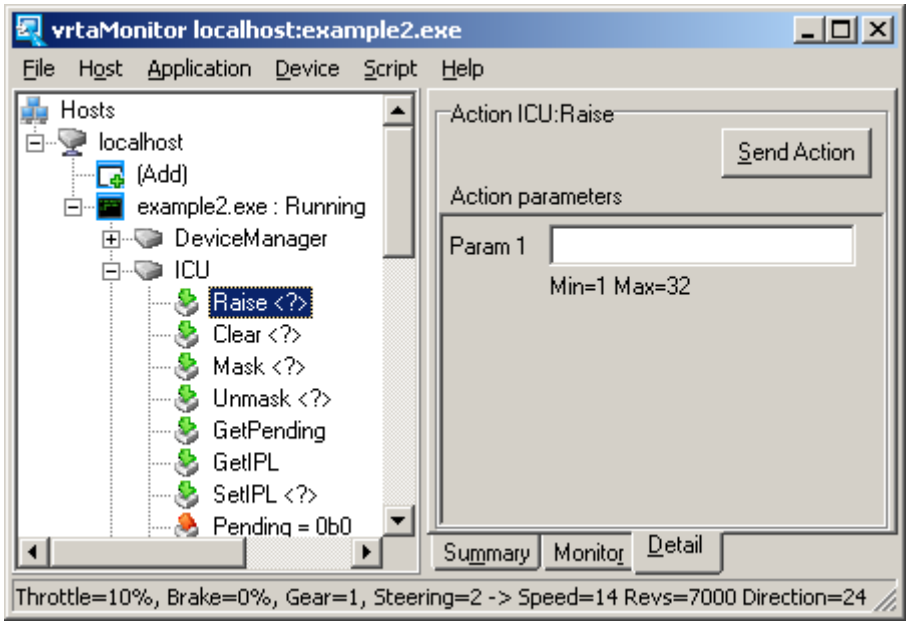
- The application's main menu
- Context menus (right-click on an element in the tree view)
- Shortcut keys (e.g. Ctrl+L to load an ECU)
- Double-clicking on an element in the tree view
- Pressing Enter on an element in the tree view

For example, you can connect to a different PC by right-clicking the **Hosts** element and selecting **Add Host…** from the context menu. You can do the same thing through the application's **File** menu.

6.1 Actions

You can send an action ( ) to a virtual device in the following ways.

- Double-click on an action in the tree view. If the action does not require any data then the action is sent immediately (e.g. Application-Manager / Pause). If the action requires input data (e.g ICU / Raise) then `vrtaMonitor` asks you to enter it the first time round, then re-sends the same value on subsequent double-clicks.
  (If you want to change the data that gets sent, select the Params… option from the context or main Device menu or press Ctrl+Alt+S).
- Press Ctrl+S when an action is selected. This is the same as a double-click above.
- Right-click the action and select Send from the context menu.
- Select the main menu item Device / Current Action / Send.
- Go to the Detail tab on the right-hand side. You can enter data (where needed) and send it by pressing the Send Action button.



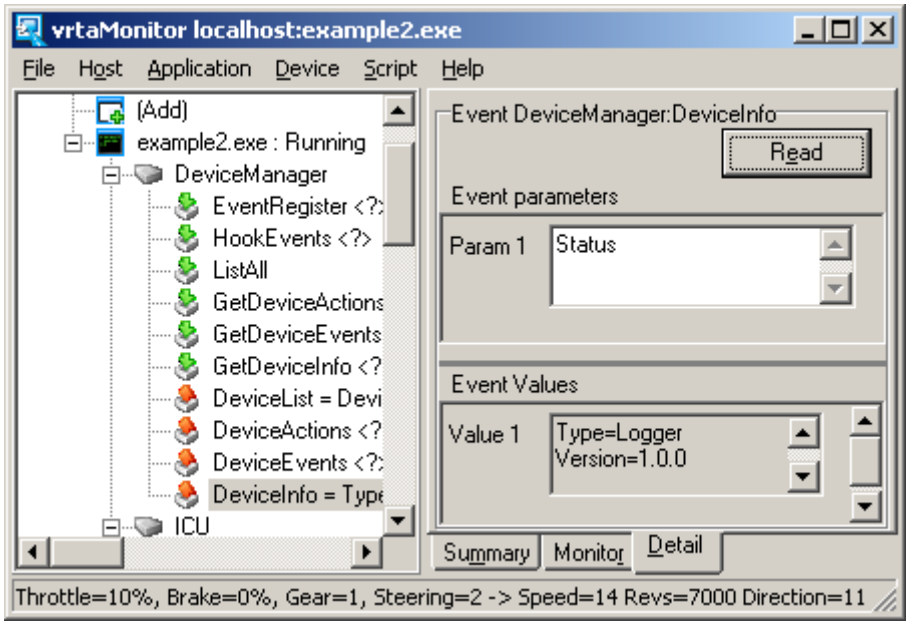
6.2 Events

6.2.1 Query

You can query the state of any event ( ) in a virtual device in the following ways.

- Double-click on an event in the tree view. If the event does not require any data then the current value of the event is read immediately (e.g. ICU / Pending). If the event requires input data (e.g DeviceManager /DeviceInfo) then `vrtaMonitor` asks you to enter it the first time round, then re-sends the same value on subsequent double-clicks.

  (If you want to change the data that gets sent, select the Params… option from the context or main Device menu or press Ctrl+Alt+R).
- Press Ctrl+R when an event is selected. This is the same as a double-click above.
- Right-click the action and select Read from the context menu.
- Select the main menu item Device / Current Event / Read.
- Go to the Detail tab on the right-hand side. You can enter data (where needed) and query the event by pressing the Read button.



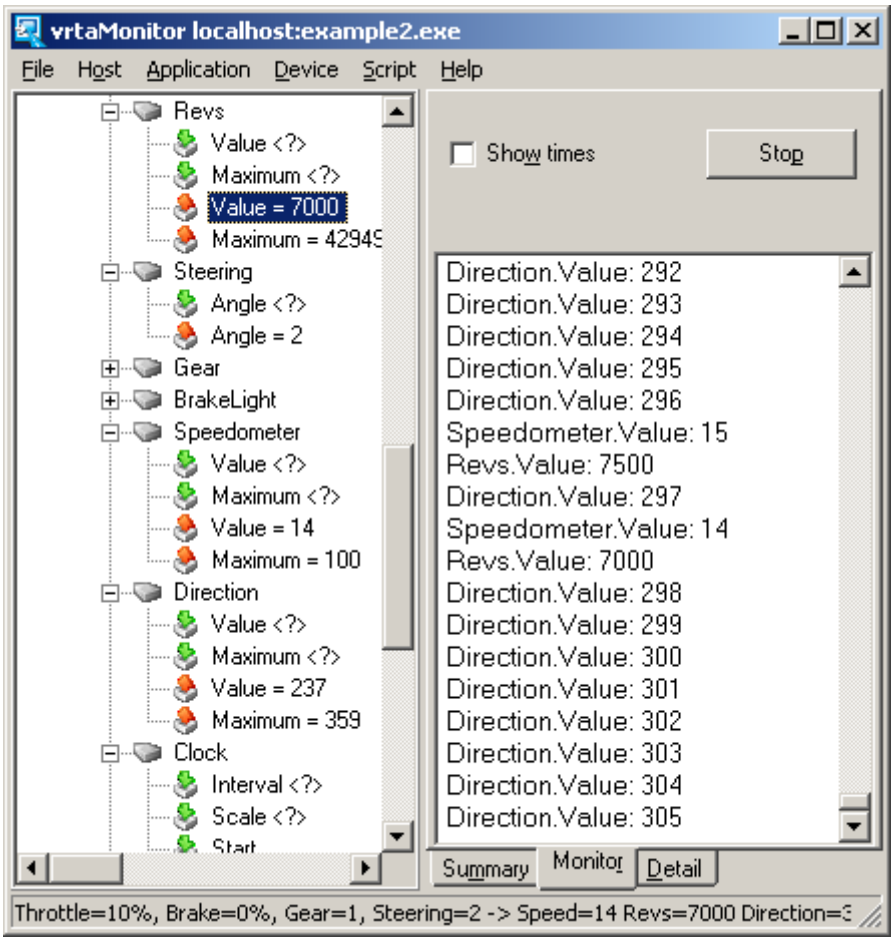
It is also possible to ask `vrtaMonitor` to query all of the events of a device automatically every second or so and update the values displayed in the tree and detail views.

You do this by selecting Auto Refresh (Ctrl+A) for the device[10].

6.2.2 Monitor

You can alternatively specify that you want to *monitor* an event rather than just querying it.

In this case, the VECU notifies `vrtaMonitor` whenever a monitored event is 'raised'[11]. All notifications are displayed in the monitor tab:



You can set up a monitor for an event using the normal application menu or context menu mechanisms, but the simplest way is just to drag the event from the tree view onto one of the right-hand side tab pages.

You can also drag a complete device across: this causes all of its events to be monitored.

---

[10] A second Ctrl+A will turn auto-refresh off again.
[11] Events are typically raised when some value in the virtual device changes.

To stop monitoring an event, just drag it (or its device) to the Stop button. Pressing the button on its own cancels all event monitors.

Scripting using vrtaMonitor

The `vrtaMonitor` command-line options can be used to support a limited form of scripting capability for VECUs. (See section 13.3 for a complete list of command line options.)

The basic scripting operations include:

- Loading a VECU
- Attaching to an existing VECU
- Sending an action to a device
- Monitoring an event
- Pausing for a set amount of time
- Waiting for a termination condition

Scripting options can be entered directly on the `vrtaMonitor` command-line, but you will probably find it easier to use a command-file using the form `vrtaMonitor @commands.txt`.

A command file is a simple text file with one option per line. A line is treated as a comment if it starts with a semi-colon, forward-slash, space or tab character. Command-files can be nested up to 5 times.

The command-line options are documented in detail in Chapter 12, but a few useful examples are presented here. They assume that you have built Application.exe and that you run '`vrtaMonitor @commands.txt`' from the directory `c:\Play\Application\MinGW\app`.

Enter the text below into `commands.txt`.

```
-k
-log=log.txt
-t1
Application.exe
```

The –k option tells `vrtaMonitor` to stop further processing of the script options if one of the following events occurs.

EVENT
FAILURE TO CONNECT TO VRTASERVER.
CANNOT ATTACH TO AN ALIAS SPECIFIED VIA –ALIAS.
CANNOT AUTO-LOAD A SPECIFIED VIRTUAL ECU.
CLOSED AS A RESULT OF A –T TIMEOUT.
FAILED TO LOAD VECU.
CLOSED AS A RESULT OF –F.
FAILED TO SEND AN ACTION OR RECEIVE AN EVENT.

The –log option causes logging information to be written to the file log.txt.

The –t1 option tells `vrtaMonitor` to run for 1 second (after processing its command-line options) before quitting.

The `Application.exe` parameter tells `vrtaMonitor` to load and run the VECU `Application.exe` (without showing its embedded GUI).

By contrast, if you use the option –d *before* naming the VECU then the VECU will load (and its devices become accessible), but the OS thread will not be started.

Similarly if you use the –g option then the VECU will show its embedded GUI:

```
-k
-log=log.txt
-t1
-d
-g
```

You can attach the monitor to a VECU that is already loaded by specifying its *alias*:

```
-k
-log=log.txt
-alias=Application.exe
```

You can send an action to a device. The commands below will attach to an existing VECU and then terminate it.

```
-k
-log=log.txt
-alias=Application.exe
-send=ApplicationManager.Terminate
```

You can also monitor events. The example below runs `Application.exe`, monitors event `PollCompare.Match` for 5000ms then terminates. The file `log.txt` contains the results from the monitor window.

```
-k
-log=log.txt
-t10
Application.exe
-mon=PollCompare.Match
-wait=5000
-send=ApplicationManager.Terminate
-quit
```

7 vrta command-line monitor

The `vrta.exe` program is a command-line version of `vrtaMonitor` that is designed to be used to manage and control the execution of VECUs from scripts / batch files.

The basic usage for `vrta` is:

```
vrta <command> [options] [arguments]
```

The command must be first because each command has different options. The available commands are:

- load
- start
- run
- pause
- resume
- reload
- kill
- list
- describe
- help

and are described in a moment. You can also enter `vrta help <command>` to get a description of each command and its options.

7.1 Options

There are 2 forms of option.

The short form starts with - and has a single letter abbreviation. If it takes a value, the value starts immediately after the abbreviation.

e.g.

```
vrta list -h192.168.1.14
```

The long form starts with two - characters and has a more descriptive name. If it takes a value, the value is separated from the name with a space.

e.g.

```
vrta list --host 192.168.1.14
```

As can be seen, some options have both a long and a short form.

7.1.1 Common options

There are a few options that are common to all (or most) commands, and these are described here.

7.1.1.1 --format

Commands such as **list** will normally send their results to the screen in a 'human friendly' way. You can use **--format xml** to change the output to XML that may be more easily read by your tools. e.g.

```
<vrta>
  <command>list</command>
  <exit_status>0</exit_status>
  <host>hostname</host>
</vrta>
```

7.1.1.2 --verbose

(Also **–v)**. Use verbose output. This may result in some extra information, depending on the command.

7.1.1.3 --host ‹name›

(Also –h‹name›) The name or IP address of the PC that you want to connect to. Defaults to 'localhost'

7.1.1.4 --file ‹file name›

The option **--file ‹file name›** allows command-line arguments to be read in from a command file.

The form **@‹file name›** may be used as an equivalent syntax for compatibility with other RTA products. The @ form can be used as the first (or only) command-line argument. This is not true of the **–file** form.

The file can contain XML or plain text.

For XML, the file should take the form below:

```
<options>
  <option identifier='format'>xml</option>
  <option identifier='core'>2</option>
```

```
</options>
```

Otherwise the file is assumed to be simple text and contain one argument per line

```
load ; Commands can be here if using the @ syntax
myecu.vrta_ecu
--format=xml ; output as XML
--core=2
```

Text following a semicolon and lines with no arguments are ignored.

Command files may include references to other command-files, provided that no command file gets referenced twice (preventing infinite recursion).

Arguments in a command-file are processed before the next argument on the command line. Arguments are processed in the order that they appear, so a later option may override an earlier one. If a command-file appears first on the command-line then its options may be overridden. If it appears last then it may override earlier options.

7.2 Load command

This command loads one or more VECUs. A VECU is specified by the path name to the executable file on the Host PC.

Loading a VECU means that the main thread is running, but the OS thread is not running.

e.g.

```
vrta load --host=MyPC /usr/x/my_vecu.vrta_vecu
```

Each loaded VECU is assigned a name (alias) by vrtaServer that gets reported when the command completes. Other commands use the alias to refer to the loaded VECU.

Options exist to select the core on which to run the OS thread (VRTA-ux only), the real-time priority for the OS thread (0…5) and the scheduling algorithm to use (VRTA-ux)

7.2.1.1 Additional option: --home <dir>

By default, a VECU that is started by vrta will be loaded with its working directory set to the same directory where the vrtaServer executable resides.

The --home option can be used to force a specific directory to be used.

The special value # can be used to represent the directory in which the VECU's executable lives.

7.3 Start command

This command starts the OS thread of one or more loaded VECUs. The VECUs must be specified by their aliases, or you can use the word 'all' to start all loaded VECUs.

e.g.

```
vrta start --host=MyPC all
```

Options exist to insert a delay between starting different VECUs.

7.4 Run command

This command loads and starts one or more VECUs. A VECU is specified by the path name to the executable file on the Host PC.

This is equivalent to running load and then start.

e.g.

```
vrta run --host=MyPC d:\my_vecu.exe
```

7.4.1.1 Additional option: --home <dir>

By default, a VECU that is started by vrta will be loaded with its working directory set to the same directory where the vrtaServer executable resides.

The --home option can be used to force a specific directory to be used.

The special value # can be used to represent the directory in which the VECU's executable lives.

7.5 Pause command

This command pauses the OS thread of one or more started VECUs. The VECUs must be specified by their aliases, or you can use the word 'all' to pause all running VECUs.

7.6 Resume command

This command resumes the OS thread of one or more paused VECUs. The VECUs must be specified by their aliases, or you can use the word 'all' to resume all paused VECUs.

7.7 Reload command

This command reloads (resets) the specified VECUs. The VECUs must be specified by their aliases, or you can use the word 'all' to reload all running VECUs.

7.8 Kill command

This command terminates the specified VECUs. The VECUs must be specified by their aliases, or you can use the word 'all' to terminate all running VECUs.

7.9 List command

This command reports information about the state of the specified VECUs. The VECUs must be specified by their aliases, or you can use the word 'all' to list all VECUs.

Options exist to select VECUs that are in a specific state

e.g.

```
vrta list --paused all
```

7.10 Describe command

This command describes the internal structure of the specified VECUs, including version information and virtual device configuration. The VECUs must be specified by their aliases, or you can use the word 'all' to describe all VECUs.

8 Migration from a VECU to Real Hardware

Assume that you have developed an application using *VRTA* and now need to migrate the application to your real hardware.

We will assume that you have obtained a version of RTA-OS that will run on your target hardware. You also need a compatible compiler and some way to load the compiled code onto your hardware.

This chapter covers the migration issues that you should expect to face.

8.1 ARXML file

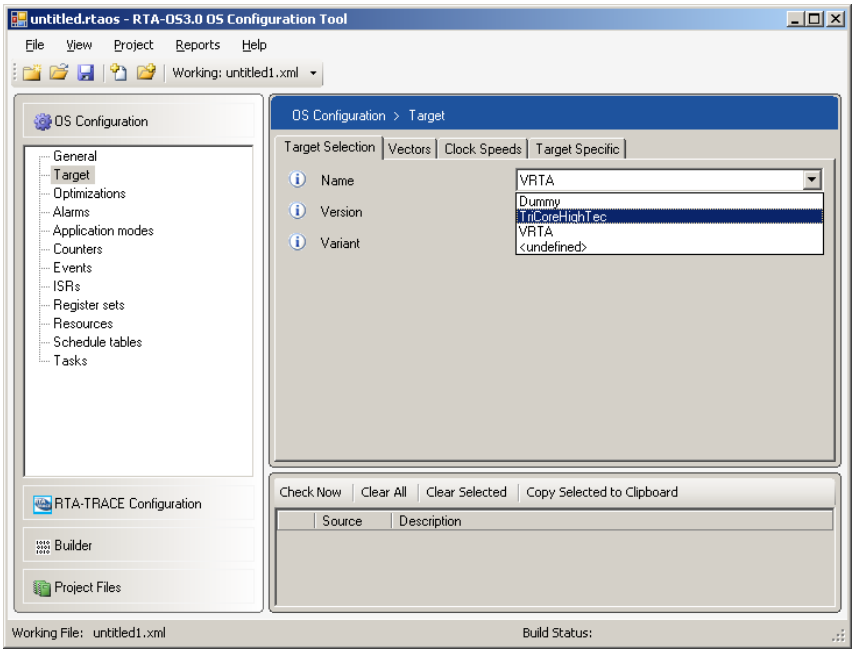
Your VECU application is described by a combination of the C source code, build scripts and the *RTA-OS* project file. The project file references one or more files encoded in AUTOSAR's ARXML syntax and is called something like *myproject.rtaos*.

If you are migrating to a non *RTA-OS* system, then you will have to refer to its documentation to discover how to encode the extra information that you might need.

Migrating to an *RTA-OS* based implementation is relatively easy. The elements that you might have to modify in your configuration.

8.1.1 Target and variant

You need to open the project in **RTAOSCfg** and then change the target and variant:



8.1.2 Interrupts

VRTA simulates an interrupt controller with a fixed number of interrupt sources, each with one a priorities. It allows you to decide in software how to map your (virtual) hardware to an interrupt source.

The interrupt controller in your target hardware will have similar capabilities, but will probably have different vectors and available priorities.

Therefore you will need to assign new target-based vector numbers and priorities for each ISR in your application[12].

8.1.3 Number of tasks

VRTA ECUs can use up to 1024 tasks. While all 32-bit ports of RTA-OS support 1024 tasks, you make be targeting a 16-bit or 8-bit device which will support fewer tasks (typically 256).

If the RTA-OS port can handle fewer tasks than your VECU currently uses, you will have to do some re-engineering of your application.

8.2 Hardware drivers

In the VECU, most if not all of your hardware is simulated via virtual devices. These obviously need to be replaced in your real application.

By a fortuitous coincidence, the inability of C code to interact directly with the C++ code has already meant that you will have written some C / C++ interface functions such as the one below.

```
//---------------------------------------
int left_pressed(void)
{
        return Left.Value();
}
```

You simply provide different implementations for each of these interface functions so that they map onto your target hardware.

e.g.

[12] Unless you were far-sighted enough to configure your VECU to use the same interrupt model as in your target ECU.

```
//----------------------------------------
int left_pressed(void)
{
        return inp(0x1001);
}
```

As long as you take care that each interface function has the same behavior as in the virtual device, this part of the migration should be straightforward.

8.3 Initialization

You will need to add code to your application to initialize your ECU's hardware. In particular you may need to configure the interrupt controller and any clock/compare devices. In effect you will need to provide a hardware specific version of the `InitializeDevices()` function used in *VRTA* applications. You must refer to the user-guide for your hardware for the best way to do this.

8.4 Interrupts

Depending on your interrupt controller and the ECU hardware, you may need to add code to your ISRs that tells the hardware that the interrupt source has been serviced.

You can use macros to implement conditional compilation of ISR code so that it can adapt to different platforms.

e.g.

```
ISR(isrAccel){
     change_speed(+1);

#ifdef TARGET_VRTA
     // Nothing needed to clear interrupt for VRTA
#endif

#ifdef TARGET_HC12X
     output(0x12,99);
#endif
}
```

8.5 Building the application

You should refer to the script that is supplied with the example program provided with the *RTA-OS* version for a guide as to what is needed to build the application. Essentially you need to build an OS library and link against it.

9 VRTA API Reference

This chapter gives a detailed description of the VRTA API calls, listed in alphabetical order.

9.1 General notes

9.1.1 API Header Files

The file `vrtaCore.h` must be included to use the API calls listed in this chapter. `vrtaCore.h` contains prototype declarations for the API calls described here. It also file #includes the files `vrtaTypes.h` and `vrtaVM.h`.

9.1.2 Linkage

Unless specified otherwise all API calls use C linkage (i.e. no C++ name mangling) and so may be called from C or C++ source.

9.2 Common Data Types

These data types are all declared in `vrtaTypes.h`.

9.2.1 vrtaDevID

A scalar value that contains the ID of a virtual device. Devices 0, 1 and 2 are the standard VRTA devices DeviceManager, ICU and ApplicationManager respectively.

9.2.2 vrtaActionID

A scalar value that contains the ID of an action in a virtual device. Actions IDs start at 1.

9.2.3 vrtaEventID

A scalar value that contains the ID of an event in a virtual device. Events IDs start at 1.

9.2.4 vrtaActEvID

A scalar value that contains the ID of an action or event in a virtual Device.

9.2.5 vrtaISRID

A scalar value that contains the number of an ISR. ISRs range from 1 to 32, but a `vrtaISRID` can sometimes be set to zero to mean 'no ISR'.

9.2.6 vrtaIntPriority

A scalar value that contains an interrupt priority. Priorities from zero (no ISR) to 32.

9.2.7 vrtaErrType

This scalar value gets used as a status return type by many of the API functions. It can take one of the values:

Value	Description
RTVECUErr_NONE	No error / success
RTVECUErr_Dev	Device fault. Typically invalid device ID
RTVECUErr_ID	ID fault. Typically invalid action or event ID
RTVECUErr_VAL	Value fault. Typically value is out of range.
RTVECUErr_Conn	Connection fault. Occurs with remote monitor applications if the link to the VECU fails.

9.2.8 vrtamillisecond

A scalar representing an *interval* in milliseconds.

9.2.9 vrtaTimestamp

A scalar representing the *current ECU time* in microseconds. This is normally the number of microseconds since (just before) `OS_MAIN()` was called[13].

9.2.10 vrtaBoolean

The scalar type `vrtaBoolean` is used to represent boolean values. In this document a `vrtaBoolean` type is taken to be “true” when it is non-zero and “false” when it is zero.

9.2.11 vrtaByte

Represents a single byte of data passed into or out of a device. Normally part of an array of bytes.

[13] If a timestamp is taken before OS_MAIN(), then the time recorded is the number of microseconds since the VM was loaded. This allows events to be timed where a VECU is started in slave mode.

9.2.12 vrtaDataLen

A scalar that represents the size (in bytes) of some data being passed into or out of a device. The maximum value that this can take is given by the value of the macro `vrtaDataLenMax`. (Currently 0xffff.)

9.2.13 vrtaEmbed

`vrtaEmbed` is a 'C' union containing the following fields:

Field Type	Field Name
int	iVal
unsigned	uVal
double	dVal
vrtaByte	bVal[16]

`vrtaEmbed` is used to support data passing operations in and out of virtual devices via `vrtaAction` and `vrtaEvent` (described in a moment). Both of these data structures embed an instance of `vrtaEmbed` within themselves.

Whenever the amount of data passed in or out of an action or event will fit inside an instance of `vrtaEmbed`, then the data *must be* passed in it.

In all common situations the data passed easily fits within the `vrtaEmbed` instance, so low-overhead code such as this is common:

```
thisEvent.devEmbeddedData.uVal = 32;
```

It is only where larger amounts of data need to be passed that we need to worry about allocating data buffers and data ownership issues.

9.2.14 vrtaAction

The `vrtaAction` structure is used to pass data value(s) to a specific action in a virtual device. The fields in `vrtaAction` are:

DECLARATION (TYPE AND NAME)	DESCRIPTION
`vrtaDevID devID`	THE ID OF THE DEVICE CONTAINING THE ACTION.
`vrtaActionID devAction`	THE ID OF THE ACTION.
`vrtaDataLen devActionLen`	THE NUMBER OF BYTES OF DATA. THIS CAN

	BE ZERO. IF ITS VALUE IS FROM 1 TO 16 INCLUSIVE, THEN THE DATA IS PRESENT IN THE `DEVEMBEDDEDDATA` UNION. IF IT IS MORE THAN 16 THEN `DEVEVENTDATA` CONTAINS THE ADDRESS OF THE DATA (SEE NOTE BELOW).
`const void * devActionData`	IF `DEVEVENTLEN` IS FROM ZERO TO 16 INCLUSIVE THEN `DEVEVENTDATA` MUST EITHER BE NULL OR THE ADDRESS OF `DEVEMBEDDEDDATA`. IF THERE ARE MORE THAN 16 BYTES OF DATA THEN THE STORAGE FOR THE INPUT DATA MUST BE PROVIDED BY THE CREATOR OF THE `VRTAACTION` AND `DEVACTIONDATA` MUST POINT TO THIS STORAGE. IF YOU ALLOCATE MEMORY FOR THE SENT DATA, THEN YOU NEED TO DECIDE WHETHER TO FREE THE MEMORY IN THE RECEIVER OR THE SENDER. OTHERWISE YOU MAY END UP LEAKING MEMORY. WHERE `VRTASENDACTION()` IS CALLED TO SEND DATA TO AN ACTION, ANY DATA REFERENCED BY `DEVACTIONDATA` MUST NOT CHANGE DURING THE CALL.
`vrtaEmbed devEmbeddedData`	THE UNION THAT CONTAINS THE DATA WHERE IT IS NO LARGER THAN 16 BYTES.
`vrtaTimestamp devTimeStamp`	THIS FIELD IS UPDATED BY VRTA JUST BEFORE PASSING THE ACTION TO THE DEVICE.
DECLARATION (TYPE AND NAME)	DESCRIPTION
`vrtaDevID devID`	THE ID OF THE DEVICE CONTAINING THE ACTION.
`vrtaActionID devAction`	THE ID OF THE ACTION.
`vrtaDataLen devActionLen`	THE NUMBER OF BYTES OF DATA. THIS CAN BE ZERO. IF ITS VALUE IS FROM 1 TO 16 INCLUSIVE,

	THEN THE DATA IS PRESENT IN THE `DEVEMBEDDEDDATA` UNION. IF IT IS MORE THAN 16 THEN `DEVEVENTDATA` CONTAINS THE ADDRESS OF THE DATA (SEE NOTE BELOW).
`const void * devActionData`	IF `DEVEVENTLEN` IS FROM ZERO TO 16 INCLUSIVE THEN `DEVEVENTDATA` MUST EITHER BE NULL OR THE ADDRESS OF `DEVEMBEDDEDDATA`. IF THERE ARE MORE THAN 16 BYTES OF DATA THEN THE STORAGE FOR THE INPUT DATA MUST BE PROVIDED BY THE CREATOR OF THE `VRTAACTION` AND `DEVACTIONDATA` MUST POINT TO THIS STORAGE. IF YOU ALLOCATE MEMORY FOR THE SENT DATA, THEN YOU NEED TO DECIDE WHETHER TO FREE THE MEMORY IN THE RECEIVER OR THE SENDER. OTHERWISE YOU MAY END UP LEAKING MEMORY. WHERE `VRTASENDACTION()` IS CALLED TO SEND DATA TO AN ACTION, ANY DATA REFERENCED BY `DEVACTIONDATA` MUST NOT CHANGE DURING THE CALL.
`vrtaEmbed devEmbeddedData`	THE UNION THAT CONTAINS THE DATA WHERE IT IS NO LARGER THAN 16 BYTES.
`vrtaTimestamp devTimeStamp`	THIS FIELD IS UPDATED BY VRTA JUST BEFORE PASSING THE ACTION TO THE DEVICE.

9.2.15 vrtaEvent

The `vrtaEvent` structure is used to pass state information about a specific event in a virtual device. The fields in `vrtaEvent` are:

DECLARATION (TYPE AND NAME)	DESCRIPTION
`vrtaDevID devID`	THE ID OF THE DEVICE CONTAINING THE EVENT.
`vrtaEventID devEvent`	THE ID OF THE EVENT.

`vrtaEventID devEventLen`	THE NUMBER OF BYTES OF DATA. THIS CAN BE ZERO. IF ITS VALUE IS FROM 1 TO 16 INCLUSIVE, THEN THE DATA IS PRESENT IN THE `DEVEMBEDDEDDATA` UNION. IF IT IS MORE THAN 16 THEN `DEVEVENTDATA` CONTAINS THE ADDRESS OF THE DATA (SEE NOTE BELOW).
`const void * devEventData`	IF `DEVEVENTLEN` IS FROM ZERO TO 16 INCLUSIVE THEN `DEVEVENTDATA` MUST EITHER BE NULL OR THE ADDRESS OF `DEVEMBEDDEDDATA`. IF THERE ARE MORE THAN 16 BYTES OF DATA THEN THE STORAGE FOR THE INPUT DATA MUST BE PROVIDED BY THE CREATOR OF THE `VRTAEVENT` AND `DEVEVENTDATA` MUST POINT TO THIS STORAGE. SEE THE NOTE BELOW ON DATA OWNERSHIP.
`vrtaEmbed devEmbeddedData`	THE UNION THAT CONTAINS THE DATA WHERE IT IS NO LARGER THAN 16 BYTES.
`vrtaTimestamp devTimeStamp`	THIS FIELD IS UPDATED BY VRTA TO SHOW ONE OF: THE TIME THAT A QUERY WAS MADE INTO `VRTAGETSTATE()`. THE TIME THAT THE DATA WAS PASSED TO `VRTARAISEEVENT()`.

Ownership of `devEventData`:

- Where the current value of an event is being queried, data can be passed *into* the event via `vrtaGetState()`. The caller of `vrtaGetState()` ensures that any data referenced by `devEventData` does not change during the call.
- Where the current value of an event is being queried via `vrtaGetState()`, the data that is passed *out of* the event via `devEventData` may not change from the time that the call returns up to *the next time that* `vrtaGetState()` *is called from the same thread.* This can clearly be very complicated to achieve. However this is almost never

necessary in real applications. Most return data fits within `devEmbeddedData`, and in most other cases the data does not change anyway[14].

- Where an event is being raised via `vrtaRaiseEvent()`, the device may supply data to its listeners. Listeners must take a copy of any data that they need, so the caller of `vrtaRaiseEvent()` only has to ensure that the data does not change during the call.

9.2.16 vrtaTextPtr

A pointer to a simple ASCIIZ string.

e.g.

```
vrtaTextPtr tp = "Hello World";
```

9.2.17 vrtaStringlistPtr

A pointer to an ASCIIZ string that comprises zero or more \n separated list items.

e.g.

```
vrtaStringlistPtr lp = "One\nTwo\nThree";
```

9.2.18 vrtaOptStringlistPtr

A pointer to an ASCIIZ string that comprises zero or more \n separated option items each with the form ‹name› = ‹value›.

e.g.

```
vrtaOptStringlistPtr op = "Name=Bill\nAge=51\nWife= Melinda";
```

9.3 Data format Strings

9.3.1 Overview

Virtual device actions and events commonly have some data associated with them.

For example the ICU's Raise action has to be passed an integer in the range 1 to 32 (VRTA-win). The ApplicationManager's State event supplies a value that represents the OS thread state (Loaded I Running I Paused I Terminating I Resetting).

Within the VECU, all data is handled in native machine format, i.e. an 'int' for an integer value. *It is your responsibility* in VECU code to send data of the correct type

[14] For example the DeviceEvents event in the DeviceManager often returns string data that is larger than 16 bytes. The strings that it returns are all allocated during initialization and do not change during the application, so no special protection is necessary.

between actions and events. As long as you trust your code to pass the right type of data, you can choose to omit range checks within your devices[15].

While this is a reasonable assumption in the C and C++ code that gets compiled into the VECU, it clearly does not hold where external programs such as `vrtaMonitor` access the data. For this reason a device must supply a description of the data that is used by its actions and events. This information is used by remote programs to format the data sent to a device and interpret the data it sends back. VRTA performs size and range checking on data from remote programs so that it does not have to be done by each device itself.

Example format strings are:

STRING	DESCRIPTION
%D	32-BIT SIGNED INTEGER
%D:;1;32	INTEGER THAT CAN TAKE VALUES 1 THROUGH 32.
%B	32-BIT INTEGER, NORMALLY REPRESENTED IN BINARY BY A MONITOR PROGRAM.
LOADEDIRUN-NINGIPAUSEDITERMINAT-INGIRESETTING	32-BIT VALUE WITH VALUES ZERO THROUGH 4 THAT IS NORMALLY REPRESENTED BY A MONITOR PROGRAM AS ONE OF THE I SEPA-RATED STRINGS
%D,%D,%D	A STRUCTURE COMPRISING 3 32-BIT SIGNED INTEGERS.

9.3.2 Definition

A data format string consists of one or more data-item descriptors. If there are multiple data-item descriptors then they are separated by ','s. The data-item descriptors are as follows (text inside "`[]`" is optional):

DATA-ITEM DESCRIPTOR	DESCRIPTION
`%d[:<cons>]`	THE DATA-ITEM IS A SIGNED NUMBER. BY DEFAULT THIS IS STORED IN 32 BITS (A C `INT`).

[15] By all means add range checking code if you wish. Our design aim is to allow (but not force) devices to have a very small run-time overhead.

	THE RANGE IS DETERMINED BY THE DATA-ITEM SIZE UNLESS THERE IS A CONSTRAINT.
%u[:<cons>]	THE DATA-ITEM IS AN UNSIGNED NUMBER. BY DEFAULT THIS IS STORED IN 32 BITS (A C UNSIGNED). THE RANGE IS DETERMINED BY THE DATA-ITEM SIZE UNLESS THERE IS A CONSTRAINT.
%f[:<cons>]	THE DATA-ITEM IS FLOATING-POINT NUMBER. BY DEFAULT THIS IS STORED IN 64 BITS (A C DOUBLE). THE RANGE IS DETERMINED BY THE DATA-ITEM SIZE UNLESS THERE IS A CONSTRAINT.
%x[:<cons>]	THE DATA-ITEM IS AN UNSIGNED NUMBER THAT SHOULD REPRESENTED IN HEXIDECIMAL. BY DEFAULT THIS IS STORED IN 32 BITS (A C UNSIGNED). THE RANGE IS DETERMINED BY THE DATA-ITEM SIZE UNLESS THERE IS A CONSTRAINT.
%b[:<cons>]	THE DATA-ITEM IS AN UNSIGNED NUMBER THAT SHOULD REPRESENTED IN BINARY. BY DEFAULT THIS IS STORED IN 32 BITS (A C UNSIGNED). THE RANGE IS DETERMINED BY THE DATA-ITEM SIZE UNLESS THERE IS A CONSTRAINT.
<a>|<b>|…[:<cons>]	THE DATA-ITEM IS AN UNSIGNED NUMBER THAT SHOULD REPRESENTED AS A SERIES OF ENUMERATION VALUES. ENUMERATION VALUE <A> CORRESPONDS TO THE NUMBER 0, ENUMERATION VALUE <B> TO THE NUMBER 1, AND SO ON. BY DEFAULT THIS IS STORED IN 32 BITS (A C UNSIGNED).
%s[:<size>]	THE DATA-ITEM IS AN ASCII STRING – WHICH MAY OR MAY NOT HAVE A TRAILING '\0'. IF NO <SIZE> VALUE IS GIVEN THEN THE SIZE OF THE STRING IS INFERRED FROM THE LENGTH OF THE ACTION OR EVENT DATA. IF A <SIZE> VALUE IS GIVEN THEN IT SPECIFIES THE SIZE OF THE STRING (INCLUDING A TRAILING '\0' IF THERE IS ONE).

`%a[:<size>]`	THE DATA-ITEM IS AN ARRAY OF BYTES. IF NO `<SIZE>` VALUE IS GIVEN THEN THE SIZE OF THE ARRAY IS INFERRED FROM THE LENGTH OF THE ACTION OR EVENT DATA. IF A `<SIZE>` VALUE IS GIVEN THEN IT SPECIFIES THE SIZE OF THE ARRAY.

A data-item descriptor may optionally include a constraint ‹cons›. A constraint has the form: [‹bits›[!‹width›]][;‹min›;‹max›]. Where:

Constraint Item	Description
‹bits›	The number of bits used to store a numeric value. This can be 8, 16, 32 or 64.
‹width›	The number of bytes between the start of the data-item and the start of the next data-item.
‹min› and ‹max›	The minimum and maximum values that may be stored in a numeric data-item.

9.3.2.1 Examples

`"%d"`

There is a single signed number that will be stored in 32 bits (a C `int`).

`"%u:;1;10"`

There is an unsigned number in the range 1 to 10 inclusive that will be stored in 32 bits (a C `unsigned`).

`"%x:16,%b:8"`

There is an unsigned number that will be stored in 16 bits (a C `unsigned short`) and should be displayed in hexidecimal. This is immediately followed by an unsigned number that will be stored in 8 bits (a C `unsigned char`) and should be displayed in binary.

`"%s:10,%u:8!4,%u:64;1;100"`

There is a 10 character string. This is followed by an unsigned number stored in 8 bits. There are then 3 bytes of padding; since the next data-item is stored 4 bytes after the start of the 8 bit value. This padding is followed by an unsigned number in the range 1 to 100 inclusive stored in 64 bits

9.4 API Functions

Each API call is described in this section using the following standard format:

9.4.1.1 The title gives the name of the API call.

A brief description of the API call is provided.

9.4.1.2 Function declaration:

Interface in C syntax.

9.4.1.3 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
`Parameter Name`	INPUT/OUTPUT	DESCRIPTION.

9.4.1.4 Description:

Explanation of the API call functionality.

9.4.1.5 Return values:

VALUE	DESCRIPTION
`RETURN VALUES.`	DESCRIPTION OF RETURN VALUE.

9.4.1.6 Notes:

Usage restrictions and notes for the API call.

9.4.1.7 See also:

List of related API calls.

9.4.2 InitializeDevices()

Device initialization hook function.

9.4.2.1 Function declaration:

```
void InitializeDevices(void)
```

9.4.2.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
`<none>`		

9.4.2.3 Description:

You must provide the `InitializeDevices()` hook function in your application code. It is called by the OS thread immediately before it calls `OS_MAIN()`. `InitializeDevices()` is normally used to execute code that carries out initialization of virtual devices. By the time that `InitializeDevices()` is called, each virtual device will have been sent a Reset action to inform it that the OS thread is about to start.

9.4.2.4 Return values:

VALUE	DESCRIPTION
`<none>`	

9.4.2.5 Notes:

This function has C++ linkage and so must be implemented in a C++ source module, typically `devices.cpp`.

9.4.2.6 See also:

`OS_MAIN()`
`vrtaStart()`

9.4.3 OS_MAIN()

The entry-point for the OS thread .

9.4.3.1 Function declaration:

```
OS_MAIN()
```

9.4.3.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
`<none>`		

9.4.3.3 Description:

The `OS_MAIN()` function is provided by the Virtual ECU application code and is the entry-point for the OS thread.

Typically an application will make some initialization calls and then start the OS kernel via `StartOS()`. The call does not return.

If the Virtual ECU has been loaded in autostart mode (the default) then the complete VECU will terminate automatically.

If, however, the VECU was been loaded in slave mode then the VECU will wait for a Terminate action to be received by the Application Manager. This allows the state of the VECU's devices to be queried after the OS thread has terminated.

9.4.3.4 Return values:

VALUE	DESCRIPTION
<none>	

9.4.3.5 Notes:

`OS_MAIN()` normally has the structure shown below in a VRTA application. The call to `vrtaIsAppFinished()` should be made so that external requests to terminate the program get recognized promptly. If you don't quit `OS_MAIN()` when such a request occurs then VRTA will forcibly terminate the OS thread after a few seconds.

```
OS_MAIN()
{
        initialize_something();
        StartOS(OSDEFAULTAPPMODE);
}
```

9.4.3.6 See also:

`InitializeDevices()`
`vrtaStart()`
`vrtaIsAppFinished()`
`vrtaIsIdle()`

9.4.4 vrtaEnterUninterruptibleSection()

Enter a critical section that cannot be interrupted.

9.4.4.1 Function declaration:

```
void vrtaEnterUninterruptibleSection(void)
```

9.4.4.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
<none>		

9.4.4.3 Description:

(VRTA-win only)

This function enters a critical section. Only one thread at a time may be in the critical section.

Calling `vrtaEnterUninterruptibleSection()` will block the calling thread if another thread is already in the critical section.

If the OS thread calls `vrtaEnterUninterruptibleSection()` then it cannot be interrupted until it leaves the critical section.

Note

If the OS thread needs to call any Windows API function or non-reentrant C/C++ runtime library function then it must call `vrtaEnterUninterruptibleSection()` before making the call and `vrtaLeaveUninterruptibleSection()` afterwards. Windows API functions and non-reentrant C/C++ runtime library functions cannot cope with the stack manipulation that occurs when a VRTA interrupt executes.

9.4.4.4 Return values:

VALUE	DESCRIPTION
`<none>`	

9.4.4.5 Notes:

None.

9.4.4.6 See also:

`vrtaLeaveUninterruptibleSection()`

9.4.5 vrtaEventRegister()

Register an event handler.

9.4.5.1 Function declaration:

```
vrtaEventListener vrtaEventRegister(
                    vrtaEventCallback eCallback,
                    const void        *tag)
```

9.4.5.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
`eCallback`	INPUT	A POINTER TO AN EVENT HANDLING FUNCTION.
`tag`	INPUT	A CALLER PROVIDED VALUE THAT WILL BE PASSED AS AN ARGUMENT TO `ECALLBACK`.

9.4.5.3 Description:

This API call registers an event hook callback routine with VRTA. The `vrtaEvent-Listener` handle is needed when calling `vrtaHookEvent()` so that it can identify `eCallback` as the function to call when the specified event is raised.

`eCallback` is of type `vrtaEventCallback` which is defined as follows:

```
typedef vrtaErrType (*vrtaEventCallback)(
    const void *instance, const vrtaEvent *event);
```

When `eCallback` is called its `instance` argument will be set to the `tag` argument passed to `vrtaEventRegister()` and its `event` argument will contain the event raised[16]. `eCallback` should always return `RTVECUErr_NONE`.

The content of the `vrtaEvent` structure pointed to by `event` is only valid for the duration of the call to `eCallback`. If you need to use this data after `eCallback` has returned then you must take a copy of the data.

9.4.5.4 Return values:

VALUE	DESCRIPTION
<a handle>	A HANDLE FOR THE EVENT HANDLER.

9.4.5.5 Notes:

The event hook callback function gets called during the execution of `vrtaRaiseEvent()`. Your function must be thread-safe because it is quite normal for devices to raise events from threads that are independent of the OS thread. Any event that gets provoked from an external monitor application *will* be in a different thread.

9.4.5.6 See also:

`vrtaEventUnregister()`
`vrtaHookEvent()`
`vrtaRaiseEvent()`

9.4.6 vrtaEventUnregister()

Unregister an event handler.

9.4.6.1 Function declaration:

```
vrtaErrType vrtaEventUnregister(
                vrtaEventListener listener)
```

[16] See the description of vrtaGetState() for more information about the vrtaEvent type.

9.4.6.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
`listener`	INPUT	AN EVENT-HANDLER HANDLE RETURNED BY `VRTAEVENTREGISTER()`.

9.4.6.3 Description:

This API call unregisters an event handler previously registered with `vrtaEventRegister()`. Any events that have been hooked by the event handler are unhooked.

9.4.6.4 Return values:

VALUE	DESCRIPTION
`RTVECUErr_NONE`	THE API CALL WAS SUCCESSFUL.
`RTVECUErr_VAL`	THE `LISTENER` ARGUMENT IS INVALID.

9.4.6.5 Notes:

This API cannot be called from within an event handler.

9.4.6.6 See also:

```
vrtaEventRegister()
vrtaHookEvent()
vrtaRaiseEvent()
```

9.5 vrtaGetState()

Query the current state (value) of an event.

9.5.1.1 Function declaration:

```
vrtaErrType vrtaGetState(vrtaDevID id, vrtaEvent *ev)
```

9.5.1.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
`id`	INPUT	THE ID OF THE VIRTUAL DEVICE TO BE QUERIED.
`ev`	INPUT/OUTPUT	A POINTER TO THE STRUCTURE THAT SPECIFIES THE EVENT AND ITS DATA.

9.5.1.3 Description:

This API is used to obtain the current value of an event supported by a virtual device. Virtual devices raise events at appropriate times and these can be multicast to interested receivers. However, sometimes it is useful to be able to discover the "current value" of an event. This makes most sense for events that contain data. For example, one may wish to discover the current interrupt priority (IPL) level of the interrupt control unit rather than waiting for an event to be raised when the IPL changes. Events which do not contain data but simply indicate that something has happened can still be queried, but it is not really useful so to do.

When calling `vrtaGetState()`, you must set the correct device and event IDs in `ev`. If the event needs to be passed some data as part of the query (e.g. the name of the device for the DeviceManager's DeviceAction event), then the data must be set up before the call. If no data is needed, set the `devEventLen` field to zero.

On successful return from `vrtaGetState()`, the data in `ev` now references the current value of the event.

9.5.1.4 Return values:

VALUE	DESCRIPTION
`RTVECUErr_NONE`	THE API CALL WAS SUCCESSFUL.
`RTVECUErr_Dev`	THE SPECIFIED DEVICE ID IS INVALID.
`RTVECUErr_ID`	THE SPECIFIED EVENT ID IS INVALID.
`RTVECUErr_VAL`	THE DATA PROVIDED IN THE EVENT IS INVALID (OUT OF RANGE?).

9.5.1.5 Notes:

During the call of `vrtaGetState()` VRTA passes the `vrtaEvent` structure to the device. The device determines the event's value (possibly using the input data) and either copies the event data into the `devEmbeddedData` field or places the data in storage that it has allocated and sets the `devEventData` field to point to this storage. The queried device is responsible for managing any storage that it allocates. Refer to section 9.2.15 for details.

9.5.1.6 See also:

```
vrtaRegisterVirtualDevice()
vrtaRaiseEvent()
vrtaEventRegister()
vrtaEventUnregister()
vrtaHookEvent()
```

9.5.2 vrtaHookEvent()

Hook or unhook an event so that an event handler is or is not called when the event is raised.

9.5.2.1 Function declaration:

```
vrtaErrType vrtaHookEvent(vrtaEventListener listener,
                          vrtaDevID dev,
                          vrtaEventID ev,
                          vrtaBoolean capture)
```

9.5.2.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
listener	INPUT	AN EVENT-HANDLER HANDLE RETURNED BY VRTAEVENTREGISTER().
dev	INPUT	THE ID OF THE DEVICE.
ev	INPUT	THE ID OF THE EVENT.
capture	INPUT	TRUE TO HOOK AN EVENT, FALSE TO UNHOOK AN EVENT.

9.5.2.3 Description:

If capture is true then this call 'hooks' the event so that the event handler associated with listener is called when the event gets raised.

If capture is false then this call unhooks one or more events previously hooked with this API call. The behavior of the call when capture is false depends on the values of dev and ev as follows:

DEV	EV	RESULT
0	0	ALL EVENT HOOKS ARE REMOVED FROM LISTENER.
Non-zero	0	ALL EVENT HOOKS FOR EVENTS OWNED BY THE SPECIFIED DEVICE ARE REMOVED FROM LISTENER.
Non-zero	Non-zero	THE EVENT HOOK FOR THE SPECIFIED EVENT IS REMOVED FROM LISTENER.

9.5.2.4 Return values:

VALUE	DESCRIPTION
`RTVECUErr_NONE`	THE API CALL WAS SUCCESSFUL.
`RTVECUErr_Dev`	THE SPECIFIED DEVICE ID IS INVALID.
`RTVECUErr_ID`	THE SPECIFIED EVENT ID IS INVALID.
`RTVECUErr_VAL`	THE `LISTENER` ARGUMENT IS INVALID OR CALLED FROM INSIDE AN EVENT HANDLER.

9.5.2.5 Notes:

This API may not be called from inside an event handler.

9.5.2.6 See also:

```
vrtaEventRegister()
vrtaEventUnregister()
vrtaRaiseEvent()
```

9.5.3 vrtaInitialize()

Initialize VRTA.

9.5.3.1 Function declaration:

```
void vrtaInitialize(int argc,
                    char* argv[],
                    const vrtaVectorTable* vecTable)
```

9.5.3.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
`argc`	INPUT	THE NUMBER OF COMMAND LINE ARGUMENTS ON THE VIRTUAL ECU'S COMMAND LINE.
`argv`	INPUT	THE ARRAY OF COMMAND LINE ARGUMENTS FROM THE VIRTUAL ECU'S COMMAND LINE.
`vecTable`	INPUT	A POINTER TO THE INTERRUPT VECTOR TABLE.

9.5.3.3 Description:

This API call is used to initialize VRTA – it must be called after `vrtaLoadVM()` and before `vrtaStart()`.

For an RTA-OS VECU, this API is called automatically for you.

The `argc` and `argv` arguments should be taken directly from the `argc` and `argv` arguments of the Virtual ECU's `main()` entry-point.

`vectTable` points to a `vrtaVectorTable` structure containing the interrupt vector table. `vrtaVectorTable` is defined as follows:

```
#define RTVECU_NUM_VECTORS  (32)
typedef struct {
    unsigned      numVectors;
    vrtaIntVector vectors[RTVECU_NUM_VECTORS];
} vrtaVectorTable;
```

The `numVectors` field must be 32. This is checked during `vrtaInitialize()` and a fatal error generated if the value is not 32. `vectors[]` is an array of 32 interrupt vectors. The interrupt control unit (ICU) numbers interrupt vectors from 1 to 32 (0 is used to mean no interrupt). ICU interrupt vector number 1 corresponds to `vectors[0]`, ICU interrupt vector number 2 corresponds to `vectors[1]`, and so on up to ICU interrupt vector number 32 which corresponds to `vectors[31]`.

Each interrupt vector is defined as follows:

```
typedef struct {
    vrtaIntHandler  handler;
    vrtaIntPriority priority;
    vrtaAppTag      tag;
} vrtaIntVector;
```

The `handler` field points to the interrupt handler to be run when the corresponding interrupt arrives. `priority` is the priority of the corresponding interrupt – this must be a number in the range 1 to 32 inclusive (1 is the lowest priority and 32 is the highest priority). `tag` is application data that that is passed to the interrupt handler when it is called.

An interrupt handler has the following definition:

```
typedef void (*vrtaIntHandler)(vrtaAppTag tag,
    vrtaIntPriority oldIPL);
```

When an interrupt handler is called its `tag` argument is set to the `tag` argument in the corresponding interrupt vector and its `oldIPL` argument is set to the priority of the interrupted code – zero for code not running in an interrupt handler or the priority of the interrupt for code running in an interrupt handler.

A trivial example of starting a Virtual ECU might look like:

```
void IntHandler(vrtaAppTag tag,
                vrtaIntPriority oldIPL) {
    /* Handle interrupt. */
}

vrtaVectorTable IntVectors = {
    RTVECU_NUM_VECTORS,
    {{IntHandler, 1, (vrtaTag) 1},
     {IntHandler, 2, (vrtaTag) 2},
/* < ...snip... > */
     {IntHandler, 32, (vrtaTag) 32}}
};

OS_MAIN() {
    /* The OS thread starts here. */
/* < ...snip... > */
}

void main(int argc, char * argv[]) {
    vrtaLoadVM();
    vrtaInitialize(argc, argv, &IntVectors);
    vrtaStart();
    /* Control returns here when the application
     * thread terminates. */
}
```

9.5.3.4 Return values:

VALUE	DESCRIPTION
<none>	

9.5.3.5 Notes:

If a Virtual ECU is using RTA-OS then this API should not be called explicitly as it is called automatically by the `main()` program.

9.5.3.6 See also:

`vrtaLoadVM()`
`vrtaStart()`

9.6 vrtaIsAppFinished()

Determine if the OS thread has terminated or is about to terminate.

9.6.1.1 Function declaration:

```
vrtaBoolean vrtaIsAppFinished(void)
```

9.6.1.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
<none>		

9.6.1.3 Description:

This API call returns true if the OS thread has terminated or is about to terminate, or false otherwise. This API call may be used by the OS thread to discover if it is about to be forcibly terminated – e.g. because another thread has called `vrtaTerminate()` or a Terminate action has been sent to the ApplicationManager Device. This API may also be used in *VRTA* threads to discover if they should terminate themselves.

9.6.1.4 Return values:

VALUE	DESCRIPTION
`true`	THE OS THREAD HAS TERMINATED OR IS ABOUT TO TERMINATE.
`false`	THE OS THREAD HAS NOT TERMINATED AND IS NOT ABOUT TO TERMINATE.

9.6.1.5 Notes:

None.

9.6.1.6 See also:

`vrtaSpawnThread()`

9.6.2 vrtaIsAppThread()

Determine if the calling thread is the OS thread.

9.6.2.1 Function declaration:

```
vrtaBoolean vrtaIsAppThread(void)
```

9.6.2.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
`<none>`		

9.6.2.3 Description:

This API call returns true if the calling thread is the OS thread or false if the calling thread is not the OS thread.

9.6.2.4 Return values:

VALUE	DESCRIPTION
`true`	THE CALLING THREAD IS THE OS THREAD.
`false`	THE CALLING THREAD IS NOT THE OS THREAD.

9.6.2.5 Notes:

None.

9.6.2.6 See also:

None.

9.6.3 vrtaIsIdle()

Report that the processor is in the OS idle task/callback.

9.6.3.1 Function declaration:

```
void vrtaIsIdle(vrtamillisecond msecs)
```

9.6.3.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
`msecs`	INPUT	THE NUMBER FOR MILLISECONDS TO YIELD FOR.

9.6.3.3 Description:

This API tells VRTA that the application is fully idle – typically it is in Os_Cbk_Idle(). This allows VRTA to yield the thread so that the host PC can do more important work. It is also the hook where important background tasks get run. It must **only** be run from the idle state. Use vrtaYield() otherwise.

9.6.3.4 Return values:

VALUE	DESCRIPTION
`<none>`	

9.6.3.5 Notes:

None

9.6.3.6 See also:

vrtaYield.

9.6.4 vrtaYield()

Yield the processor.

9.6.4.1 Function declaration:

```
void vrtaYield(vrtamillisecond msecs)
```

9.6.4.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
`msecs`	INPUT	THE NUMBER FOR MILLISECONDS TO YIELD FOR.

9.6.4.3 Description:

This API call tells the VRTA that the calling thread will be idle for the specified number of milliseconds. Ideally a thread that is idle should call this API rather than busy-waiting. Doing so allows VRTA to suspend the thread so that the host PC can use the processor to run another thread.

The OS thread will still respond to interrupts and run the corresponding ISRs while inside a call of `vrta`Yield`()`. For example, if at time `t` the OS thread calls `vrta`Yield`(100)` and at time `t+10` an interrupt arrives, the corresponding ISR will be run by the OS thread at, or shortly after, `t+10`.

9.6.4.4 Return values:

VALUE	DESCRIPTION
`<none>`	

9.6.4.5 Notes:

None

9.6.4.6 See also:

vrtaIsIdle.

9.6.5 vrtaLeaveUninterruptibleSection()

Leave a critical section.

9.6.5.1 Function declaration:

```
void vrtaLeaveUninterruptibleSection(void)
```

9.6.5.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
`<none>`		

9.6.5.3 Description:

(VRTA-win only)

This function leaves a critical section previously entered by calling `vrtaEnterUninterruptibleSection()`.

9.6.5.4 Return values:

VALUE	DESCRIPTION
`<none>`	

9.6.5.5 Notes:

None.

9.6.5.6 See also:

`vrtaEnterUninterruptibleSection()`

9.6.6 vrtaLoadVM()

9.6.6.1 Function declaration:

```
void vrtaLoadVM(void)
```

9.6.6.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
`<none>`		

9.6.6.3 Description:

This API call loads the VRTA DLL and prepares its API for use.

9.6.6.4 Return values:

VALUE	DESCRIPTION
`<none>`	

9.6.6.5 Notes:

This API must be called before any other VRTA API is called.

The correct sequence of API calls to start a Virtual ECU running is: `vrtaLoadVM()`, `vrtaInitialize()` and `vrtaStart()`.

Note

If a Virtual ECU is using RTA-OS this API does not need to be called explicitly as it is called by the `main` program

9.6.6.6 See also:

```
vrtaInitialize()
vrtaStart()
```

9.6.7 vrtaNoneUserThread()

Generate a fatal error when an RTA-OS API call is made by a thread other than the OS thread.

9.6.7.1 Function declaration:

```
void vrtaNoneUserThread(const char * funcName)
```

9.6.7.2 Parameters:

Not documented.

9.6.7.3 Description:

This API call is used from inside RTA-OS support files to generate a fatal error when an RTA-OS API call is made by a thread other than the OS thread. **This API call is not for application use.**

9.6.7.4 Return values:

Not documented.

9.6.7.5 Notes:

None.

9.6.8 vrtaOSGetIPL()

Get the interrupt priority level.

9.6.8.1 Function declaration:

```
vrtaIntPriority vrtaOSGetIPL(void)
```

9.6.8.2 Parameters:

Not documented.

9.6.8.3 Description:

This API call is used from inside RTA-OS support files to get the interrupt priority level within VRTA. **This API call is not for application use.**

9.6.8.4 Return values:

Not documented.

9.6.8.5 Notes:

None.

9.6.9 vrtaOSSetIPL()

Set the interrupt priority level.

9.6.9.1 Function declaration:

```
vrtaIntPriority vrtaOSSetIPL(vrtaIntPriority newIPL)
```

9.6.9.2 Parameters:

Not documented.

9.6.9.3 Description:

This API call is used from inside RTA-OS support files to set the interrupt priority level within VRTA. **This API call is not for application use.**

9.6.9.4 Return values:

Not documented.

9.6.9.5 Notes:

None.

9.6.10 vrtaRaiseEvent()

Raise an event.

9.6.10.1 Function declaration:

```
vrtaErrType vrtaRaiseEvent(vrtaDevID dev,
                           const vrtaEvent *ev)
```

9.6.10.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
`dev`	INPUT	THE ID OF THE VIRTUAL DEVICE RAISING THE EVENT.
`ev`	INPUT	A POINTER TO A STRUCTURE THAT CONTAINS THE EVENT TO BE RAISED.

9.6.10.3 Description:

This API is used by a virtual device to raise an event.

9.6.10.4 Return values:

VALUE	DESCRIPTION
`RTVECUErr_NONE`	THE API CALL WAS SUCCESSFUL.
`RTVECUErr_Dev`	THE SPECIFIED DEVICE ID IS INVALID.

RTVECUErr_ID	THE SPECIFIED EVENT ID IS INVALID.
RTVECUErr_VAL	THE DATA PROVIDED IN THE EVENT IS INVALID.

9.6.10.5 Notes:

VRTA calls any event handlers that have hooked the event during `vrtaRaiseEvent()` and passes them the `vrtaEvent` structure as an argument.

9.6.10.6 See also:

```
vrtaRegisterVirtualDevice()
vrtaGetState()
vrtaEventRegister()
vrtaEventUnregister()
vrtaHookEvent()
```

9.6.11 vrtaReadHPTime()

Read the PC's high-performance timer.

9.6.11.1 Function declaration:

```
unsigned vrtaReadHPTime(unsigned desired_ticks_per_s)
```

9.6.11.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
desired_ticks_per_s	INPUT	THE NUMBER OF "TICKS" REQUIRED PER SECOND.

9.6.11.3 Description:

This API call returns the number of caller defined "ticks" that have elapsed since the Virtual ECU started. The value is derived by reading the PC's high-performance timer; therefore the resolution of the value returned depends on the details of the PC. In practice this appears to be a low multiple of the CPU clock speed on a typical PC.

9.6.11.4 Return values:

VALUE	DESCRIPTION
<number of ticks>	THE NUMBER OF "TICKS" THAT HAVE ELAPSED SINCE THE VIRTUAL ECU STARTED. THE NUMBER OF "TICKS" IN A SECOND IS SET BY THE API CALL ARGUMENT `DESIRED_TICKS_PER_S`.

9.6.11.5 Notes:

None.

9.6.11.6 See also:

None.

9.7 vrtaRegisterVirtualDevice()

Register a virtual device.

9.7.1.1 Function declaration:

```
vrtaDevID vrtaRegisterVirtualDevice(
            const vrtaTextPtr name,
            const vrtaOptStringlistPtr info,
            const vrtaOptStringlistPtr events,
            const vrtaOptStringlistPtr actions,
            const vrtaActionCallback aCallback,
            const vrtaStateCallback sCallback,
            const void *tag)
```

9.7.1.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
`name`	INPUT	A UNIQUE NAME FOR THE VIRTUAL DEVICE.
`info`	INPUT	A '`\N`' SEPARATED STRING CONTAINING INFORMATION ABOUT THE VIRTUAL DEVICE.
`events`	INPUT	A '`\N`' SEPARATED STRING CONTAINING DESCRIPTIONS OF THE EVENTS SUPPORTED BY THE VIRTUAL DEVICE.
`actions`	INPUT	A '`\N`' SEPARATED STRING CONTAINING DESCRIPTIONS OF THE ACTIONS SUPPORTED BY THE VIRTUAL DEVICE.
`aCallback`	INPUT	A POINTER TO AN ACTION CALLBACK FUNCTION CALLED TO HANDLE ACTIONS SENT TO THE VIRTUAL DEVICE.
`sCallback`	INPUT	A POINTER TO A STATE CALLBACK FUNCTION CALLED TO HANDLE `VRTA-GETSTATE()` REQUESTS.
`tag`	INPUT	APPLICATION DATA PASSED TO THE `ACALLBACK` AND `SCALLBACK` FUNCTIONS.

9.7.1.3 Description:

This API call is used to register a virtual device.

`name`: the `name` argument specifies a unique name for the virtual device. If the name is not unique VRTA will generate a fatal error.

`info`: the `info` argument describes the virtual device. This should be a string of the form: `"Type=<type>\nVersion=<version>\n"` where `<type>` is the type of device e.g. `"clock"`, `"actuator"` or `"CAN Channel"`, and `<version>` is the version of the device. VRTA does not prescribe the values of `<type>` and `<version>` as these are simply information items that can be obtained by querying the VECU's device manager (e.g. with `vrtaMonitor`).

`events`: the `events` argument describes the events supported by the virtual device. Each event supported is described by a string of one of the following forms:

Events Format	Description
`"<name>"`	Describes an event that does not have any associated data. This would be used for an event that simply happens at some point in time.
`"<name>=<format>"`	Describes an event that contains data as described by `<format>`.
`"<name>=<format>(<format>)"`	Describes an event that contains data as described by `<format>` before the "`()`" and input data when queried by `vrtaGetState()` as described by `<format>` inside the "`()`".

`<name>` is the name of the event and `<format>` is a data format string as defined in section 9.3. Where device supports multiple events then the event descriptions are separated using '`\n`'.

The first event in the list has event ID 1, the second has event ID 2, and so on.

`actions`: the `actions` argument describes the actions supported by device. Each action supported is described by a string of one of the following forms:

Action Format	Description
"<name>"	Describes an action that does not have any associated data
"<name>=<format>"	Describes an action that contains data as described by <format>.

Where <name> is the name of the action and <format> is a data format string as defined in section 9.3. Where device supports multiple actions then the event descriptions are separated using '\n'.

The first action in the list has action ID 1, the second has action ID 2, and so on.

aCallback: the aCallback argument points to an action callback function that is called to handle actions sent to the virtual device. The action callback function has the type vrtaActionCallback with the following definition:

```
typedef vrtaErrType (*vrtaActionCallback)(
    void *instance, const vrtaAction *action);
```

When the action callback function is called its instance argument is set the value of the tag argument passed to vrtaRegisterVirtualDevice() and the action argument points to the vrtaAction structure containing the action sent to the device (see vrtaSendAction() for a description of the contents of the vrtaAction structure in section 9.2.14).

The action callback function should determine what action is to be carried out by examining the devAction field of action. It should then extract any data required from the devEmbeddedData or devActionData fields of action (again see vrtaSendAction()).

The action callback function should return RTVECUErr_NONE on success, RTVECUErr_ID if the action ID in the devAction field of action is invalid, or RTVECUErr_VAL if the data in action is invalid.

The vrtaAction structure pointed to by action and any storage pointed to by the devActionData field of action are only valid for the duration of the action callback function. If the application wishes to use this data after the action callback function has returned it must copy the data into its own storage.

In addition to handling actions described in the actions argument passed to vrtaRegisterVirtualDevice() a virtual device will also been sent a special

Reset command with action ID zero. In this case the action data will be a copy of a `vrtaDevResetInfo` structure defined as follows:

```
enum vrtaResetTypes {
    vrtaDevStart,
    vrtaDevStop,
    vrtaDevWriteToPersistentStorage,
    vrtaDevReadFromPersistentStorage};
```

```
typedef struct {
    vrtaDataLen * vPSLen;
    vrtaByte **   vPSAddr;
    vrtaByte      vResetType;
} vrtaDevResetInfo;
```

The `vResetType` field describes the reason for the "reset" action as follows:

VRESETTYPE VALUE	REASON FOR "RESET" ACTION
`vrtaDevStart`	THE OS THREAD IS ABOUT TO START RUNNING.
`vrtaDevStop`	THE VIRTUAL ECU IS ABOUT TO TERMINATE.
`vrtaDevWriteToPersistentStorage`	THE VIRTUAL ECU IS ABOUT TO BE RESET. THE VIRTUAL DEVICE MAY WISH TO ARRANGE FOR DATA TO BE PROPAGATED ACROSS THE RESET. IF IT DOES IT SHOULD SET `*VPSLEN` TO THE NUMBER OF BYTES OF DATA TO PROPAGATE AND `*VPSADDR` TO POINT TO THE DATA TO PROPAGATE.
`vrtaDevReadFromPersistentStorage`	THE VIRTUAL ECU HAS BEEN RESET. THE VIRTUAL DEVICE MAY HAVE ARRANGED TO PROPAGATE DATA FROM BEFORE THE RESET. IF IT DID THEN `*VPSLEN` WILL CONTAIN THE NUMBER OF BYTES OF DATA PROPAGATED AND `*VPSADDR` WILL POINT TO THE DATA PROPAGATED. THE VIRTUAL DEVICE MUST COPY THE DATA FROM `*VPSADDR` BEFORE THE ACTION CALLBACK HANDLER RETURNS.

`sCallback`: the `sCallback` argument points to a state callback function that is called when `vrtaGetState()` is called to query one of the events supported by the

virtual device. The state callback function has the type `vrtaStateCallback` with the following definition:

```
typedef vrtaErrType (*vrtaStateCallback)(
    void *instance, vrtaEvent *state);
```

When the state callback function is called its `instance` argument is set to the value of the `tag` argument passed to `vrtaRegisterVirtualDevice()` and the `state` argument points to the `vrtaEvent` structure containing the event to be queried. There may be incoming data in the `vrtaEvent` structure. See section 9.2.15 for a description of the contents of the `vrtaEvent` structure.

The state callback function should determine what event is to be queried by examining the `devEvent` field of `state`. It should then extract any input data required from the `devEmbeddedData` or `devEventData` fields of `state` and store the result of the query in the `devEmbeddedData` or `devEventData` fields.

The function should return `RTVECUErr_NONE` on success, `RTVECUErr_ID` if the event ID in the `devEvent` field of `state` is invalid, or `RTVECUErr_VAL` if the data in `state` is invalid.

9.7.1.4 Return values:

VALUE	DESCRIPTION
`<device ID>`	THE ID OF THE VIRTUAL DEVICE.

9.7.1.5 Notes:

The callback functions can be called from different threads, so they must be thread-safe.

9.7.1.6 See also:

```
vrtaGetState()
vrtaRaiseEvent()
vrtaEventRegister()
vrtaEventUnregister()
vrtaHookEvent()
vrtaSendAction()
```

9.7.2 vrtaReset()

Reset the Virtual ECU.

9.7.2.1 Function declaration:

```
void vrtaReset(void)
```

9.7.2.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
`<none>`		

9.7.2.3 Description:

This API call instructs VRTA to reset. It does this by creating a new process and running a new copy of the VECU in it. Certain information such as command line options and connections to external programs are propagated to the new process. This creates the effect of an ECU being reset and starting execution from its reset vector. Since connections to external programs are propagated to the new process, programs communicating with the VECU (such as `vrtaServer` or `vrtaMonitor`) continue to be able to communicate with the VECU after reset and don't notice the handover of processes.

After `vrtaReset()` has been called the OS thread is allowed approximately 10 seconds to terminate cleanly (either by returning from `OS_MAIN()` or calling `vrtaTerminate()`). If the OS thread does not terminate within 10 seconds it is forcibly terminated.

Once the OS thread has terminated VRTA sends a Reset action to each virtual device to inform it that the VECU is about to reset.

Next VRTA saves certain state information (such as connections to external programs) in a temporary file.

The call of `vrtaStart()` that started the OS thread then returns.

Once this has happened the new process starts running and the `main()` entry-point of the VECU is called by the C/C++ start-up code. This entry-point carries out the normal initialization sequence of calling `vrtaLoadVM()`, `vrtaInitialize()` and `vrtaStart()`. However, when `vrtaInitialize()` is called VRTA determines that it has been reset and restores state information from the temporary file created by the original process.

9.7.2.4 Return values:

VALUE	DESCRIPTION
`<none>`	

9.7.2.5 Notes:

None.

9.7.2.6 See also:

```
vrtaStart()
vrtaTerminate()
```

9.7.3 vrtaSendAction()

Send an action to a virtual device.

9.7.3.1 Function declaration:

```
vrtaErrType vrtaSendAction(vrtaDevID id,
                           const vrtaAction *a)
```

9.7.3.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
`id`	INPUT	THE ID OF THE VIRTUAL DEVICE TO WHICH THE ACTION SHOULD BE SENT.
`a`	INPUT	A POINTER TO A STRUCTURE THAT CONTAINS THE ACTION TO BE SENT.

9.7.3.3 Description:

This API call causes the data in the `vrtaAction` structure to be sent to the virtual device.

9.7.3.4 Return values:

VALUE	DESCRIPTION
`RTVECUErr_NONE`	THE API CALL WAS SUCCESSFUL.
`RTVECUErr_Dev`	THE SPECIFIED DEVICE ID IS INVALID.
`RTVECUErr_ID`	THE SPECIFIED ACTION ID IS INVALID.
`RTVECUErr_VAL`	THE DATA PROVIDED IN THE ACTION IS INVALID.

9.7.3.5 Notes:

The action callback function of the target device is called by the same thread that calls `vrtaSendAction()`.

9.7.3.6 See also:

```
vrtaRegisterVirtualDevice()
```

9.7.4 vrtaSpawnThread()

Create a new *VRTA* thread.

9.7.4.1 Function declaration:

```
void vrtaSpawnThread(void (*func)(void))
```

9.7.4.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
`func`	INPUT	THE ENTRY FUNCTION FOR THE NEW THREAD.

9.7.4.3 Description:

This API call creates a new *VRTA* worker thread. It should be used in preference to creating native threads so that VRTA can keep track of the workers.

A thread created with `vrtaSpawnThread()` should terminate itself as soon as it discovers that the VECU is about to terminate. This is normally done by polling `vrtaIsAppFinished()` regularly. If a thread does not do this it will continue running until forcibly terminated when the VECU process terminates.

9.7.4.4 Return values:

VALUE	DESCRIPTION
`<none>`	

9.7.4.5 Notes:

You will normally call this API during the initialization of your devices, before `OS_MAIN()` starts.

If you call it from within the OS thread after `OS_MAIN()` starts, you must ensure that it cannot be interrupted.

9.7.4.6 See also:

```
vrtaIsAppFinished()
vrtaEnterUninterruptibleSection()
vrtaLeaveUninterruptibleSection()
```

9.7.5 vrtaStart()

Start the OS thread.

9.7.5.1 Function declaration:

```
void vrtaStart(void)
```

9.7.5.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
`<none>`		

9.7.5.3 Description:

This API call requests VRTA to start the OS thread.

If the VECU has been loaded in autostart mode (default) then the OS thread is started as soon as `vrtaStart()` is called. If the VECU has been loaded in slave mode then the OS thread is not started until a Start action is sent to the ApplicationManager device.

VRTA sends a Reset action to each virtual device just before the OS thread starts to inform them that the OS thread starting.

This is followed by a call to the application-provided function `InitializeDevices()` that should carry out any necessary virtual device initialization. Finally the main Virtual ECU application entry-point function `OS_MAIN()` is called.

`vrtaStart()` does not return until VRTA terminates (e.g. because the OS thread or another thread calls `vrtaTerminate()` or a Terminate action is sent to the ApplicationManager device).

9.7.5.4 Return values:

VALUE	DESCRIPTION
`<none>`	

9.7.5.5 Notes:

The correct sequence of API calls to start a Virtual ECU running is: `vrtaLoadVM()`, `vrtaInitialize()` and `vrtaStart()`.

Note

If Virtual ECU is using RTA-OS then this API should not be called explicitly as it is called by the `main program`.

9.7.5.6 See also:

```
InitializeDevices()
OS_MAIN()
vrtaLoadVM()
vrtaInitialize()
```

9.7.6 vrtaTerminate()

Terminate the Virtual ECU.

9.7.6.1 Function declaration:

```
void vrtaTerminate(void)
```

9.7.6.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
<none>		

9.7.6.3 Description:

This API call instructs VRTA to terminate. If this API is called by the OS thread then it never returns. If this API is called by any other thread then it does return.

If `vrtaTerminate()` is called by a thread other than the OS thread then the OS thread is allowed approximately 10 seconds to terminate cleanly (either by returning from `OS_MAIN()` or calling `vrtaTerminate()`). If the OS thread does not terminate within 10 seconds it is forcibly terminated.

Once the OS thread has terminated VRTA sends a Reset action to each virtual device to inform it that the VECU is about to terminate.

Finally the call of `vrtaStart()` that started the OS thread returns.

9.7.6.4 Return values:

VALUE	DESCRIPTION
<none>	

9.7.6.5 Notes:

None.

9.7.6.6 See also:

```
vrtaStart()
vrtaReset()
```

10 Standard Device Reference

A VRTA ECU has three standard devices that are used to provide services to applications.

This chapter describes the purpose of these 3 devices and the actions and events they support.

10.1 vrtaStdDevices.h

The header file vrtaStdDevices.h contains definitions of the device, action and event IDs used by the internal devices. vrtaStdDevices.h is automatically included if you include vrtaCore.h.

10.2 Action and Event Descriptions

Each action or event supported by an internal device is described by a standard table, as below, followed by text to explain the purpose of the action or event.

ID	DATA FORMAT
`YYYY`	`ZZZZ`

The table contains the ID of the action or event (`YYYY`), and the format of the action or event data (`ZZZZ`).

For actions, `ZZZZ` will be a data format string describing the format of the data in the action (e.g. "`%s`" for string data).

For events that do not require any input data to be supplied when they are queried, `ZZZZ` will be a data format string describing the format of the data in the event (e.g. "`%u`" for a single unsigned numeric value).

For events that do require input data to be supplied when they are queried, `ZZZZ` will be a data format string followed by a second data format string enclosed in "`()`". The first data format string describes the format of the data in the event. The data second data format string describes the format of the input data required when the event is queried (e.g. "`%s(%u)`" for an event that contains string data and requires a single unsigned numeric value as input data when queried).

Refer to section 9.3 for a description of data format strings.

10.3 Device Manager

The Device Manager (DM) is the internal device that manages all devices. The DM is identified as follows:

DEVICE ID CONSTANT	NAME
DM_DEVICE_ID	DEVICEMANAGER

10.3.1 Action: EventRegister

ACTION ID CONSTANT	DATA FORMAT
DM_ACTION_ID_EventRegister	%S

This action is only available for use via the Device Communication Link and is only used by external monitor programs.

This action registers a TCP/IP port wishing to hook events. The data is a string containing a list of '\n' separated values. The first value is the IP address of the listener - e.g. "192.168.0.100" or "localhost". The second value is the TCP port number in decimal - e.g. "2034".

This action causes the Device Communication Link to open a network connection to the specified TCP/IP port and then associate a 'listener' instance with it. Each connection can have at most one such listener.

The listener is removed if the connection breaks or if another EventRegister action is received.

10.3.2 Action: HookEvents

ACTION ID CONSTANT	DATA FORMAT
DM_ACTION_ID_HookEvents	%s

This action is only available for use via the Device Communication Link and is only used by external monitor programs.

This action specifies which events to hook for the Device Communication Link connection on which the action is received. The data is a '\n' separated list of items of the form <device>=<event1>,<event2>,…,<eventN>. Where <device> is a device ID in decimal and <eventi> is an event ID in decimal. Such an item means hook the events with IDs <event1>…<eventN> for the device with ID <device> - e.g. "3=1,3,6\n5=2\n7=2,3".

The data forms the complete list of events to hook for the connection. Subsequent HookEvents actions replace the events hooked rather than add to them. An empty list means that no events will be hooked.

10.3.3 Action: ListAll

ACTION ID CONSTANT	DATA FORMAT
DM_ACTION_ID_ListAll	<none>

This action causes the DM to raise a DeviceList event.

10.3.4 Action: GetDeviceActions

ACTION ID CONSTANT	DATA FORMAT
DM_ACTION_ID_GetDeviceActions	%s

This action causes the DM to raise a DeviceActions event for the device named by the action data.

10.3.5 Action: GetDeviceEvents

ACTION ID CONSTANT	DATA FORMAT
DM_ACTION_ID_GetDeviceEvents	%s

This action causes the DM to raise a DeviceEvents event for the device named by the action data.

10.3.6 Action: GetDeviceInfo

ACTION ID CONSTANT	DATA FORMAT
DM_ACTION_ID_GetDeviceInfo	%s

This action causes the DM to raise a DeviceInfo event for the device named by the action data.

10.3.7 Event: DeviceList

EVENT ID CONSTANT	DATA FORMAT
DM_EVENT_ID_DeviceList	%s

The data is a '\n' separated list of all of the devices registered with the Virtual ECU.

10.3.8 Event: DeviceActions

EVENT ID CONSTANT	DATA FORMAT
DM_EVENT_ID_DeviceActions	%s(%s)

The data is a '\n' separated list of all of the actions supported by the named device in the same form as used for specifying the list of actions supported by a virtual device in the vrtaRegisterVirtualDevice() call. If the event is raised in response to a GetDeviceActions action then the device is named by the action data. If the event is queried the device is named by the event input data.

10.3.9 Event: DeviceEvents

EVENT ID CONSTANT	DATA FORMAT
DM_EVENT_ID_DeviceEvents	%s(%s)

The data is a '\n' separated list of all of the events supported by the named device in the same form as used for specifying the list of events supported by a virtual device

in the `vrtaRegisterVirtualDevice()` call. If the event is raised in response to a GetDeviceEvents action then the device is named by the action data. If the event is queried the device is named by the event input data.

10.3.10 Event: DeviceInfo

EVENT ID CONSTANT	DATA FORMAT
`DM_EVENT_ID_DeviceInfo`	`%s(%s)`

The data is information about the named device in the same form as used for specifying virtual device information in the `vrtaRegisterVirtualDevice()` call. If the event is raised in response to a GetDeviceInfo action then the device is named by the action data. If the event is queried the device is named by the event input data.

10.4 Interrupt Control Unit

The Interrupt Control Unit (ICU) is the internal device that manages interrupts within the VECU and arranges for interrupt handlers to be invoked within the OS thread.

The ICU implements a multilevel interrupt controller. There are 32 interrupts on VRTA-win numbered 1 to 32 inclusive. Interrupt number n corresponds to interrupt vector number n. Each interrupt has a priority in the range 1 to 32 inclusive for VRTA-win. The priority of an interrupt is set in the corresponding interrupt vector table entry. See the `vrtaInitialize()` call for a description of the interrupt vector table.

The ICU maintains the current interrupt priority level (IPL). This is a number in the range zero to 32 inclusive. If an interrupt handler is running then the current IPL is equal to the priority of the corresponding interrupt. If non interrupt code is running the current IPL is zero.

Each interrupt has a pending flag and may be masked (disabled) or unmasked (enabled). An interrupt is made pending (i.e. its pending flag is set) by sending a Raise action to the ICU. The ICU invokes the interrupt handler for the highest priority pending and unmasked interrupt that has a priority higher than the current IPL. If an interrupt is pending but is masked its handler will not be invoked until the interrupt is unmasked. If an interrupt is pending but the current IPL is higher or equal to the priority of the interrupt then its handler will not be invoked until the IPL drops below the priority of the interrupt.

An interrupt's pending flag is cleared just before its handler is invoked. Therefore if the handler for an interrupt sends a Raise action to the ICU for the same interrupt the interrupt will become pending again and a second instance of the interrupt handler will run as soon as the first ends.

A higher priority interrupt handler can pre-empt a lower priority interrupt handler.

If two interrupts of the same priority are pending then the one with the lower interrupt vector number is handled first.

When the Virtual ECU starts all interrupts are masked (disabled) and the current IPL is zero.

The ICU is identified as follows:

DEVICE ID CONSTANT	NAME
ICU_DEVICE_ID	ICU

10.4.1 Action: Raise

ACTION ID CONSTANT	DATA FORMAT
ICU_ACTION_ID_Raise	%d:;1;32

This action makes the specified interrupt number pending (i.e. sets the interrupt's pending flag).

10.4.2 Action: Clear

ACTION ID CONSTANT	DATA FORMAT
ICU_ACTION_ID_Clear	%d:;1;32

This action clears the specified interrupt number's pending flag. Note that it is not necessary to send this action to the ICU in an interrupt handler to clear the pending flag of the interrupt being handled since an interrupt's pending flag is cleared just before its handler is invoked.

10.4.3 Action: Mask

ACTION ID CONSTANT	DATA FORMAT
ICU_ACTION_ID_Mask	%d:;1;32

This action masks (disables) the specified interrupt number.

10.4.4 Action: Unmask

ACTION ID CONSTANT	DATA FORMAT
ICU_ACTION_ID_Unmask	%d:;1;32

This action unmasks (enables) the specified interrupt number.

10.4.5 Action: GetPending

ACTION ID CONSTANT	DATA FORMAT
ICU_ACTION_ID_GetPending	<none>

This action causes a Pending event to be raised.

10.4.6 Action: GetIPL

ACTION ID CONSTANT	DATA FORMAT
ICU_ACTION_ID_GetIPL	<none>

This action causes an IPL event to be raised.

10.4.7 Action: SetIPL

ACTION ID CONSTANT	DATA FORMAT
ICU_ACTION_ID_SetIPL	%d:;0;32

This action sets the IPL to the specified value.

10.4.8 Event: Pending

EVENT ID CONSTANT	DATA FORMAT
ICU_EVENT_ID_Pending	%b

This event contains a list of all of the currently pending interrupts. Bit n is set in the event if interrupt number n is pending (where bit 1 is the least significant bit).

The event is raised in response to a GetPending action or when the list of pending interrupts changes.

10.4.9 Event: Start

EVENT ID CONSTANT	DATA FORMAT
ICU_EVENT_ID_Start	%d

This event is raised just before an interrupt handler is invoked. The number of the interrupt is specified in the event data.

10.4.10 Event: Stop

EVENT ID CONSTANT	DATA FORMAT
ICU_EVENT_ID_Stop	%d

This event is raised just after an interrupt handler has ended. The number of the interrupt is specified in the event data.

10.4.11 Event: IPL

EVENT ID CONSTANT	DATA FORMAT
ICU_EVENT_ID_IPL	%d

This event contains the current IPL. The event is raised in response to a GetIPL action or when the current IPL changes.

10.4.12 Event: EnabledVecs

EVENT ID CONSTANT	DATA FORMAT
ICU_EVENT_ID_MASKS	%b

This event contains a list of all of the currently enabled (unmasked) interrupts. Bit n is set in the event if interrupt number n is enabled (where bit 1 is the least significant bit).

The event is raised when the list of enabled interrupts changes.

10.5 Application Manager

The Application Manager (AM) is the internal device that manages the Virtual ECU application

The AM is identified as follows:

DEVICE ID CONSTANT	NAME
AM_DEVICE_ID	APPLICATIONMANAGER

10.5.1 Action: Start

ACTION ID CONSTANT	DATA FORMAT
AM_ACTION_ID_Start	<none>

This action starts the OS thread running in a Virtual ECU that was loaded in *slave* mode.

10.5.2 Action: Terminate

ACTION ID CONSTANT	DATA FORMAT
`AM_ACTION_ID_Terminate`	`<none>`

This action tells the Virtual ECU to terminate. It has the same effect as the `vrtaTerminate()` call.

10.5.3 Action: Pause

ACTION ID CONSTANT	DATA FORMAT
`AM_ACTION_ID_Pause`	`<none>`

This action tells the Virtual ECU to suspend execution of the OS thread. It has no effect when called from the OS thread.

10.5.4 Action: Restart

ACTION ID CONSTANT	DATA FORMAT
`AM_ACTION_ID_Restart`	`<none>`

This action tells the Virtual ECU to restart execution of the OS thread after it has previously been suspended. It has no effect when called from the OS thread.

10.5.5 Action: Reset

ACTION ID CONSTANT	DATA FORMAT
`AM_ACTION_ID_Reset`	`<none>`

This action tells the Virtual ECU to reset. It has the same effect as the `vrtaReset()` call.

10.5.6 Action: GetInfo

ACTION ID CONSTANT	DATA FORMAT
`AM_ACTION_ID_GETINFO`	`<none>`

This action causes an Info event to be raised.

10.5.7 Action: TestOption

ACTION ID CONSTANT	DATA FORMAT
AM_ACTION_ID_TestOption	%s

This action causes an Option event to be raised to signal if the named command-line option exists. The option prefix ('-' or '/') is not specified.

10.5.8 Action: ReadOption

ACTION ID CONSTANT	DATA FORMAT
AM_ACTION_ID_ReadOption	%s

This action causes an OptionText event to be raised containing the full text of the command-line option that starts with the specified string.

10.5.9 Action: ReadParam

ACTION ID CONSTANT	DATA FORMAT
AM_ACTION_ID_ReadParam	%u

This action causes a ParamText event to be raised containing the full text of the specified command-line parameter. The first command line parameter (the executable name) is number zero, the second parameter is number one, and so on.

10.5.10 Event: Started

EVENT ID CONSTANT	DATA FORMAT
AM_EVENT_ID_Started	<none>

This event is raised to indicate that the OS thread has started.

10.5.11 Event: Paused

EVENT ID CONSTANT	DATA FORMAT
AM_EVENT_ID_Paused	<none>

This event is raised to indicate that the OS thread has been suspended.

10.5.12 Event: Restarted

EVENT ID CONSTANT	DATA FORMAT
AM_EVENT_ID_Restarted	<none>

This event is raised to indicate that the OS thread has been restarted.

10.5.13 Event: Reset

EVENT ID CONSTANT	DATA FORMAT
AM_EVENT_ID_Reset	<none>

This event is raised to indicate that the Virtual ECU has been reset.

10.5.14 Event: Terminated

EVENT ID CONSTANT	DATA FORMAT
AM_EVENT_ID_Terminated	<none>

This event is raised to indicate that the Virtual ECU is about to terminate.

10.5.15 Event: Info

EVENT ID CONSTANT	DATA FORMAT
AM_EVENT_ID_Info	%s

This event contains version information about the Virtual ECU application and VRTA.

10.5.16 Event: Option

EVENT ID CONSTANT	DATA FORMAT
AM_EVENT_ID_Option	%u(%s)

This event contains the number 1 if the named option is present on the command line or zero if it is not. If the event is raised in response to a TestOption action then the option is named in the action data. If the event is queried then the option is named in the input data. The option prefix ('-' or '/') is not included in the name.

10.5.17 Event: OptionText

EVENT ID CONSTANT	DATA FORMAT
AM_EVENT_ID_OptionText	%s(%s)

This event contains an empty string if a command line options starting with the specified prefix does not exist. If a command line option starting with the specified prefix does exist then the event contains the full text of the option. If the event is raised in response to a ReadOption action then the prefix is the action data. If the event is queried then the prefix is the input data.

10.5.18 Event: ParamText

EVENT ID CONSTANT	DATA FORMAT
AM_EVENT_ID_ParamText	%s(%u)

This event contains the full text of the specified command-line parameter. The first command line parameter (the executable name) is zero, the second parameter is

number one, and so on. If the specified command line parameter does not exist the event contains the empty string. If the event is raised in response to a ReadParam action then the parameter number is in the action data. If the event is queried then the parameter number is in the input data.

10.5.19 Event: State

EVENT ID CONSTANT	DATA FORMAT

`AM_EVENT_ID_State`	`Loaded`
	`Running`
	`Paused`
	`Terminating`
	`Resetting`

This event contains the current state of the Virtual ECU.

11 Sample Device Reference

VRTA includes a collection of sample virtual devices that implement commonly used devices such as clocks, counters, comparators, actuators and sensors. These are contained in the kernel library and exported through vrtaSampleDevices.h. All of the sample virtual devices make use of (i.e. are derived from) the C++ virtual device framework provided by the vrtaDevice base class (see the file vrtaDevice.h).

11.1 Summary

The following sample devices are provided:

DEVICE	DESCRIPTION
`vrtaClock`	A CLOCK SOURCE FOR COUNTER DEVICES.
`vrtaUpCounter`	A COUNTER THAT COUNTS UPWARDS.
`vrtaDownCounter`	A COUNTER THAT COUNTS DOWNWARDS.
`vrtaSensor`	A GENERIC SENSOR.
`vrtaSensorToggleSwitch`	A TWO POSITION “TOGGLE” SWITCH.
`vrtaSensorMultiwaySwitch`	A MULTI-POSITION SWITCH.
`VRTAACTUATOR`	A GENERIC ACTUATOR.
`vrtaActuatorLight`	A LIGHT THAT CAN BE ON OR OFF.
`vrtaActuatorDimmableLight`	A LIGHT THAT CAN BE SET TO DIFFERENT LEVELS OF BRIGHTNESS.
`vrtaActuatorMultiColorLight`	A LIGHT THAT CAN BE SET TO DIFFERENT COLORS.
`vrtaCompare`	A COMPARATOR

	THAT CAN GENERATE AN INTERRUPT WHEN OTHER ANOTHER DEVICE REACHES A SPECIFIED VALUE.
`vrtaIO`	AN I/O SPACE.

11.2 Device Descriptions

Each sample device is described in the same way. The description starts with an introduction to the purpose and operation of the device. This is followed by a description of the C++ methods exported by the device class and then the actions and events supported by the device.

11.2.1 Methods

Each C++ method is described in a standard form as follows:

11.2.1.1 The title gives the name of the method.

A brief description of the method is provided.

11.2.1.2 Method declaration:

Interface in C++ syntax.

11.2.1.3 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
`PARAMETER NAME`	INPUT/OUTPUT	DESCRIPTION.

11.2.1.4 Description:

Explanation of the functionality of the method.

11.2.1.5 Return values:

VALUE	DESCRIPTION
`RETURN VALUES.`	DESCRIPTION OF RETURN VALUE.

11.2.2 Actions and Events

Each action or event supported by a sample device is described by a standard table, as below, followed by text to explain the purpose of the action or event.

ID	DATA FORMAT
XXXX	YYYY

The table contains the numeric ID of the action or event (XXXX), and the format of the action or event data (YYYY).

11.3 vrtaClock

A vrtaClock device provides a time source for vrtaUpCounter and vrta-DownCounter counter devices.

A vrtaClock device ticks an attached counter every T milliseconds. T is calculated by multiplying the vrtaClock device's clock tick interval by its scaling factor. The scaling factor has a multiplier and a divisor. If the clock tick interval (set in the constructor or with the SetInterval() method) is interval, and the scaling factor (set with the SetScale() method) is mult / div, then an attached counter would be ticked every (interval * mult / div) milliseconds. By default the scale factor is 1 / 1.

Multiple counters may be attached to the same vrtaClock device.

11.3.1 Method: vrtaClock()

The constructor.

11.3.1.1 Method declaration:

```
vrtaClock(const vrtaTextPtr name, unsigned interval)
```

11.3.1.2 Parameters:

PARAME-TER	INPUT/OUT-PUT	DESCRIPTION
name	INPUT	THE NAME OF THE VIRTUAL DEVICE.
interval	INPUT	THE NUMBER OF MILLISECONDS IN ONE CLOCK TICK.

11.3.1.3 Description:

This is the constructor used to create an instance of a vrtaClock device.

11.3.1.4 Return values:

VALUE	DESCRIPTION
<none>	

11.3.2 Method: SetInterval()

Set the tick interval.

11.3.2.1 Method declaration:

```
void SetInterval(unsigned interval)
```

11.3.2.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
`interval`	INPUT	THE NUMBER OF MILLISECONDS IN ONE CLOCK TICK.

11.3.2.3 Description:

This method is used to change the number of milliseconds in one clock tick.

11.3.2.4 Return values:

VALUE	DESCRIPTION
`<none>`	

11.3.3 Method: SetScale()

Set the scaling factor.

11.3.3.1 Method declaration:

```
void SetScale(unsigned mult, unsigned div)
```

11.3.3.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
`mult`	INPUT	THE MULTIPLIER FOR THE SCALING FACTOR.
`div`	INPUT	THE DIVISOR FOR THE SCALING FACTOR.

11.3.3.3 Description:

This method sets the scaling factor for the clock.

11.3.3.4 Return values:

VALUE	DESCRIPTION
`<none>`	

11.3.4 Method: Start()

Start the clock ticking.

11.3.4.1 Method declaration:

```
void Start(void)
```

11.3.4.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
<none>		

11.3.4.3 Description:

This method starts the clock device ticking any attached counters.

11.3.4.4 Return values:

VALUE	DESCRIPTION
<none>	

11.3.5 Method: Stop()

Stop the clock ticking.

11.3.5.1 Method declaration:

```
void Stop(void)
```

11.3.5.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
<none>		

11.3.5.3 Description:

This method stops the clock device ticking any attached counters.

11.3.5.4 Return values:

VALUE	DESCRIPTION
<none>	

11.3.6 Action: Interval

ACTION ID	DATA FORMAT
1	%u

This action sets the number of milliseconds in a clock tick. The action data is the number of milliseconds in a clock tick. This action is equivalent to the `SetInterval()` method.

11.3.7 Action: Scale

ACTION ID	DATA FORMAT
2	`%u,%u`

This action sets the device's scaling factor. The first number in the action data is the scaling factor multiplier and the second number is the divisor. This action is equivalent to the `SetScale()` method.

11.3.8 Action: Start

ACTION ID	DATA FORMAT
3	`<none>`

This action starts the clock device ticking any attached counters. This action is equivalent to the `Start()` method.

11.3.9 Action: Stop

ACTION ID	DATA FORMAT
4	`<none>`

This action stops the clock device ticking any attached counters. This action is equivalent to the `Stop()` method.

11.3.10 Event: Interval

EVENT ID	DATA FORMAT
1	`%u`

This event is raised when the clock device's tick interval changes. The event data is the new tick interval.

11.3.11 Event: Scale

EVENT ID	DATA FORMAT
2	`%u,%u`

This event is raised when the clock device's scaling factor changes. The event data is the new scaling factor multiplier followed by the new divisor.

11.3.12 Event: Running

EVENT ID	DATA FORMAT
3	`%u:;0;1`

This event is raised when the clock is started or stopped. The event data is 1 if the clock is now running (i.e. has been started) or zero if the clock is not now running (i.e. has been stopped).

11.4 vrtaUpCounter

A `vrtaUpCounter` is a counter device that is driven by a `vrtaClock` device. It has a minimum value, a maximum value and a current value. When a `vrtaUpCounter` device is ticked by a `vrtaClock` device and its current value is less than its maximum value then its current value is incremented. When the `vrtaUpCounter` is ticked and its current value is equal to its maximum value then its current value is set back to its minimum value. The cyclic period of a `vrtaUpCounter` is thus (maximum – minimum) + 1.

By default the minimum value is zero, the maximum value is 4294967295, and the current value starts at zero.

11.4.1 Method: vrtaUpCounter()

The constructor.

11.4.1.1 Method declaration:

```
vrtaUpCounter(const vrtaTextPtr name, vrtaClock &clock)
```

11.4.1.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
`name`	INPUT	THE NAME OF THE VIRTUAL DEVICE.
`clock`	INPUT	THE `VRTACLOCK` DEVICE THAT WILL BE USED TO DRIVE THE COUNTER.

11.4.1.3 Description:

This is the constructor used to create an instance of a `vrtaUpCounter` device.

11.4.1.4 Return values:

VALUE	DESCRIPTION
<none>	

11.4.2 Method: Min()

Get the minimum value.

11.4.2.1 Method declaration:

```
unsigned Min(void)
```

11.4.2.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
<none>		

11.4.2.3 Description:

This method is used to get the minimum value of the counter.

11.4.2.4 Return values:

VALUE	DESCRIPTION
<a value>	THE MINIMUM VALUE OF THE COUNTER.

11.4.3 Method: Max()

Get the maximum value.

11.4.3.1 Method declaration:

```
unsigned Max(void)
```

11.4.3.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
<none>		

11.4.3.3 Description:

This method is used to get the maximum value of the counter.

11.4.3.4 Return values:

VALUE	DESCRIPTION
<a value>	THE MAXIMUM VALUE OF THE COUNTER.

11.4.4 Method: Value()

Get the current value.

11.4.4.1 Method declaration:

```
unsigned Value(void)
```

11.4.4.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
<none>		

11.4.4.3 Description:

This method is used to get the current value of the counter.

11.4.4.4 Return values:

VALUE	DESCRIPTION
<a value>	THE CURRENT VALUE OF THE COUNTER.

11.4.5 Method: SetMin()

Set the minimum value.

11.4.5.1 Method declaration:

```
void SetMin(unsigned v)
```

11.4.5.2 Parameters:

PARAME-TER	INPUT/OUT-PUT	DESCRIPTION
v	INPUT	THE NEW MINIMUM VALUE FOR THE COUNTER.

11.4.5.3 Description:

This method is used to set the minimum value of the counter. If the current value of the counter is smaller than the new minimum value then the current value is set to the new minimum value.

11.4.5.4 Return values:

VALUE	DESCRIPTION
<none>	

11.4.6 Method: SetMax()

Set the maximum value.

11.4.6.1 Method declaration:

```
void SetMax(unsigned v)
```

11.4.6.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
v	INPUT	THE NEW MAXIMUM VALUE FOR THE COUNTER.

11.4.6.3 Description:

This method is used to set the maximum value of the counter. If the current value of the counter is greater than the new maximum value then the current value is set to the minimum value.

11.4.6.4 Return values:

VALUE	DESCRIPTION
<none>	

11.4.7 Method: SetVal()

Set the current value.

11.4.7.1 Method declaration:

```
void SetVal(unsigned v)
```

11.4.7.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
v	INPUT	THE NEW VALUE FOR THE COUNTER.

11.4.7.3 Description:

This method is used to set the current value of the counter. If the new current value of the counter is smaller than the minimum value or greater than the maximum value then the current value is set to the minimum value.

11.4.7.4 Return values:

VALUE	DESCRIPTION
`<none>`	

11.4.8 Method: Start()

Start the counter counting.

11.4.8.1 Method declaration:

```
void Start(void)
```

11.4.8.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
`<none>`		

11.4.8.3 Description:

This method is used to start the counter counting when ticked by the attached `vrtaClock` device.

11.4.8.4 Return values:

VALUE	DESCRIPTION
`<none>`	

11.4.9 Method: Stop()

Stop the counter counting.

11.4.9.1 Method declaration:

```
void Stop(void)
```

11.4.9.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
`<none>`		

11.4.9.3 Description:

This method is used to stop the counter counting when ticked by the attached `vrtaClock` device.

11.4.9.4 Return values:

VALUE	DESCRIPTION
<none>	

11.4.10 Action: Minimum

ACTION ID	DATA FORMAT
1	%u

This action sets the minimum value of the counter. It is the equivalent of the `SetMin()` method.

11.4.11 Action: Maximum

ACTION ID	DATA FORMAT
2	%u

This action sets the maximum value of the counter. It is the equivalent of the `SetMax()` method.

11.4.12 Action: Set

ACTION ID	DATA FORMAT
3	%u

This action sets the current value of the counter. It is the equivalent of the `SetVal()` method.

11.4.13 Action: Start

ACTION ID	DATA FORMAT
4	<none>

This method is used to start the counter counting when ticked by the attached `vrtaClock` device. It is the equivalent of the `Start()` method.

11.4.14 Action: Stop

ACTION ID	DATA FORMAT
5	<none>

This method is used to stop the counter counting when ticked by the attached `vrtaClock` device. It is the equivalent of the `Stop()` method.

11.4.15 Action: Report

ACTION ID	DATA FORMAT
6	<none>

This action causes a Set event to be raised.

11.4.16 Event: Set

EVENT ID	DATA FORMAT
1	%u

This event contains the current value of the counter. It is raised in response to a Report action.

11.5 vrtaDownCounter

A `vrtaDownCounter` is a counter device that is driven by a `vrtaClock` device. It has a minimum value, a maximum value and a current value. When a `vrtaDownCounter` device is ticked by a `vrtaClock` device and its current value is greater than its minimum value then its current value is decremented. When the `vrtaDownCounter` is ticked and its current value is equal to its minimum value then its current value is set back to its maximum value. The cyclic period of a `vrtaDownCounter` is thus (maximum – minimum) + 1.

By default the minimum value is zero, the maximum value is 4294967295, and the current value starts at zero.

11.5.1 Method: vrtaDownCounter()

The constructor.

11.5.1.1 Method declaration:

```
vrtaDownCounter(const     vrtaTextPtr name,
                vrtaClock &clock)
```

11.5.1.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
name	INPUT	THE NAME OF THE VIRTUAL DEVICE.
clock	INPUT	THE `VRTACLOCK` DEVICE THAT WILL BE

		USED TO DRIVE THE COUNTER.

11.5.1.3 Description:

This is the constructor used to create an instance of a `vrtaDownCounter` device.

11.5.1.4 Return values:

VALUE	DESCRIPTION
`<none>`	

11.5.2 Method: Min()

Get the minimum value.

11.5.2.1 Method declaration:

```
unsigned Min(void)
```

11.5.2.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
`<none>`		

11.5.2.3 Description:

This method is used to get the minimum value of the counter.

11.5.2.4 Return values:

VALUE	DESCRIPTION
`<a value>`	THE MINIMUM VALUE OF THE COUNTER.

11.5.3 Method: Max()

Get the maximum value.

11.5.3.1 Method declaration:

```
unsigned Max(void)
```

11.5.3.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
`<none>`		

11.5.3.3 Description:

This method is used to get the maximum value of the counter.

11.5.3.4 Return values:

VALUE	DESCRIPTION
<a value>	THE MAXIMUM VALUE OF THE COUNTER.

11.5.4 Method: Value()

Get the current value.

11.5.4.1 Method declaration:

```
unsigned Value(void)
```

11.5.4.2 Parameters:

PARAMETER	INPUT/OUT-PUT	DESCRIPTION
<none>		

11.5.4.3 Description:

This method is used to get the current value of the counter.

11.5.4.4 Return values:

VALUE	DESCRIPTION
<a value>	THE CURRENT VALUE OF THE COUNTER.

11.5.5 Method: SetMin()

Set the minimum value.

11.5.5.1 Method declaration:

```
void SetMin(unsigned v)
```

11.5.5.2 Parameters:

PARAME-TER	INPUT/OUT-PUT	DESCRIPTION
v	INPUT	THE NEW MINIMUM VALUE FOR THE COUNTER.

11.5.5.3 Description:

This method is used to set the minimum value of the counter. If the current value of the counter is smaller than the new minimum value then the current value is set to the new minimum value.

11.5.5.4 Return values:

VALUE	DESCRIPTION
<none>	

11.5.6 Method: SetMax()

Set the maximum value.

11.5.6.1 Method declaration:

```
void SetMax(unsigned v)
```

11.5.6.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
v	INPUT	THE NEW MAXIMUM VALUE FOR THE COUNTER.

11.5.6.3 Description:

This method is used to set the maximum value of the counter. If the current value of the counter is greater than the new maximum value then the current value is set to the minimum value.

11.5.6.4 Return values:

VALUE	DESCRIPTION
<none>	

11.5.7 Method: SetVal()

Set the current value.

11.5.7.1 Method declaration:

```
void SetVal(unsigned v)
```

11.5.7.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
v	INPUT	THE NEW VALUE FOR THE COUNTER.

11.5.7.3 Description:

This method is used to set the current value of the counter. If the new current value of the counter is smaller than the minimum value or greater than the maximum value then the current value is set to the minimum value.

11.5.7.4 Return values:

VALUE	DESCRIPTION
`<none>`	

11.5.8 Method: Start()

Start the counter counting.

11.5.8.1 Method declaration:

```
void Start(void)
```

11.5.8.2 Parameters:

PARAMETER	INPUT/OUT-PUT	DESCRIPTION
`<none>`		

11.5.8.3 Description:

This method is used to start the counter counting when ticked by the attached `vrtaClock` device.

11.5.8.4 Return values:

VALUE	DESCRIPTION
`<none>`	

11.5.9 Method: Stop()

Stop the counter counting.

11.5.9.1 Method declaration:

```
void Stop(void)
```

11.5.9.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
`<none>`		

11.5.9.3 Description:

This method is used to stop the counter counting when ticked by the attached `vrtaClock` device.

11.5.9.4 Return values:

VALUE	DESCRIPTION
`<none>`	

11.5.10 Action: Minimum

ACTION ID	DATA FORMAT
1	`%u`

This action sets the minimum value of the counter. It is the equivalent of the `SetMin()` method.

11.5.11 Action: Maximum

ACTION ID	DATA FORMAT
2	`%u`

This action sets the maximum value of the counter. It is the equivalent of the `SetMax()` method.

11.5.12 Action: Set

ACTION ID	DATA FORMAT
3	`%u`

This action sets the current value of the counter. It is the equivalent of the `Set-Val()` method.

11.5.13 Action: Start

ACTION ID	DATA FORMAT
4	`<none>`

This method is used to start the counter counting when ticked by the attached `vrtaClock` device. It is the equivalent of the `Start()` method.

11.5.14 Action: Stop

ACTION ID	DATA FORMAT
5	<none>

This method is used to stop the counter counting when ticked by the attached `vrtaClock` device. It is the equivalent of the `Stop()` method.

11.5.15 Action: Report

ACTION ID	DATA FORMAT
6	<none>

This action causes a Set event to be raised.

11.5.16 Event: Set

EVENT ID	DATA FORMAT
1	%u

This event contains the current value of the counter. It is raised in response to a Report action.

11.6 vrtaSensor

A `vrtaSensor` device models a sensor. That is, a device which takes input from one source, stores that input and then allows the input to be read by an application.

`vrtaSensor` represents a generic sensor; `vrtaSensorToggleSwitch` and `vrtaSensorMultiwaySwitch` are derived from `vrtaSensor` and represent more specialized sensors.

A sensor has a current value and a maximum value. The current value of the sensor can be set to a value between zero and the maximum value inclusive. Events are raised whenever the current value or maximum value changes.

When a sensor is created the current value is zero and the maximum value is 4294967295.

11.6.1 Method: vrtaSensor()

The constructor.

11.6.1.1 Method declaration:

```
vrtaSensor(const vrtaTextPtr name)
```

11.6.1.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
`Name`	INPUT	THE NAME OF THE VIRTUAL DEVICE.

11.6.1.3 Description:

This is the constructor used to create an instance of a `vrtaSensor` device.

11.6.1.4 Return values:

VALUE	DESCRIPTION
`<none>`	

11.6.2 Method: GetMax()

Get the maximum value.

11.6.2.1 Method declaration:

```
unsigned GetMax(void)
```

11.6.2.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
`<none>`		

11.6.2.3 Description:

This method is used to get the maximum value of the sensor.

11.6.2.4 Return values:

VALUE	DESCRIPTION
`<a value>`	THE MAXIMUM VALUE OF THE SENSOR.

11.6.3 Method: Value()

Get the current value.

11.6.3.1 Method declaration:

```
unsigned Value(void)
```

11.6.3.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
<none>		

11.6.3.3 Description:

This method is used to get the current value of the sensor.

11.6.3.4 Return values:

VALUE	DESCRIPTION
<a value>	THE CURRENT VALUE OF THE SENSOR.

11.6.4 Method: SetMax()

Set the maximum value.

11.6.4.1 Method declaration:

```
void SetMax(unsigned v)
```

11.6.4.2 Parameters:

PARAME-TER	INPUT/OUT-PUT	DESCRIPTION
V	INPUT	THE NEW MAXIMUM VALUE FOR THE SENSOR.

11.6.4.3 Description:

This method is used to set the maximum value of the sensor. If the current value of the sensor is greater than the new maximum value then the current value is set to zero.

11.6.4.4 Return values:

VALUE	DESCRIPTION
<none>	

11.6.5 Method: SetVal()

Set the current value.

11.6.5.1 Method declaration:

```
void SetVal(unsigned v)
```

11.6.5.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
`v`	INPUT	THE NEW VALUE FOR THE SENSOR.

11.6.5.3 Description:

This method is used to set the current value of the sensor. If the new current value of the sensor is greater than the maximum value then the current value is not set.

11.6.5.4 Return values:

VALUE	DESCRIPTION
`<none>`	

11.6.6 Action: Value

ACTION ID	DATA FORMAT
`1`	`%u`

This action sets the current value of the sensor. It is the equivalent of the `SetVal()` method.

11.6.7 Action: Maximum

ACTION ID	DATA FORMAT
`2`	`%u`

This action sets the maximum value of the sensor. It is the equivalent of the `SetMax()` method.

11.6.8 Event: Value

EVENT ID	DATA FORMAT
`1`	`%u`

This event contains the current value of the sensor. It is raised whenever the value of the sensor changes.

11.6.9 Event: Maximum

EVENT ID	DATA FORMAT
2	`%u`

This event contains the maximum value of the sensor. It is raised whenever the maximum value of the sensor changes.

11.7 vrtaSensorToggleSwitch

A `vrtaSensorToggleSwitch` is a special form of a sensor that has only two possible values, zero and one, corresponding to “off” and “on”.

When a `vrtaSensorToggleSwitch` is created its current value is zero.

11.7.1 Method: vrtaSensorToggleSwitch()

The constructor.

11.7.1.1 Method declaration:

```
vrtaSensorToggleSwitch(const vrtaTextPtr name)
```

11.7.1.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
`name`	INPUT	THE NAME OF THE VIRTUAL DEVICE.

11.7.1.3 Description:

This is the constructor used to create an instance of a `vrtaSensorToggleSwitch` device.

11.7.1.4 Return values:

VALUE	DESCRIPTION
`<none>`	

11.7.2 Method: Value()

Get the current value.

11.7.2.1 Method declaration:

```
unsigned Value(void)
```

11.7.2.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
<none>		

11.7.2.3 Description:

This method is used to get the current value of the sensor.

11.7.2.4 Return values:

VALUE	DESCRIPTION
<a value>	THE CURRENT VALUE OF THE SENSOR.

11.7.3 Method: SetVal()

Set the current value.

11.7.3.1 Method declaration:

```
void SetVal(unsigned v)
```

11.7.3.2 Parameters:

PARAME-TER	INPUT/OUT-PUT	DESCRIPTION
v	INPUT	THE NEW VALUE FOR THE SEN-SOR.

11.7.3.3 Description:

This method is used to set the current value of the sensor. If the new current value of the sensor is greater than 1 then the current value is not set.

11.7.3.4 Return values:

VALUE	DESCRIPTION
<none>	

11.7.4 Action: Position

ACTION ID	DATA FORMAT
1	%u:;0;1

This action sets the current value (position) of the sensor. It is the equivalent of the `SetVal()` method.

11.7.5 Event: Position

EVENT ID	DATA FORMAT
1	%u:;0;1

This event contains the current value (position) of the sensor. It is raised whenever the value of the sensor changes.

11.8 vrtaSensorMultiwaySwitch

A `vrtaSensorMultiwaySwitch` is a special form of a sensor that represents a switch with a number of possible positions. The number of positions is set when the device is created (but can be changed later).

When a `vrtaSensorMultiwaySwitch` is created its current value is zero.

11.8.1 Method: vrtaSensorMultiwaySwitch()

The constructor.

11.8.1.1 Method declaration:

```
vrtaSensorMultiwaySwitch(const vrtaTextPtr name,
                         unsigned ways)
```

11.8.1.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
name	INPUT	THE NAME OF THE VIRTUAL DEVICE.
ways	INPUT	THE NUMBER OF POSITIONS THE SWITCH MAY TAKE MINUS 1.

11.8.1.3 Description:

This is the constructor used to create an instance of a `vrtaSensorMultiwaySwitch` device. The sensor may have a value in the range zero to `ways` inclusive.

11.8.1.4 Return values:

VALUE	DESCRIPTION
<none>	

11.8.2 Method: GetMax()

Get the maximum value.

11.8.2.1 Method declaration:

```
unsigned GetMax(void)
```

11.8.2.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
<none>		

11.8.2.3 Description:

This method is used to get the maximum value of the sensor.

11.8.2.4 Return values:

VALUE	DESCRIPTION
<a value>	THE MAXIMUM VALUE OF THE SENSOR.

11.8.3 Method: Value()

Get the current value.

11.8.3.1 Method declaration:

```
unsigned Value(void)
```

11.8.3.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
<none>		

11.8.3.3 Description:

This method is used to get the current value of the sensor.

11.8.3.4 Return values:

VALUE	DESCRIPTION
<a value>	THE CURRENT VALUE OF THE SENSOR.

11.8.4 Method: SetMax()

Set the maximum value.

11.8.4.1 Method declaration:

```
void SetMax(unsigned v)
```

11.8.4.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
`v`	INPUT	THE NEW MAXIMUM VALUE FOR THE SENSOR.

11.8.4.3 Description:

This method is used to set the maximum value of the sensor (i.e. to override the value of the `ways` argument used in the constructor). If the current value of the sensor is greater than the new maximum value then the current value is set to zero.

11.8.4.4 Return values:

VALUE	DESCRIPTION
`<none>`	

11.8.5 Method: SetVal()

Set the current value.

11.8.5.1 Method declaration:

```
void SetVal(unsigned v)
```

11.8.5.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
`v`	INPUT	THE NEW VALUE FOR THE SENSOR.

11.8.5.3 Description:

This method is used to set the current value of the sensor. If the new current value of the sensor is greater than the maximum value then the current value is not set.

11.8.5.4 Return values:

VALUE	DESCRIPTION
`<none>`	

11.8.6 Action: Value

ACTION ID	DATA FORMAT
1	`%u`

This action sets the current value of the sensor. It is the equivalent of the `SetVal()` method.

11.8.7 Action: Maximum

ACTION ID	DATA FORMAT
2	`%u`

This action sets the maximum value of the sensor. It is the equivalent of the `SetMax()` method.

11.8.8 Event: Value

EVENT ID	DATA FORMAT
1	`%u`

This event contains the current value of the sensor. It is raised whenever the value of the sensor changes.

11.8.9 Event: Maximum

EVENT ID	DATA FORMAT
2	`%u`

This event contains the maximum value of the sensor. It is raised whenever the maximum value of the sensor changes.

11.9 vrtaActuator

A `vrtaActuator` device models an actuator. That is, a device which has its value set by an application and then signals that value to entities outside of the ECU. `vrtaActuator` represents a generic actuator; `vrtaActuatorLight`, `vrtaActuatorDimmableLight` and `vrtaActuatorMultiColorLight` are derived from `vrtaActuator` and represent more specialized actuators.

An actuator has a current value and a maximum value. The current value of the actuator can be set to a value between zero and the maximum value inclusive. Events are raised whenever the current value or maximum value changes.

When an actuator is created the current value is zero and the maximum value is 4294967295.

11.9.1 Method: vrtaActuator()

The constructor.

11.9.1.1 Method declaration:

```
vrtaActuator(const vrtaTextPtr name)
```

11.9.1.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
`name`	INPUT	THE NAME OF THE VIRTUAL DEVICE.

11.9.1.3 Description:

This is the constructor used to create an instance of a `vrtaActuator` device.

11.9.1.4 Return values:

VALUE	DESCRIPTION
`<none>`	

11.9.2 Method: GetMax()

Get the maximum value.

11.9.2.1 Method declaration:

```
unsigned GetMax(void)
```

11.9.2.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
`<none>`		

11.9.2.3 Description:

This method is used to get the maximum value of the actuator.

11.9.2.4 Return values:

VALUE	DESCRIPTION
`<a value>`	THE MAXIMUM VALUE OF THE ACTUATOR.

11.9.3 Method: Value()

Get the current value.

11.9.3.1 Method declaration:

```
unsigned Value(void)
```

11.9.3.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
<none>		

11.9.3.3 Description:

This method is used to get the current value of the actuator.

11.9.3.4 Return values:

VALUE	DESCRIPTION
<a value>	THE CURRENT VALUE OF THE ACTUATOR.

11.9.4 Method: SetMax()

Set the maximum value.

11.9.4.1 Method declaration:

```
void SetMax(unsigned v)
```

11.9.4.2 Parameters:

PARAME-TER	INPUT/OUT-PUT	DESCRIPTION
V	INPUT	THE NEW MAXIMUM VALUE FOR THE ACTUATOR.

11.9.4.3 Description:

This method is used to set the maximum value of the actuator. If the current value of the actuator is greater than the new maximum value then the current value is set to zero.

11.9.4.4 Return values:

VALUE	DESCRIPTION
<none>	

11.9.5 Method: SetVal()

Set the current value.

11.9.5.1 Method declaration:

```
void SetVal(unsigned v)
```

11.9.5.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
v	INPUT	THE NEW VALUE FOR THE ACTUATOR.

11.9.5.3 Description:

This method is used to set the current value of the actuator. If the new current value of the actuator is greater than the maximum value then the current value is not set.

11.9.5.4 Return values:

VALUE	DESCRIPTION
<none>	

11.9.6 Action: Value

ACTION ID	DATA FORMAT
1	%u

This action sets the current value of the actuator. It is the equivalent of the `SetVal()` method.

11.9.7 Action: Maximum

ACTION ID	DATA FORMAT
2	%u

This action sets the maximum value of the actuator. It is the equivalent of the `SetMax()` method.

11.9.8 Event: Value

EVENT ID	DATA FORMAT
1	%u

This event contains the current value of the actuator. It is raised whenever the value of the actuator changes.

11.9.9 Event: Maximum

EVENT ID	DATA FORMAT
2	%u

This event contains the maximum value of the actuator. It is raised whenever the maximum value of the actuator changes.

11.10 vrtaActuatorLight

A `vrtaActuatorLight` is a special form of an actuator that represents a light. A `vrtaActuatorLight` has two possible values zero and one, representing “off” and “on”.

When a `vrtaActuatorLight` is created its current value is zero.

11.10.1 Method: vrtaActuatorLight()

The constructor.

11.10.1.1 Method declaration:

```
vrtaActuatorLight(const vrtaTextPtr name)
```

11.10.1.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
name	INPUT	THE NAME OF THE VIRTUAL DEVICE.

11.10.1.3 Description:

This is the constructor used to create an instance of a `vrtaActuatorLight` device.

11.10.1.4 Return values:

VALUE	DESCRIPTION
<none>	

11.10.2 Method: Value()

Get the current value.

11.10.2.1 Method declaration:

```
unsigned Value(void)
```

11.10.2.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
<none>		

11.10.2.3 Description:

This method is used to get the current value of the actuator.

11.10.2.4 Return values:

VALUE	DESCRIPTION
<a value>	THE CURRENT VALUE OF THE ACTUATOR.

11.10.3 Method: SetVal()

Set the current value.

11.10.3.1 Method declaration:

```
void SetVal(unsigned v)
```

11.10.3.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
v	INPUT	THE NEW VALUE FOR THE ACTUATOR.

11.10.3.3 Description:

This method is used to set the current value of the actuator. If the new current value of the actuator is greater than 1 then the current value is not set.

11.10.3.4 Return values:

VALUE	DESCRIPTION
<none>	

11.10.4 Action: Value

ACTION ID	DATA FORMAT
1	%u

This action sets the current value of the actuator. It is the equivalent of the `SetVal()` method.

11.10.5 Event: Value

EVENT ID	DATA FORMAT
1	%u

This event contains the current value of the actuator. It is raised whenever the value of the actuator changes.

11.11 vrtaActuatorDimmableLight

A `vrtaActuatorDimmableLight` is a special form of an actuator that represents a light whose brightness can be set. The number of possible brightness levels is set when the actuator is created (but can be changed later).

When a `vrtaActuatorDimmableLight` is created its current value is zero.

11.11.1 Method: vrtaActuatorDimmableLight()

The constructor.

11.11.1.1 Method declaration:

```
vrtaActuatorDimmableLight(const vrtaTextPtr name,
                          unsigned levels)
```

11.11.1.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
name	INPUT	THE NAME OF THE VIRTUAL DEVICE.
levels	INPUT	THE NUMBER OF BRIGHTNESS LEVELS MINUS 1

11.11.1.3 Description:

This is the constructor used to create an instance of a `vrtaActuatorDimmableLight` device. The actuator may have a value in the range zero to `levels` inclusive.

11.11.1.4 Return values:

VALUE	DESCRIPTION
<none>	

11.11.2 Method: GetMax()

Get the maximum value.

11.11.2.1 Method declaration:

```
unsigned GetMax(void)
```

11.11.2.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
<none>		

11.11.2.3 Description:

This method is used to get the maximum value of the actuator.

11.11.2.4 Return values:

VALUE	DESCRIPTION
<a value>	THE MAXIMUM VALUE OF THE ACTUATOR.

11.11.3 Method: Value()

Get the current value.

11.11.3.1 Method declaration:

```
unsigned Value(void)
```

11.11.3.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
<none>		

11.11.3.3 Description:

This method is used to get the current value of the actuator.

11.11.3.4 Return values:

VALUE	DESCRIPTION
<a value>	THE CURRENT VALUE OF THE ACTUATOR.

11.11.4 Method: SetMax()

Set the maximum value.

11.11.4.1 Method declaration:

```
void SetMax(unsigned v)
```

11.11.4.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
`v`	INPUT	THE NEW MAXIMUM VALUE FOR THE ACTUATOR.

11.11.4.3 Description:

This method is used to set the maximum value of the actuator (i.e. to override the value of the `levels` argument used in the constructor). If the current value of the actuator is greater than the new maximum value then the current value is set to zero.

11.11.4.4 Return values:

VALUE	DESCRIPTION
`<none>`	

11.11.5 Method: SetVal()

Set the current value.

11.11.5.1 Method declaration:

```
void SetVal(unsigned v)
```

11.11.5.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
`v`	INPUT	THE NEW VALUE FOR THE ACTUATOR.

11.11.5.3 Description:

This method is used to set the current value of the actuator. If the new current value of the actuator is greater than the maximum value then the current value is not set.

11.11.5.4 Return values:

VALUE	DESCRIPTION
`<none>`	

11.11.6 Action: Value

ACTION ID	DATA FORMAT
1	`%u`

This action sets the current value of the actuator. It is the equivalent of the `SetVal()` method.

11.11.7 Action: Maximum

ACTION ID	DATA FORMAT
2	`%u`

This action sets the maximum value of the actuator. It is the equivalent of the `SetMax()` method.

11.11.8 Event: Value

EVENT ID	DATA FORMAT
1	`%u`

This event contains the current value of the actuator. It is raised whenever the value of the actuator changes.

11.11.9 Event: Maximum

EVENT ID	DATA FORMAT
2	`%u`

This event contains the maximum value of the actuator. It is raised whenever the maximum value of the actuator changes.

11.12 vrtaActuatorMultiColorLight

A `vrtaActuatorMultiColorLight` is a special form of an actuator that represents a light whose color can be set. The number of possible colors is set when the actuator is created (but can be changed later).

When a `vrtaActuatorMultiColorLight` is created its current value is zero.

11.12.1 Method: vrtaActuatorMultiColorLight()

The constructor.

11.12.1.1 Method declaration:

```
vrtaActuatorMultiColorLight(const vrtaTextPtr name,
                            unsigned colors)
```

11.12.1.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
`name`	INPUT	THE NAME OF THE VIRTUAL DEVICE.
`colors`	INPUT	THE NUMBER OF COLORS MINUS 1

11.12.1.3 Description:

This is the constructor used to create an instance of a `vrtaActuatorMultiColorLight` device. The actuator may have a value in the range zero to `colors` inclusive.

11.12.1.4 Return values:

VALUE	DESCRIPTION
`<none>`	

11.12.2 Method: GetMax()

Get the maximum value.

11.12.2.1 Method declaration:

```
unsigned GetMax(void)
```

11.12.2.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
`<none>`		

11.12.2.3 Description:

This method is used to get the maximum value of the actuator.

11.12.2.4 Return values:

VALUE	DESCRIPTION
`<a value>`	THE MAXIMUM VALUE OF THE ACTUATOR.

11.12.3 Method: Value()

Get the current value.

11.12.3.1 Method declaration:

```
unsigned Value(void)
```

11.12.3.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
<none>		

11.12.3.3 Description:

This method is used to get the current value of the actuator.

11.12.3.4 Return values:

VALUE	DESCRIPTION
<a value>	THE CURRENT VALUE OF THE ACTUATOR.

11.12.4 Method: SetMax()

Set the maximum value.

11.12.4.1 Method declaration:

```
void SetMax(unsigned v)
```

11.12.4.2 Parameters:

PARAME-TER	INPUT/OUT-PUT	DESCRIPTION
v	INPUT	THE NEW MAXIMUM VALUE FOR THE ACTUATOR.

11.12.4.3 Description:

This method is used to set the maximum value of the actuator (i.e. to override the value of the `colors` argument used in the constructor). If the current value of the actuator is greater than the new maximum value then the current value is set to zero.

11.12.4.4 Return values:

VALUE	DESCRIPTION
<none>	

11.12.5 Method: SetVal()

Set the current value.

11.12.5.1 Method declaration:

```
void SetVal(unsigned v)
```

11.12.5.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
`v`	INPUT	THE NEW VALUE FOR THE ACTUATOR.

11.12.5.3 Description:

This method is used to set the current value of the actuator. If the new current value of the actuator is greater than the maximum value then the current value is not set.

11.12.5.4 Return values:

VALUE	DESCRIPTION
`<none>`	

11.12.6 Action: Value

ACTION ID	DATA FORMAT
`1`	`%u`

This action sets the current value of the actuator. It is the equivalent of the `SetVal()` method.

11.12.7 Action: Maximum

ACTION ID	DATA FORMAT
`2`	`%u`

This action sets the maximum value of the actuator. It is the equivalent of the `SetMax()` method.

11.12.8 Event: Value

EVENT ID	DATA FORMAT
`1`	`%u`

This event contains the current value of the actuator. It is raised whenever the value of the actuator changes.

11.12.9 Event: Maximum

EVENT ID	DATA FORMAT
2	`%u`

This event contains the maximum value of the actuator. It is raised whenever the maximum value of the actuator changes.

11.13 vrtaCompare

A `vrtaCompare` device represents a comparator that may be attached to any of the following devices

- vrtaUpCounter
- vrtaDownCounter
- vrtaSensor
- vrtaSensorToggleSwitch
- vrtaSensorMultiwaySwitch
- vrtaActuator
- vrtaActuatorLight
- vrtaActuatorDimmableLight
- vrtaActuatorMultiColorLight

It will generate an interrupt when the current value of the attached device reaches a specified match value.

Multiple `vrtaCompare` devices may be attached to the same device.

11.13.1 Method: vrtaCompare()

The constructor.

11.13.1.1 Method declaration:

```
vrtaCompare(const vrtaTextPtr name,
            vrtaComparable &source,
            unsigned match,
            unsigned vector)
```

11.13.1.2 Parameters:

PARAM-ETER	INPUT/OUT-PUT	DESCRIPTION
`name`	INPUT	THE NAME OF THE VIRTUAL DEVICE.
`source`	INPUT	THE DEVICE TO WHICH TO ATTACH.
`match`	INPUT	THE MATCH VALUE.

vector	INPUT	THE INTERRUPT VECTOR NUMBER TO BE GENERATED OR ZERO FOR NO INTERRUPT.

11.13.1.3 Description:

This is the constructor used to create an instance of a `vrtaCompare` device. The `vrtaCompare` device will raise interrupt number `vector` when the current value of the device specified by `source` reaches the value `match`. (Note that `vrtaCompare` will not enable the interrupt vector. This must be done by sending an Unmask action to the ICU device.)

11.13.1.4 Return values:

VALUE	DESCRIPTION
<none>	

11.13.2 Method: GetMatch()

Get the match value.

11.13.2.1 Method declaration:

```
unsigned GetMatch(void)
```

11.13.2.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
<none>		

11.13.2.3 Description:

This method is used to get the current match value of the device.

11.13.2.4 Return values:

VALUE	DESCRIPTION
<a value>	THE CURRENT MATCH VALUE.

11.13.3 Method: SetMatch()

Set the match value.

11.13.3.1 Method declaration:

```
void SetMatch(unsigned val)
```

11.13.3.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
`val`	INPUT	THE NEW MATCH VALUE.

11.13.3.3 Description:

This method is used to set the match value.

11.13.3.4 Return values:

VALUE	DESCRIPTION
`<none>`	

11.13.4 Method: IncrementMatch()

Increment the match value.

11.13.4.1 Method declaration:

```
unsigned IncrementMatch(unsigned val)
```

11.13.4.2 Parameters:

PARAM-ETER	INPUT/OUT-PUT	DESCRIPTION
`val`	INPUT	THE AMOUNT BY WHICH THE MATCH VALUE SHOULD BE INCREMENTED.

11.13.4.3 Description:

This method is used to increment the match value.

11.13.4.4 Return values:

VALUE	DESCRIPTION
`<a value>`	THE NEW MATCH VALUE.

11.13.5 Method: SetVector()

Set the interrupt vector number.

11.13.5.1 Method declaration:

```
void SetVector(unsigned val)
```

11.13.5.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
`val`	INPUT	THE NEW INTERRUPT VECTOR NUMBER.

11.13.5.3 Description:

This method is used to set the interrupt vector number. If the interrupt vector number is set to zero then no interrupted will be generated.

11.13.5.4 Return values:

VALUE	DESCRIPTION
`<none>`	

11.13.6 Action: Match

ACTION ID	DATA FORMAT
`1`	`%u`

This action sets the match value. It is the equivalent of the `SetMatch()` method.

11.13.7 Action: Vector

ACTION ID	DATA FORMAT
`2`	`%u`

This action sets the interrupt vector number. It is the equivalent of the `SetVector()` method.

11.13.8 Event: Match

EVENT ID	DATA FORMAT
`1`	`%u`

This event contains the match value. It is raised whenever the current value of the attached device reaches the match value of the `vrtaCompare` device.

11.14 vrtaIO

A `vrtaIO` device represents an array of 32-bit I/O cells that may be written and read by an application.

11.14.1 Method: vrtaIO()

The constructor.

11.14.1.1 Method declaration:

```
vrtaIO(const vrtaTextPtr name, unsigned elements)
```

11.14.1.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
`name`	INPUT	THE NAME OF THE VIRTUAL DEVICE.
`elements`	INPUT	THE NUMBER OF I/O CELLS TO BE USED.

11.14.1.3 Description:

This is the constructor used to create an instance of a `vrtaIO` device. The `vrtaIO` device will contain an array of `elements` I/O cells. The I/O cells will have offsets in the range zero to `elements` – 1 inclusive.

11.14.1.4 Return values:

VALUE	DESCRIPTION
`<none>`	

11.14.2 Method: SetValue()

Set the value of an I/O cell.

11.14.2.1 Method declaration:

```
void SetValue(unsigned offset, unsigned value)
```

11.14.2.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
`offset`	INPUT	THE OFFSET OF THE I/O CELL TO BE SET.
`value`	INPUT	THE VALUE TO WRITE.

11.14.2.3 Description:

This method is used to set the value of an I/O cell.

11.14.2.4 Return values:

VALUE	DESCRIPTION
<none>	

11.14.3 Method: SetValues()

Set the value of multiple I/O cells.

11.14.3.1 Method declaration:

```
void SetValues(unsigned offset,
               const unsigned *values,
               unsigned number)
```

11.14.3.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
`offset`	INPUT	THE OFFSET OF THE FIRST I/O CELL TO BE SET.
`values`	INPUT	AN ARRAY OF VALUES TO WRITE.
`number`	INPUT	THE NUMBER OF VALUES TO WRITE.

11.14.3.3 Description:

This method is used to set the values of multiple I/O cells. The `number` values from `values[]` are written to the array of I/O cells starting at `offset`.

11.14.3.4 Return values:

VALUE	DESCRIPTION
<none>	

11.14.4 Method: GetValue()

Get the value of an I/O cell.

11.14.4.1 Method declaration:

```
unsigned GetValue(unsigned offset) const
```

11.14.4.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
`offset`	INPUT	THE OFFSET OF THE I/O CELL GET.

11.14.4.3 Description:

This method is used to get the value of an I/O cell.

11.14.4.4 Return values:

VALUE	DESCRIPTION
`<a value>`	THE VALUE OF THE SPECIFIED I/O CELL.

11.14.5 Method: GetValues()

Get the values of all I/O cells.

11.14.5.1 Method declaration:

```
const unsigned *GetValues(void) const
```

11.14.5.2 Parameters:

PARAMETER	INPUT/OUTPUT	DESCRIPTION
`<none>`		

11.14.5.3 Description:

This method is used to get the values of all I/O cells.

11.14.5.4 Return values:

VALUE	DESCRIPTION
`<a pointer>`	A POINTER TO THE ARRAY OF I/O CELLS.

11.14.6 Action: Value

ACTION ID	DATA FORMAT
1	`%u,%u`

This action sets the value of an I/O cell. The first number in the action data is the I/O cell offset and the second number is the value to write. This action is equivalent to the `SetValue()` method.

11.14.7 Action: Values

ACTION ID	DATA FORMAT
2	`%a`

This action sets the values of multiple I/O cells. The action data is an array of values to write to the I/O cell array starting at offset zero. This action is equivalent to the `SetValues()` method with the `offset` argument set to zero.

11.14.8 Action: GetValue

ACTION ID	DATA FORMAT
3	%u

This action causes a Value event to be raised for the offset specified in the action data.

11.14.9 Action: GetValues

ACTION ID	DATA FORMAT
4	<none>

This action causes a Values event to be raised.

11.14.10 Event: Value

EVENT ID	DATA FORMAT
1	%u,%u(%u)

This event contains the value of an I/O cell. The first number is the offset of the I/O cell. The second number is the value of the I/O cell. If the event is raised in response to a Value action then the I/O cell offset is in the action data. If the event is queried then the I/O cell offset is in the input data.

11.14.11 Event: Values

EVENT ID	DATA FORMAT
2	%a

This event contains the values of all of the I/O cells.

12 VRTA Communications Libraries

A key feature of VRTA ECUs is that you are able to communicate with the virtual devices inside an ECU from external programs.

You can **monitor** what is happening inside a VECU by reading data back from the virtual device's **events**.

You can **control** what is happening inside a VECU by sending data to the virtual device **actions**.

In addition to communicating with VECUs, you will also need to talk to instances of **vrtaServer** on local or remote PCs. This gives you the ability to control VECUs.

Communication with VECUs and vrtaServer is achieved using simple TCP/IP connections with open interfaces.

Whilst it is possible for you to write a client application that uses TCP/IP directly to talk to the VECU, we provide communication libraries that you can use to make this significantly easier.

This chapter describes the features of the communication libraries and how to use them for the common tasks that you are likely to need. It does not attempt to give an exhaustive parameterized API guide for each element in the libraries. That information can be more reliably obtained from a quick scan of the source code and executables that we supply. Rather it gives code snippets that show the recommended way to do typical tasks.

12.1 Languages Supported

12.1.1 .NET

The .NET VRTA communication code is supplied as a DLL that gets installed at `<RTAOS-bin>\VRTACommsLinks\DotNet\VRTAComms.DLL`. You will have to reference this DLL in your project, and ensure that it is available at run-time when you deploy your client application.

The DLL requires version 4 of the .NET Framework, because it takes advantage of dynamic types.

12.1.2 Java

The Java VRTA communication code is supplied as a JAR file that gets installed at `<RTAOS-bin>\VRTACommsLinks\Java\VRTAcomms.jar`. You will have to reference this file in your project, and ensure that it is available at run-time when you deploy your client application.

This code has only been tested on Windows PCs. It has not been tested on Linux.

12.1.3 Ruby

The Ruby VRTA communication code is supplied as source code that gets installed at `<RTAOS-bin>\VRTACommsLinks\Ruby`.

You will typically add `<RTAOS-bin>\VRTACommsLinks\Ruby` to your Ruby load path when using this library.

12.1.4 Ruby_in_Bake

Instead of using a complete Ruby installation, you can take advantage of the built-in Ruby console that is in Bake.exe, which ships with RTA-OS.

If you run `Bake -c` or `Bake --console` then you start an interactive console, just like the normal Ruby irb console.

If you add a ruby filename, Bake will run the Ruby code and quit.

Additionally if you include the `--vrta` option, Bake will preload the Ruby VRTA communications library for you.

This means that you can, for example, run the test program in the Ruby_in_Bake folder thus:

```
Bake.exe -c --vrta test.rb
```

12.1.5 jRuby

The jRuby VRTA communication code is supplied as a single file that gets installed at `<RTAOS-bin>\VRTACommsLinks\jRuby`. This file provides a small wrapper around the Java `VRTAcomms.jar` file that gives you an API identical to the standard Ruby one, but within a Java/jRuby environment.

You will typically add `<RTAOS-bin>\VRTACommsLinks\jRuby` and `<RTAOS-bin>\VRTACommsLinks\Java` to your jRuby load path when using this library.

12.1.6 Python

There is experimental Python communication code that gets installed at `<RTAOS-bin>\VRTACommsLinks\Python`. This has been contributed by a third-party and may be incomplete – especially for VRTA-ux/WSL.

12.1.7 C++

The C++ VRTA communication code is supplied as source code that gets installed at `<RTAOS-bin>\VRTACommsLinks\CPP`.

You can either reference these files directly from your project, or copy them into your project folder. The only down-side to the second option is that you won't automatically benefit from code updates if you install later versions of VRTA.

An alternative is to compile the three .cpp files (`vrtaCore.cpp`, `vrtaClientSupport.cpp` and `VESLib\VesLib.cpp`) into a library and use that in your project.

The code is portable to different C++ compilers on both Windows and Linux.

Some C++ types used in the comms code are described here.

12.1.7.1 `VesLibEcuInfoType`

The `VesLibEcuInfoType` type is used to identify Virtual ECU executables to the server library. `VesLibEcuInfoType` has the definition:

```
typedef struct {
    char path[VESLIB_MAX_PATH];
} VesLibEcuInfoType;
```

The `path` field should contain the full path of the Virtual ECU executable.

12.1.7.2 `VesLibEcuAliasType`

The `VesLibEcuAliasType` type is used to contain Virtual ECU aliases. `VesLibEcuAliasType` has the definition:

```
typedef struct {
    char name[VESLIB_MAX_PATH];
} VesLibEcuAliasType;
```

12.2 Controlling the VECU lifecycle

The **vrtaServer** program is the central component that is needed to communicate with VECUs. This is because it is normally the only VRTA component on a machine (Windows or Linux) that listens on a predetermined TCP port[17].

vrtaServer is responsible for keeping track of all the VECUs that are running on its PC. When a VECU starts running, it informs vrtaServer that it has started and which private TCP port it is listening on. When the VECU terminates, vrtaServer gets notified.

This means that an external program can ask vrtaServer what VECUs are running, and how to communicate with them. It acts as a central directory for the VECUs on its host PC.

vrtaServer is also able to start and stop VECUs. Your application can tell an instance of vrtaServer on a remote PC to load and start a VECU, and it will then tell you how to talk to that VECU.

12.2.1 Aliases

Each VECU instance that vrtaServer knows about is assigned a unique name, which we call an **alias**. This is used to ensure that programs are able to communicate with the correct VECU instance at all times. If two independent client programs each start the same VECU, vrtaServer will know that there are two instances of the VECU running and will give different aliases to each of them so that one client cannot interfere with the VECU that belongs to the other.

The actual value assigned to an alias is based on the name of the VECU on disk, but this is simply a convenience – you should not rely on the alias value containing meaningful information. An alias should be thought of as a token that vrtaServer passes to you when you ask about a VECU. That token gets used in subsequent messages that refer to the specific VECU.

Aliases should be treated a bit like file handles. When vrtaServer returns an alias to you, it is like opening a file. The alias remains in existence until (a) the VECU terminates and (b) you release (close) the alias. If you don't release the alias then vrtaServer will keep hold of 'zombie' aliases until it gets restarted[18].

---

[17] Actually it will use the first free port that it finds in the list 17185, 31765, 26000

[18] vrtaServer is not really quite that dumb – it can spot when client applications go away and do the release itself – but it can't do that for long-running client applications so you should always try to release aliases correctly.

The code examples below explain when you need to take care of this issue.

12.2.2 Talking to vrtaServer

Before we can talk to vrtaServer, we need to know one thing – its network location (IP address). This will be a text string that might be of the numeric form such as "192.168.1.14", or the DNS name form such as "MyPC". If vrtaServer is on the same PC, the forms "127.0.0.1" or "localhost" are commonly used.

Obviously vrtaServer must be running before you can talk to it. However, as a convenience, the communications library code will try to start vrtaServer.exe automatically on Windows if it is not detected initially. This only happens when the host PC is the same one where the code is running (localhost / 127.0.0.1).

12.2.2.1 .NET

The vrtaServer communications are handled by the `VECUFinder` class. You can create an independent instance for each vrtaServer connection that you need.

```
using VRTAComms;
…
var serverLink = new VECUFinder(host_address);
    // Raises Exception on failure

// Connection is made
// serverLink calls can be made
…
```

12.2.2.2 Java

The vrtaServer communications are handled by the `VECUFinder` class. You can create an independent instance for each vrtaServer connection that you need.

```
import com.etas.vrta.comms.*;
…
VECUFinder serverLink = new VECUFinder(host_address);
    // Raises RuntimeException on failure

// Connection is made
// serverLink calls can be made
…
```

12.2.2.3 Ruby

The vrtaServer communications are handled by the `VECUFinder` class. You can create an independent instance for each vrtaServer connection that you need.

```
require 'VRTAComms'
…
server_link = VECUFinder.new(host_address)
    # Raises String exception on failure
# Connection is made
# server_link calls can be made
```

```
…
```

12.2.2.4 C++ source

The code to communicate with vrtaServer is split out into the 'VESLib[19]', which gets installed at `<RTAOS-bin>\VRTACommsLinks\CPP\VESLib`. (Bear in mind that it makes use of the support code in the file `vrtaClientSupport.cpp`, so that still needs to be part of your project.)

```
#include "VesLib.h
#include "vrtaClientSupport.h"
…
// Specify address of the PC that is running vrtaServer
// The 0 means communicate on the normal port number(s)
VesLibSelectServer(host_address, 0);
if (VesLibInitialize() != VESLIB_STATUS_OK) {
    // Error connecting
    …
}
// Connection is made
// Other VESLib calls can be made here
…
// Finished talking to vrtaServer
VesLibExit();
```

Note that the VESLib implementation is designed to handle the connection to a single vrtaServer instance. You should call `VesLibExit` before re-initializing to connect to a different vrtaServer host.

12.2.2.5 C++ VesLib.dll

The same code that is described above is available in a Windows DLL called VesLib.dll. This gets installed at `<RTAOS-bin>`. An import library is not provided for the DLL since the format of import libraries varies between compilers. If you wish to use `veslib.dll` you will need to use the Windows `LoadLibrary()` function to load the DLL and then the Windows `GetProcAddress()` function to get pointers to the library API functions within the DLL. Prototypes for all of the library API functions can be found in `veslib.h`. Any source files that wish to use the library should include the `veslib.h` header file.

[19] Virtual ECU Server library

12.2.3 Starting a new VECU

You can start a VECU running in one of two ways.

Loading a VECU causes the application to get loaded from disk and starts all communication-related threads. It does not allow the main OS thread to start. You can communicate with the virtual devices, but the OS thread will not start until you send a start message to the ApplicationManager device.

Running a VECU causes the application to get loaded from disk and starts all threads – including the main OS thread.

Note that the path to the executable must be from the point of view of vrtaServer running on the host machine. On Windows this might be "`c:\vecus\my_vecu.exe`", whilst on Linux it might be "`/home/user/vecus/my_vecu.vrta_ecu`".

12.2.3.1 .NET

```
using VRTAComms;

// Load but don't start OS
vecuLink1 = serverLink.LoadVECU(path_to_exe);
    // Returns null if the VECU is not loaded
…
// Start OS
serverLink.StartVECU(vecuLink1);

// Load and start OS
vecuLink2 = serverLink.RunVECU(path_to_exe);
    // Returns null if the VECU is not loaded

// Release the aliases when no longer needed
serverLink.ReleaseVECU(vecuLink1);
serverLink.ReleaseVECU(vecuLink2);
```

By default, a VECU that is started by vrtaServer will be loaded with its working directory set to the same directory where the vrtaServer executable resides. You can override this behavior when using the `LoadVECU` or `RunVECU` methods by passing in an additional `-home` option:

```
// Start the VECU in the directory where its
// executable resides
vecu_link1 = serverLink.LoadVECU(path_to_exe, "-home=@");

# Start the VECU in the specified directory
vecu_link2 = server_link.RunVECU(path_to_exe, "-home=c:\\");
```

Any other options that are supplied are passed directly to the VECU's command-line.

The VECUFinder class has convenience methods that you can use to load/run a VECU in a single step, without having a serverLink:

```
using VRTAComms;

// Load but don't start OS
vecuLink1 = VECUFinder.Load(host_address, path_to_exe);
    // Returns null if the VECU is not loaded
…
// Start OS
vecuLink1.Start();

// Load and start OS
vecuLink2 = VECUFinder.Run(host_address, path_to_exe);
    // Returns null if the VECU is not loaded

// Alias does not have to be released
vecuLink1.Terminate();
vecuLink2.Terminate();
```

The `VECUFinder.Load` and `VECUFinder.Run` methods can be passed additional options in the same way as `LoadVECU` and `RunVECU`.

12.2.3.2 Java

```
import com.etas.vrta.comms.*;
…
// Load but don't start OS
vecuLink1 = serverLink.loadVECU(path_to_exe);
    // Returns null if the VECU is not loaded

// Start OS
serverLink.startVECU(vecuLink1);

// Load and start OS
vecuLink2 = serverLink.runVECU(path_to_exe);
    // Returns null if the VECU is not loaded

// Release the aliases when no longer needed
serverLink.releaseVECU(vecuLink1);
serverLink.releaseVECU(vecuLink2);
```

By default, a VECU that is started by vrtaServer will be loaded with its working directory set to the same directory where the vrtaServer executable resides. You can override this behavior when using the `LoadVECU` or `RunVECU` methods by passing in an additional `-home` option:

```
// Start the VECU in the directory where its
// executable resides
vecu_link1 = serverLink.loadVECU(path_to_exe, "-home=@");

# Start the VECU in the specified directory
vecu_link2 = server_link.runVECU(path_to_exe, "-home=c:\\");
```

Any other options that are supplied are passed directly to the VECU's command-line.

The VECUFinder class has convenience methods that you can use to load/run a VECU in a single step, without having a serverLink:

```
import com.etas.vrta.comms.*;
…

// Load but don't start OS
vecuLink1 = VECUFinder.load(host_address, path_to_exe);
    // Returns null if the VECU is not loaded

// Start OS
vecuLink1.start();

// Load and start OS
vecuLink2 = VECUFinder.run(host_address, path_to_exe);
    // Returns null if the VECU is not loaded

// Alias does not have to be released
vecuLink1.terminate();
vecuLink2.terminate();
```

The `VECUFinder.load` and `VECUFinder.run` methods can be passed additional options in the same way as `loadVECU` and `runVECU`.

12.2.3.3 Ruby

In Ruby, you simply pass the path of the executable to a server link, and it will return an instance of a VECULink that you can use to communicate with the VECU (see later).

```
require 'VRTAComms'
…
# Load but don't start OS
vecu_link1 = server_link.load_vecu(path_to_exe)
# Returns nil if the VECU is not loaded
…
# Start OS
server_link.start_vecu(vecu_link1)


# Load and start OS
vecu_link2 = server_link.run_vecu(path_to_exe)
# Returns nil if the VECU is not running

# Release the aliases when no longer needed
server_link.release_vecu(vecu_link1)
server_link.release_vecu(vecu_link2)
```

By default, a VECU that is started by vrtaServer will be loaded with its working directory set to the same directory where the vrtaServer executable resides. You can override this behavior when using the `load_vecu` or `run_vecu` methods by passing in an additional `:home` option:

```
# Start the VECU in the directory where its
# executable resides
vecu_link1 = server_link.load_vecu(path_to_exe, :home => '@')

# Start the VECU in the specified directory
vecu_link2 = server_link.run_vecu(path_to_exe, :home =>
"C:\\my_exe_location")
```

Other options that can be passed are:

- `:gui => true` : On VRTA-win, the VECU's GUI will be shown
- `:priority => num` : The command-line option '`-priority=num`' is passed to the VECU
- `:core => num` : The command-line option '`-core=num`' is passed to the VECU
- `:params => ['-a', '-b']` : The options in the array are passed as command-line options to the VECU

The VECUFinder class has convenience methods that you can use to load/run a VECU in a single step, without having to create a server_link:

```
require 'VRTAComms'
…
vecu_link1 = VECUFinder.load(host_address, path_to_exe)
    # Returns nil if the VECU is not loaded

vecu_link2 = VECUFinder.run(host_address, path_to_exe)
    # Returns nil if the VECU is not running
…
# Alias does not have to be released
vecu_link1.terminate
vecu_link2.terminate
```

The same options that are supported by the `load_vecu` and `run_vecu` methods are also available here.

12.2.3.4 C++

You currently have to do somewhat more work for the C++ interface to load a VECU. First you must get an alias, then you use it to load the VECU. If you get a port number back, then you can create a vrtaDiagnosticLinkClient and subsequently a vrtaECU instance.

A startMode of VesLibSMSlave is the equivalent of 'load' in the other libraries – the OS thread is not started.

A startMode of VesLibSMAuto is the equivalent of 'run' in the other libraries – the OS thread is started.

```
#include "VesLib.h
#include "vrtaClientSupport.h"
…
vrtaECU *vecuLink;
VesLibStatusType status;
int port = 0;
VesLibEcuInfoType ecu;
VesLibEcuAliasType alias;

// Create a new alias for the VECU
strcpy(ecu.name, path_to_exe);
status = VesLibCreateAlias(&ecu, &alias);
…

// Load the VECU
status = VesLibLoadECU(
  &alias,      // (VesLibEcuAliasType *)
  startMode,   // VesLibSMAuto or VesLibSMSlave
  displayMode, // VesLibDMSilent or VesLibDMSilent
  cmd,         // (char *) Extra command-line options
  &port        // (int *) The returned port number
);

// Connect to the VECU
if (port != 0) {
 vrtaDiagnosticLinkClient *link=new vrtaDiagnosticLinkClient();
 link->Connect(host_address, port);
 if (link->Connected()) {
  vecu_link = new vrtaECU();
  vecu_link->Load(link);
 }
}

// vecu_link gets used from here
```

Note that if additional command-line options such as the `-home` option described above for .Net and Java are desired, they must be passed in the `cmd` parameter to `VesLibLoadECU`. They must be \n separated, not space-separated.

12.2.4 Connecting to an existing VECU

If you know that a specific VECU is already running, you can 'attach' to it and get back an object that you can use to talk to its virtual devices.

12.2.4.1 .NET

In .NET, you can pass the full path to a VECU to a serverLink, and it will return an instance of a VECULink that you can use to communicate with the VECU (see later).

If there is more than one instance of the same VECU running then the one which started earliest is returned.

```
using VRTAComms;
…
// Attach to ECU
vecuLink = serverLink.AttachVECU(path_to_exe);
    // Returns null if the VECU is not loaded
serverLink.ReleaseVECU(vecuLink);
```

12.2.4.2 Java

```
import com.etas.vrta.comms.*;
…
// Attach to ECU
vecuLink = serverLink.attachVECU(path_to_exe);
    // Returns null if the VECU is not loaded
serverLink.releaseVECU(vecuLink);
```

12.2.4.3 Ruby

In Ruby, you can pass the full path to a VECU to a server_link, and it will return an instance of a VECULink that you can use to communicate with the VECU (see later).

If there is more than one instance of the same VECU running then the one which started earliest is returned.

```
require 'VRTAComms'
…
vecu_link = server_link.attach_vecu(path_to_exe)
    # Returns nil if the VECU is not running
…
# Release the alias when no longer needed
server_link.release_vecu(vecu_link)
```

Hint: The VECUFinder class has a convenience method that you can use to attach to a VECU in a single step, without having a server_link:

```
require 'VRTAComms'
…
vecu_link = VECUFinder.attach(host_address, path_to_exe)
    # Returns nil if the VECU is not running
```

```
…
# Alias does not have to be released
```

12.2.4.4 C++

The code to attach to an existing VECU is similar to that used to load one.

```
#include "VesLib.h
#include "vrtaClientSupport.h"
…
int port = 0;
int count;
VesLibStatusType status;
VesLibEcuAliasType *aliases;
VesLibEcuInfoType ecu;
vrtaECU *vecuLink = NULL;

// Find alias for the VECU
strcpy(ecu.name, path_to_exe);
status = VesLibGetAliases(&ecu, &aliases, &count);
if (count == 0) {
   … // Cannot attach - not loaded
}

// Attach to the VECU
status = VesLibAttachToECU(
  aliases[0],  // (VesLibEcuAliasType *)
  &port        // (int *) The returned port number
);

VesLibFreeMemory(aliases); // Allocated in VesLibGetAliases

// Connect to the VECU
if (port != 0) {
 vrtaDiagnosticLinkClient *link=new vrtaDiagnosticLinkClient();
 link->Connect(host_address, port);
 if (link->Connected()) {
  vecu_link = new vrtaECU();
  vecu_link->Load(link);
 }
}

// vecu_link gets used from here
```

12.2.5 Detaching from a VECU

You can detach from a VECU without terminating it, and it will continue running normally.

12.2.5.1 .NET

```
using VRTAComms;
…
serverLink.ReleaseVECU(vecuLink);
```

12.2.5.2 Java

```
import com.etas.vrta.comms.*;
…
serverLink.releaseVECU(vecuLink);
```

12.2.5.3 Ruby

```
require 'VRTAComms'
…

server_link.release_vecu(vecu_link)
```

12.2.5.4 C++

```
#include "VesLib.h
#include "vrtaClientSupport.h"
…
VesLibExit();
delete vecu_link;
```

12.2.6 Terminating a VECU

You can terminate any running VECU. This will cause an orderly shutdown of the application and communications will cease.

12.2.6.1 .NET

```
using VRTAComms;
…
// Easiest
serverLink.CloseVECU(vecuLink);

// Alternative
serverLink.TerminateVECU(vecuLink);
serverLink.ReleaseVECU(vecuLink);
```

12.2.6.2 Java

```
import com.etas.vrta.comms.*;
…
// Easiest
serverLink.closeVECU(vecuLink1);

// Alternative
serverLink.terminateVECU(vecuLink2);
serverLink.releaseVECU(vecuLink2);
```

12.2.6.3 Ruby

In Ruby you tell the server link to close/terminate a vecu link that you have previously obtained.

```
require 'VRTAComms'
…
# Easiest
server_link.close_vecu(vecu_link)

# Alternative
server_link.terminate_vecu(vecu_link)
server_link.release_vecu(vecu_link)
```

12.2.6.4 C++

```
#include "VesLib.h
#include "vrtaClientSupport.h"
…
vecu_link["ApplicationManager"]->
   SendAction(AM_ACTION_ID_Terminate);
```

12.2.7 Finding existing instances of a VECU

Sometimes you may want to know if a specific VECU is running already, and how many instances there are. You can use this information to attach or terminate them all.

12.2.7.1 .NET

```
using VRTAComms;
…
var aliases = serverLink.AliasesFor(path_to_exe);
foreach (var alias_name in aliases) {
    var vecu = serverLink.AttachAlias(alias_name);
    …
    serverLink.CloseVECU(vecu);
}
serverLink.ReleaseAliases(aliases);
…
```

12.2.7.2 Java

```
import com.etas.vrta.comms.*;
…
String[] aliases = serverLink.aliasesFor(path_to_exe);
for (String alias_name : aliases) {
  VECUConnection vecu = serverLink.attachAlias(alias_name);
  //…
  serverLink.closeVECU(vecu);
}
serverLink.releaseAliases(aliases);
```

12.2.7.3 Ruby

```
require 'VRTAComms'
…
aliases = server_link.aliases_for(path_to_exe)
aliases.each do |al|
  vecu_link = server_link.attach(al)

  server_link.close_vecu(vecu_link)
end
server_link.release_aliases(aliases)
…
```

12.2.7.4 C++

```
#include "VesLib.h
#include "vrtaClientSupport.h"
…
int count;
VesLibStatusType status; VesLibEcuAliasType *aliases; VesLibEcuInfo-
Type ecu;

// Find aliases for the VECU
strcpy(ecu.name, path_to_exe);
status = VesLibGetAliases(&ecu, &aliases, &count);
if (count == 0) {
   … // Cannot attach - not loaded
}
for(int i=0; i<count; i++) {
    // Connect to alias aliases[i]
```

```
    // Close connection
    VesLibFreeAlias(aliases[i]);
}

VesLibFreeMemory(aliases); // Allocated in VesLibGetAliases
```

12.2.8 Finding all loaded VECUs

Sometimes you may want to know if any VECUs are loaded, and how many there are. You can use this information to attach or terminate them all.

12.2.8.1 .NET

```
using VRTAComms;
…
var aliases = serverLink.LoadedAliases();
foreach (var alias_name in aliases) {
    var vecu = serverLink.AttachAlias(alias_name);
    …
    serverLink.CloseVECU(vecu);
}
serverLink.ReleaseAliases(aliases);
```

12.2.8.2 Java

```
import com.etas.vrta.comms.*;
…
String[] aliases = serverLink.loadedAliases();
for (String alias_name : aliases) {
  VECUConnection vecu = serverLink.attachAlias(alias_name);
  //…
  serverLink.closeVECU(vecu);
}
serverLink.releaseAliases(aliases);
```

12.2.8.3 Ruby

```
require 'VRTAComms'
…
aliases = server_link.loaded_aliases
aliases.each do |al|
  vecu_link = server_link.attach(al)

  server_link.close_vecu(vecu_link)
end
server_connection.release_aliases(aliases)
…
```

12.2.8.4 C++

```
#include "VesLib.h
#include "vrtaClientSupport.h"
…
int count;
VesLibStatusType status;
VesLibEcuAliasType *aliases;

// Find aliases
status = VesLibListLoadedECUs(&aliases, &count);
if (count == 0) {
```

```
   … // Cannot attach - not loaded
}

for(int i=0; i<count; i++) {
    // Connect to alias aliases[i]

    // Close connection
    VesLibFreeAlias(aliases[i]);
}

VesLibFreeMemory(aliases); // Allocated in VesLibListLoadedECUs
```

12.2.9 Searching for VECUs on disk

Client programs such as vrtaMonitor allow the user to navigate the file system of the vrtaServer host PC in order to locate VECU executables. This is achieved by requesting file and directory information from vrtaServer. An initial path is sent to vrtaServer, and it responds with a collection of information about files and directories.

You typically use this iteratively and interactively. You might start at c:\, but you can then show the user its sub-folders. If one is chosen, you then repeat the call for that, e.g. c:\MyECUs.

12.2.9.1 .NET

In .NET, the file system information is returned in a `FileSystemInfo` struct.

```
using VRTAComms;
…
var info = serverLink.FindECUs(".");
if (info.Path.Length > 0) {
    String path = info.Path;
    String[] drives = info.Drives;
    String[] entries = info.Entries;
    String[] folders = info.Folders;
}
```

12.2.9.2 Java

In Java, the file system information is returned in a `FileSystemInfo` class.

```
import com.etas.vrta.comms.*;
…
FileSystemInfo info = serverLink.findECUs(".");
if (info.Path.length() > 0) {
    String path = info.Path;
    String[] drives = info.Drives;
    String[] entries = info.Entries;
    String[] folders = info.Folders;
}
```

12.2.9.3 Ruby

In Ruby, the file system information is returned in a Hash

```
require 'VRTAComms'
…
info = server_link.find_ecus('c:\my_vecus')

# info[:path]echos back the requested path
#            but with trailing separator
# info[:drives] is an array of drive IDs
#            e.g. ['C:\', 'D:\']
# info[:folders] is an array of subfolders
#            e.g. ['.', '..', 'lib']
# info[:ecus] is an array of VECU file names in the path
#            e.g. ['one.exe', 'two.exe']
```

12.2.9.4 C++

In C++, the first parameter to VesLibFindECUs is the path to search. On successful return `*results` points to a '`\n`' separated list of the VECU executables found in the specified directory plus directory information that allows remote navigation of the directories available on `vrtaServer`'s PC.

`*results` contains the following items in the order specified below:

- The current path.
- A comma separated list of drives that are readable.
- The subdirectories of `dir` (one per line), including "`..`" for a non-root directory.
- A blank line.
- A list of files in `dir` that may be valid Virtual ECU applications.

```
#include "VesLib.h
#include "vrtaClientSupport.h"
…
VesLibStatusType status;
char * results;

status = VesLibFindECUs(".", &results);

VesLibFreeMemory(results);
```

12.2.10 Find files

Client programs can also navigate the file system of the vrtaServer host PC in order to locate arbitrary files – e.g. test logs. This is done in the same way as finding VECUs, but a second file-pattern gets sent to help select file types.

12.2.10.1 .NET

In .NET, the file system information is returned in a `FileSystemInfo` struct.

```
using VRTAComms;
…
var info = serverLink.FindFiles(".", "*.log");
if (info.Path.Length > 0) {
    String path = info.Path;
    String[] drives = info.Drives;
    String[] entries = info.Entries;
    String[] folders = info.Folders;
}
```

12.2.10.2 Java

In Java, the file system information is returned in a `FileSystemInfo` class.

```
import com.etas.vrta.comms.*;
…
FileSystemInfo info = serverLink.findFiles(".", "*.log");
if (info.Path.length() > 0) {
    String path = info.Path;
    String[] drives = info.Drives;
    String[] entries = info.Entries;
    String[] folders = info.Folders;
}
```

12.2.10.3 Ruby

In Ruby, the file system information is returned in a Hash

```
require 'VRTAComms'
…
info = server_link.find_files('c:\my_vecus', '*.log')

# info[:path]echos back the requested path
#            but with trailing separator
# info[:drives] is an array of drive IDs
#            e.g. ['C:\', 'D:\']
# info[:folders] is an array of subfolders
#            e.g. ['.', '..', 'lib']
# info[:files] is an array of file names in the path
#            e.g. ['one.log']
```

12.2.10.4 C++

In C++, the first parameter to VesLibFindECUs is the path to search. The second is the pattern to match. On successful return `*results` points to a '`\n`' separated list of the matching files found in the specified directory plus directory information that allows remote navigation of the directories available on `vrtaServer`'s PC.

`*results` contains the following items in the order specified below:

- The current path.
- A comma separated list of drives that are readable.
- The subdirectories of `dir` (one per line), including "`..`" for a non-root directory.
- A blank line.
- A list of files in `dir matching the pattern`.

```
#include "VesLib.h
#include "vrtaClientSupport.h"
…
VesLibStatusType status;
char * results;

status = VesLibFindFiles(".","*.log", &results);

VesLibFreeMemory(results);
```

12.2.11 Read file

Client programs such can read files from the file system of the vrtaServer host. Often this is used to read back test log files.

12.2.11.1 .NET

```
using VRTAComms;
…
String txt = serverLink.ReadFile("c:\\my_vecus\\one.log");
```

12.2.11.2 Java

```
import com.etas.vrta.comms.*;
…
String txt = serverLink.readFile("c:\\my_vecus\\one.log");
```

12.2.11.3 Ruby

In Ruby, simply pass the name of the file to the server link. The file content is returned as a string. Files do not have to be text files – the content is returned byte-for-byte.

```
require 'VRTAComms'
…
content = server_link.read_file('c:\my_vecus\one.log')
```

12.2.11.4 C++

In C++, the file content is read into a buffer that must be freed when no longer needed.

```
#include "VesLib.h
#include "vrtaClientSupport.h"
…
VesLibStatusType status;
char * content;

status = VesReadFile("c:\\my_vecus\\one.log", &content);

VesLibFreeMemory(content);
```

12.3 Communication with VECUs

The previous section showed how to get hold of an object that provides a communications channel to a VECU.

This section describes what you can do with this object.

12.3.1 Get / change VECU state

12.3.1.1 .NET

```
using VRTAComms;
…
// Read VECU state as string.
// Return one of 'Loaded', 'Running', 'Paused',
// 'Terminating', 'Resetting' or 'Terminated'
String state = vecuLink.State();

// Start a loaded VECU
vecuLink.Start();

// Pause the OS thread of the VECU
vecuLink.Pause();

// Resume a paused thread
vecuLink.Resume();

// Reset the VECU
vecuLink.Reset();

// Terminate the VECU
vecuLink.Terminate();
```

12.3.1.2 Java

```
import com.etas.vrta.comms.*;
…
// Read VECU state as string.
// Return one of 'Loaded', 'Running', 'Paused',
// 'Terminating', 'Resetting' or 'Terminated'
String state = vecuLink.state();
System.out.println(state);

// Start a loaded VECU
vecuLink.start();

// Pause the OS thread of the VECU
vecuLink.pause();

// Resume a paused thread
vecuLink.resume();

// Reset the VECU
vecuLink.reset();

// Terminate the VECU
vecuLink.terminate();
```

12.3.1.3

12.3.1.4 Ruby

```
require 'VRTAComms'
```

```
…

# Read VECU state as string.
# Return one of 'Loaded', 'Running', 'Paused',
# 'Terminating', 'Resetting' or 'Terminated'
state = vecu_link.state

# Start a loaded VECU
vecu_link.start

# Pause the OS thread of the VECU
vecu_link.pause

# Resume a paused thread
vecu_link.resume

# Reset the VECU
vecu_link.reset

# Terminate the VECU
vecu_link.terminate
```

12.3.1.5 C++

```
#include "VesLib.h
#include "vrtaClientSupport.h"
…

// Get a handle on the ApplicationManager
vrtaECUDevice *app_manager_device = vecu_link[AM_DEVICE_ID];

// Get a handle on its State event
vrtaECUDeviceEvent *app_state_event =
   app_manager_device->Events()[AM_EVENT_ID_State];

// Read VECU state as string.
// Return one of 'Loaded', 'Running', 'Paused',
// 'Terminating', 'Resetting' or 'Terminated'
// Note data is buffered in 'LastState'
app_state_event->GetState();
printf("State=%s\n", app_state_event->LastState());

// Start a loaded VECU
app_manager_device->Actions()[AM_ACTION_ID_Start]
     ->SendAction();

// Pause the OS thread of the VECU
app_manager_device->Actions()[AM_ACTION_ID_Pause]
     ->SendAction();

// Resume a paused thread
app_manager_device->Actions()[AM_ACTION_ID_Restart]
     ->SendAction();

// Reset the VECU
app_manager_device->Actions()[AM_ACTION_ID_Reset]
     ->SendAction();

// Terminate the VECU
app_manager_device->Actions()[AM_ACTION_ID_Terminate]
     ->SendAction();
```

12.3.2 Get device information

You can ask a VECU what virtual devices it supports by name.

12.3.2.1 .NET

```
using VRTAComms;
…
var device_names = vecuLink.DeviceList;
foreach (string device_name in device_names) {
    …
}
```

12.3.2.2 Java

```
import com.etas.vrta.comms.*;
…
String[] device_names = vecuLink.deviceList();
for (String device_name : device_names) {
    …
}
```

12.3.2.3 Ruby

```
require 'VRTAComms'
…

# Get array of names of the devices
device_names = vecu_link.devicelist
# => ["DeviceManager", "ICU", "ApplicationManager",…]
```

12.3.2.4 C++

In C++, the device names are returned as a string with a '\n' separator.

```
#include "VesLib.h
#include "vrtaClientSupport.h"
…
// Get a handle on the DeviceManager
vrtaECUDevice *dev_manager_device = vecu_link[DM_DEVICE_ID];

// Get a handle on its List event
vrtaECUDeviceEvent *list_event =
   dev_manager_device->Events()[DM_EVENT_ID_DeviceList];

list_event->GetState();
char *result = info_event->LastState();
```

12.3.3 Get device instances

You can alternatively ask a VECU to give you an object that represents a device. (The behavior of a device object is described in section 12.4)

12.3.3.1 .NET

```
using VRTAComms;
…
// There are 2 ways to get a VECUDevice instance.
// They all return the same object
```

```
// (null if the named device does not exist)
var system_clock_device = vecuLink["SystemClock"];
var system_clock_device = vecuLink.Device["SystemClock"];

// Alternatively you can get a collection of all devices
var devices = vecuLink.Devices;
```

12.3.3.2 Java

```
import com.etas.vrta.comms.*;
…
// Get a VECUDevice instance.
// (null if the named device does not exist)
VECUDevice system_clock_device =
   vecuLink.device("SystemClock");
// Alternatively you can get a collection of all devices
List<VECUDevice> devices = vecuLink.devices();
```

12.3.3.3 Ruby

```
require 'VRTAComms'
…

# There are 3 ways to get a VECUDevice instance.
# They all return the same object
# (nil if the named device does not exist)
system_clock_device = vecu_link.device('SystemClock')
system_clock_device = vecu_link.SystemClock
system_clock_device = vecu_link['SystemClock']

# Alternatively you can get an array of all devices
all_devices = vecu_link.devices()
```

Note in particular `vecu_link.SystemClock`. This is a method created dynamically on the VECU instance that has the same name as the device instance that it returns.

12.3.3.4 C++

```
#include "VesLib.h
#include "vrtaClientSupport.h"
…
// By name
vrtaECUDevice *icu_device = vecu_link["ICU"];

// By ID
vrtaECUDevice *icu_device = vecu_link[ICU_DEVICE_ID];
```

12.4 Communication with Devices

Section 12.3.3 showed how to get hold of an object that represents a virtual device on a VECU. This section describes what you can do with it.

12.4.1 Device information

12.4.1.1 .NET

In .NET, information about the device and its actions and events can be read as a `KeyValuePair<string, string>` collection.

```
using VRTAComms;
…
// Get info for a device
var info = system_clock_device.Info;
foreach (KeyValuePair<string, string> inf in info) {
    Console.WriteLine("{0}=>{1}", inf.Key, inf.Value);
}
// "Type"=>"SystemClock",
// "Description"=>"VECU master 1ms clock.",
// "Version"=>"5.3.0",
// "Action Scale"=>"Change… (Updates Scale Event)",
// …
// "Event Scale"=>"The tick duration in ms (Raise,Poll)",
// …
```

12.4.1.2 Java

In Java, information about the device and its actions and events can be read as a `Map.Entry<String, String>` collection.

```
import com.etas.vrta.comms.*;
…
OptList info = system_clock_device.info();
for (Iterator<Map.Entry<String, String>> it =
    info.entrySet().iterator(); it.hasNext();) {
  Map.Entry<String, String> entry = it.next();
  String key = entry.getKey();
  Object value = entry.getValue();
  System.out.printf("%s=>%s\n", key, value);
}
// "Type"=>"SystemClock",
// "Description"=>"VECU master 1ms clock.",
// "Version"=>"5.3.0",
// "Action Scale"=>"Change… (Updates Scale Event)",
// …
// "Event Scale"=>"The tick duration in ms (Raise,Poll)",
// …
```

12.4.1.3 Ruby

In Ruby, information about the device and its actions and events can be read as a hash.

```
require 'VRTAComms'
…

# Get info for a device
```

```
info = system_clock_device.info

# {
# "Type"=>"SystemClock",
# "Description"=>"VECU master 1ms clock.",
# "Version"=>"5.3.0",
# "Action Scale"=>"Change… (Updates Scale Event)",
# …
# "Event Scale"=>"The tick duration in ms (Raise,Poll)",
# …
# }
```

12.4.1.4 C++

In C++, the device info is returned as a string of “name=value” pairs, separated by '\n'.

```
#include "VesLib.h
#include "vrtaClientSupport.h"
…
// Get a handle on the DeviceManager
vrtaECUDevice *dev_manager_device = vecu_link[DM_DEVICE_ID];

// Get a handle on its Info event
vrtaECUDeviceEvent *info_event =
   dev_manager_device->Events()[DM_EVENT_ID_DeviceInfo];

info_event->GetState("DevName", 7); // 7 is length of DevName
char *result = info_event->LastState();
```

12.4.2 Actions

You can ask a device to give you an object to represent one of its actions. Its behavior is explained in section 12.5.

12.4.2.1 .NET

```
using VRTAComms;
…
// Returns a VECUDeviceAction, or null
VECUDeviceAction scale_action =
  system_clock_device.Action["Scale"];

// Alternatively you can get an array of all actions
var actions = system_clock_device.Actions;

// If you just want to send an action, use the
// Send() method
scale_action.Send(1);
calculator_action.Send(2, 4, "multiply");
```

12.4.2.2 Java

```
import com.etas.vrta.comms.*;
…
// Returns a VECUDeviceAction, or null
VECUDeviceAction scale_action =
  system_clock_device.action("Scale");

// Alternatively you can get an array of all actions
List<VECUDeviceAction> actions =
```

```
   system_clock_device.actions();

// If you just want to send an action, use the
// send() method
scale_action.send(1);
calculator_action.send(2, 4, "multiply");
```

12.4.2.3 Ruby

```
require 'VRTAComms'
…

# Returns a VECUDevice::Action, or nil
action = system_clock_device.action('Scale')

# Alternatively you can get an array of all actions
actions = device.actions

# If you just want to send an action, use the
# action! methods. e.g.
system_clock_device.Start!
calculator_device.Calculate!(2, 4, :multiply)
```

Note the action methods. These are created dynamically on the VECU instance, and use the name of the action followed by the '!' character (following normal Ruby conventions to indicate a possibly dangerous action).

12.4.2.4 C++

```
#include "VesLib.h
#include "vrtaClientSupport.h"
…
vrtaECUDevice *system_clock_device = vecu_link["SystemClock"];

vrtaECUDeviceAction action =
  system_clock_device->Actions()["Scale"];
```

12.4.3 Events

You can ask a device to give you an object to represent one of its events. Its behavior is explained in section 0.

12.4.3.1 .NET

```
using VRTAComms;
…
// Returns a VECUDeviceEvent, or null
VECUDeviceEvent scale_event =
  system_clock_device.Event["Scale"];

// Alternatively you can get an array of all events
var events = system_clock_device.Events;

// If you just want to read the value of an event, use the
// State() method
var scale = scale_event.State();
var value = matrix_read_event.State(1, 3);
```

12.4.3.2 Java

```
import com.etas.vrta.comms.*;
…
// Returns a VECUDeviceEvent, or null
VECUDeviceEvent scale_event =
  system_clock_device.event("Scale");

// Alternatively you can get an array of all events
List<VECUDeviceEvent>  events =
    system_clock_device.events();

// If you just want to read the value of an event, use the
// state() method
int scale = (Integer)scale_event.state();
int value = (Integer)matrix_read_event.state(1, 3);
```

Note that for Java, the state() method returns an object or object-array depending on the number of returned parameters. The types of the object(s) are automatically converted as appropriate based on the event format declaration (numeric, string etc), but Java requires that you cast them correctly when you read them into variables.

12.4.3.3 Ruby

```
require 'VRTAComms'
…
# Returns a VECUDevice::Event, or nil
event = system_clock_device.event('Scale')

# Alternatively you can get an array of all events
events = device.events

# If you just want to read the value of an event, use the
# event state methods. e.g.
scale = system_clock_device.Scale
value = matrix_device.Read(1, 3)
```

Note the event state methods. These are created dynamically on the VECU instance, and use the name of the event.

12.4.3.4 C++

```
#include "VesLib.h
#include "vrtaClientSupport.h"
…
#include "VesLib.h
#include "vrtaClientSupport.h"
…
vrtaECUDevice *system_clock_device = vecu_link["SystemClock"];

vrtaECUDeviceEvent event =
  system_clock_device->Events()["Scale"];
```

12.5 Communication with Actions

Section 12.4.2 showed how to get hold of an object that represents an action on a VECU. This section describes what you can do with it.

12.5.1 Sending the action

Actions are created with zero or more parameters. You can use an action object to send an action message to the VECU

12.5.1.1 .NET

.NET translates the parameters that you supply into the correct types for the VECU. You should use numeric types for parameters that have a %d/%u/%f format and strings for other types.

```
using VRTAComms;
…
// Send an action with no parameters
var action1 =  system_clock_device.Action["Pause"];
action1.Send();

// Send an action with 2 parameters
var action2 = calculator_device.Action("Calculate");
action2.Send(2, 4, "multiply");
```

12.5.1.2 Java

Java translates the parameters that you supply into the correct types for the VECU. You should use numeric types for parameters that have a %d/%u/%f format and strings for other types.

```
import com.etas.vrta.comms.*;
…
// Send an action with no parameters
VECUDeviceAction action1 =
    system_clock_device.action("Pause");
action1.send();

// Send an action with 2 parameters
VECUDeviceAction action2 =
    calculator_device.action("Calculate");
action2.send(2, 4, "multiply");
```

12.5.1.3 Ruby

Ruby translates the parameters that you supply into the correct types for the VECU. You should use numeric types for parameters that have a %d/%u/%f format, symbols for enumerations and strings for other types.

```
# Send an action with no parameters
action1 = system_clock_device.action("Pause")
action1.send
# OR better
system_clock_device.Start!
```

```
# Send an action with 2 parameters
action2 = calculator_device.action("Calculate")
action2.send(2, 4, :multiply)
# OR better
calculator_device.Calculate!(2, 4, :multiply)
```

12.5.1.4 C++

For C++, you need to serialize any output parameters in a “\n” separated string.

```
#include "VesLib.h
#include "vrtaClientSupport.h"
…

action1 =  system_clock_device->Actions()["Pause"];
action1.SendAction();

// Send an action with 2 parameters
action2 = calculator_device->Actions()["Calculate"];
action2->SendAction("2\n4\nmultiply", 12);
```

12.5.2 Action parameters

The VECU supplies information about the parameters associated with each action. You can use this information to generate the correct data types.

12.5.2.1 .NET

Each action has a params method that returns a VECUParams collection of individual VECUParam instances. These can be examined to get the data for each parameter.

```
using VRTAComms;
…
// Get the parameters collection
var scale_params = scale_action.Params;
var num_params = scale_params.Length;

var first_param = scale_params[0];    // null if none…

// The name of the parameter
var name = first_param.Name;

// The descriptive information
var descr = first_param.Info;

// The format string for the parameter
var fmt = first_param.Format;
```

12.5.2.2 Java

Each action has a params method that returns a VECUParams collection of individual VECUParam instances. These can be examined to get the data for each parameter.

```
import com.etas.vrta.comms.*;
…
// Get the parameters collection
VECUParams scale_params = scale_action.params();
```

```
int num_params = scale_params.size();

VECUParam first_param = scale_params.index(0);
  // null if none…

// The name of the parameter
String name = first_param.name();

// The descriptive information
String descr = first_param.info();

// The format string for the parameter
String fmt = first_param.format();
```

12.5.2.3 Ruby

Each action has a params method that returns a VECUParams collection of individual VECUParam instances. These can be examined to get the data for each parameter.

```
# Get the parameters (Enumerable collection)
params = action.params
num_params = params.size

first_param = params[0] # nil if none…

# The name of the parameter
name = first_param.name

# The descriptive information
descr = first_param.info

# The format string for the parameter
fmt = first_param.format
```

12.5.2.4 C++

Each action has a FormatIn method that returns a vrtaFormatString pointer. This can be used to determine the action's formatting information. This API is deprecated and will be replaced in a later update.

12.6 Communication with Events

Section 12.4.3 showed how to get hold of an object that represents an event on a VECU. This section describes what you can do with it.

12.6.1 Polling the event

Events are created with zero or more parameters into the VECU and one or more out. You can use an event object to poll the value of an event from the VECU

12.6.1.1 .NET

.NET translates the parameters that you supply into the correct types for the VECU.

When sending, you should use numeric types for parameters that have a %d/%u/%f format and strings for other types.

When receiving, the types will be converted into numeric types for parameters that have a %d/%u/%f format and strings for other types.

```
using VRTAComms;
…
// Read an event with no parameters
var event1 = system_clock_device.Event["Scale"];
var scale = event1.State();

// Read an event with 2 parameters
var event2 = matrix_device.Event["Read"];
var value = event2.State(1, 3);
```

12.6.1.2 Java

Java translates the parameters that you supply into the correct types for the VECU.

When sending, you should use numeric types for parameters that have a %d/%u/%f format and strings for other types.

When receiving, the types will be converted into numeric types for parameters that have a %d/%u/%f format and strings for other types.

```
import com.etas.vrta.comms.*;
…
// Read an event with no parameters
VECUDeviceEvent event1 =
    system_clock_device.event("Scale");
int scale_value = (Integer)event1.state();

// Read an event with 2 parameters
VECUDeviceEvent event2 = matrix_device.event("Read");
int value = (Integer)event2.state(1, 3);
```

Note that for Java, the state() method returns an object or object-array depending on the number of returned parameters. The types of the object(s) are automatically converted as appropriate based on the event format declaration (numeric, string etc), but Java requires that you cast them correctly when you read them into variables.

12.6.1.3 Ruby

Ruby translates the parameters that you supply into the correct types for the VECU.

When sending, you should use numeric types for parameters that have a %d/%u/%f format, symbols for enumerations and strings for other types.

When receiving, the types will be converted into numeric types for parameters that have a %d/%u/%f format, symbols for enumerations and strings for other types.

```
# Read an event with no parameters
event1 = system_clock_device.event("Scale")
scale = event1.state
# OR better
scale = system_clock_device.Scale


# Read an event with 2 parameters
event2 = matrix_device.event("Read")
value = event2.state(1, 3)
# OR better
value = matrix_device.Read(1, 3)
```

12.6.1.4 C++

For C++, you need to serialize any output parameters in a “\n” separated string. Returned data is similarly serialized.

```
#include "VesLib.h
#include "vrtaClientSupport.h"
…

// Read an event with no parameters
event1 = system_clock_device->Events()["Scale"];
event1->GetState();
char *result1 = event1->LastState();

// Send an action with 2 parameters
event2 = calculator_device->Events()["Read"];
event2->GetState("1\n3 ", 3);
char *result2 = event2->LastState();
```

12.6.2 Event parameters

The VECU supplies information about the in/out parameters associated with each event. You can use this information to generate the correct data types.

12.6.2.1 .NET

Each event has a ParamsToECU and ParamsFromECU field that returns a VECUParams collection of individual VECUParam instances. These can be examined to get the data for each parameter in the same way as for actions.

```
using VRTAComms;
…
var scale_in_params = scale_event.ParamsToECU;
var scale_out_params = scale_event.ParamsFromECU;
```

12.6.2.2 Java

Each event has a params_to_ECU and params_from_ECU field that returns a VECUParams collection of individual VECUParam instances. These can be examined to get the data for each parameter in the same way as for actions.

```
import com.etas.vrta.comms.*;
…
VECUParams scale_in_params =
    scale_event.params_to_ECU();

VECUParams scale_out_params =
    scale_event.params_from_ECU();
```

12.6.2.3 Ruby

Each event has a params_to_ecu and params_from_ecu method that returns a VECUParams collection of individual VECUParam instances. These can be examined to get the data for each parameter.

```
# Get the parameters (Enumerable collection)
params = event.params_from_ecu
num_params = params.size

first_param = params[0] # nil if none…

# The name of the parameter
name = first_param.name

# The descriptive information
descr = first_param.info


# The format string for parameter
fmt = first_param.format
```

12.6.2.4 C++

Each event has FormatIn and FormatOut methods that return a vrtaFormatString pointer. This can be used to determine the event's formatting information. This API is deprecated and will be replaced in a later update.

12.7 Hooking Events

We have seen that you can easily read the value of events using the event state methods. Each time you call an event state method, a message is sent to the VECU and the current value of the event is returned. This is known as **polling**, and is useful in many applications.

However sometimes it becomes inefficient. If you want to know when an event value changes, then there is a temptation to keep polling the VECU at increasingly short intervals. This just increases the workload on your PC, and still runs the risk of missing changes.

It is much more efficient to have the VECU send you a message when the value of an event changes. To do this, you tell the VECU which events you want to monitor, and provide a callback that will be run when the event changes (gets 'raised') on the VECU. This is known as **hooking** an event.

Most (but not all) events can be hooked. If you look at the description of each event (e.g. using vrtaMonitor or 'vrta describe'), each event will state whether it can be polled "(Poll)", hooked "(Raise)" or both "(Raise,Poll)".

On important point to bear in mind is that the hook callbacks run in their own thread, so will execute asynchronously from your main code. You will need to consider thread-safety issues if you update data in the hook callback that is shared by your main thread.

12.7.1.1 .NET

You create an event hook by specifying the event and providing a callback. The callback arguments match the `ParamsFromEcu` declarations, and are converted to the appropriate types (Numeric types for parameters that have a %d/%u/%f format and strings for other types).

You can add and remove event hooks at any time.

Hooked events are timestamped on the VECU with microsecond precision. The timestamp is not provided to the hook by default because is it not normally needed. You can get the timestamp by reading the value `CurrentEvent.Time` within the hook.

```
using VRTAComms;
…
var values = new List<ulong>();
var times = new List<uint>();
```

```
// Hooking from the Event level
system_clock_remaining_event.Hook(value => {
    values.Add(value);
    times.Add(CurrentEvent.Time);
});

// Hooking from the VECU level
vecuLink.ApplicationManager.Hook("Paused",
    val => pauses += 1
);
vecuLink.Event("ApplicationManager.Restarted").Hook(
    val => restarts += 1
);

// Unhooking all events
vecuLink.UnhookAll();
```

12.7.1.2 Java

You create an event hook by specifying the event and providing a callback in the form of the values() method in an anonymous EventHook class. The callback arguments match the params_from_ECU declarations, and are converted to the appropriate types (Numeric types for parameters that have a %d/%u/%f format and strings for other types).

You can add and remove event hooks at any time.

Hooked events are timestamped on the VECU with microsecond precision. The timestamp is not provided to the hook by default because is it not normally needed. You can get the timestamp by reading the value `this.time` within the hook.

```
import com.etas.vrta.comms.*;
…
final List<Object> values = new ArrayList<Object>();
final List<Long> times = new ArrayList<Long>();

// Hooking from the Event level
system_clock_remaining_event.hook(new EventHook() {
  @Override
  public void values(Object... values) {
    values.add(values[0]);
    times.add(this.time);
  }
});

// Hooking from the VECU level
final int[] pauses = new int[1];
final int[] restarts = new int[1];
vecuLink.applicationManager().hook("Paused",
 new EventHook() {
  public void values(Object... values) {
    pauses[0] += 1;
  }
});
vecuLink.event("ApplicationManager.Restarted").hook(
new EventHook() {
  public void values(Object... values) {
```

```
    restarts[0] += 1;
  }
});

// Unhooking all events
vecuLink.unhookAll();
```

12.7.1.3 Ruby

You create an event hook by specifying the event and providing a block as the callback. The block arguments match the `params_from_ecu` declarations, and are converted to the appropriate types (Numeric types for parameters that have a %d/%u/%f format, symbols for enumerations and strings for other types).

You can add and remove event hooks at any time.

Hooked events are timestamped on the VECU with microsecond precision. The timestamp is not provided to the hook by default because is it not normally needed. You can get the timestamp by reading the value `VRTAComms::current_event[:time]` within the hook.

```
# Hooking from the Event level
ticks = []
system_clock_remaining_event.hook do |val|
  ticks << [val, VRTAComms::current_event[:time]]
end
…
system_clock_remaining_event.unhook

# Hooking from the VECU level
vecu_link.ApplicationManager.event('Paused').hook do
  pauses += 1
end

vecu_link.event('ApplicationManager Restarted').hook do
  restarts += 1
end

# Unhooking all events
vecu_link.unhook_all
```

12.7.1.4 C++

In C++, you have to set up a separate listening thread and manage the communication and deserialization. Code to support this is in vrtaClientSupport.cpp, but a full description is beyond the scope of this document. Please contact support for more detailed code examples.

13 Command Line Option Reference

This chapter provides a list of the command line options that are supported by Virtual ECU executables, `vrtaServer` and `vrtaMonitor`.

13.1 Virtual ECU Executables

The command line options listed below can be used when a Virtual ECU executable is run to control the behavior of the Virtual ECU. Note that options other than those listed below may be used with a Virtual ECU executable and they will be ignored by VRTA but can be recovered by your code by querying the ApplicationManager.

13.1.1 -alias=name (override the default alias)

When a Virtual ECU registers with `vrtaServer` it is normally assigned an alias that is simply the name of the Virtual ECU's executable (e.g. `vecu.exe`). The option `-alias=name` causes the token `name` to be used as the Virtual ECU's alias (if possible).

13.1.2 -priority=‹n› (set the priority)

By default a Virtual ECU runs at the `NORMAL_PRIORITY_CLASS`. If you wish to run a Virtual ECU at a different priority the `-priority=<n>` option can be used; where `<n>` is:

‹N›	PRIORITY CLASS
0	IDLE_PRIORITY_CLASS
1	BELOW_NORMAL_PRIORITY_CLASS
2	NORMAL_PRIORITY_CLASS
3	ABOVE_NORMAL_PRIORITY_CLASS
4	HIGH_PRIORITY_CLASS
5	REALTIME_PRIORITY_CLASS

Increasing the priority of a Virtual ECU will improve how closely it approximates "real-time" behavior but will negatively affect the performance of other applications running in the same PC.

13.1.3 -core=‹n› (set the physical core)

(VRTA-ux)

Normally we allow Linux to distribute VECU threads across its physical cores automatically. This generally results in best performance.

You can alternatively use this option to tell Linux to use a preferred physical core to run all threads on. This can be useful when you want to run multiple VECUs and run them on individual cores.

Selecting core 0 is the same as omitting this option. We have found that forcing a VECU on to core 0 can steal too much time from some Linux systems and make them unstable.

13.1.4 -silent (select silent or GUI mode)

(VRTA-win)

The `-silent` option causes a Virtual ECU to be loaded in silent mode. In silent mode the Virtual ECU does not display its own (embedded) GUI (it is assumed that `vrta-Monitor` or a similar program will be used to control the Virtual ECU).

If the `-silent` option is not used then the Virtual ECU is loaded in GUI mode. In *GUI* mode the Virtual ECU displays its own GUI.

13.1.5 -noserver (do not contact vrtaServer)

The `-noserver` option tells the Virtual ECU not to contact vrtaServer. This may be suitable in some environments where external programs do not need to communicate with the VECU via its diagnostic interface.

13.1.6 -slave (select slave or autostart mode)

The `-slave` option causes a Virtual ECU to be loaded in slave mode. In slave mode the OS thread is not started immediately after the `vrtaStart()` API has been called. Instead the VECU waits until a Start action is sent to the Application Manager before starting the OS thread.

In slave mode the Virtual ECU may not terminate immediately after the OS thread returns from `OS_MAIN()`. Instead the Virtual ECU waits until a Terminate action is sent to the Application Manager.

If the `-slave` option is not used then the Virtual ECU loads in autostart mode. In autostart mode the OS thread starts immediately after the `vrtaStart()` API has been called.

In autostart mode the Virtual ECU terminates immediately after the OS thread returns from `OS_MAIN()`.

13.2 vrtaServer.exe

`vrtaServer.exe` supports the command line options listed below.

13.2.1 -install (install as a service)

The `-install` option causes `vrtaServer` to install itself as a Windows service. Unless the `-silent` option is also specified a confirmation message will be displayed.

13.2.2 -p<n> (specify the TCP port)

`vrtaServer` searches a pre-defined set of TCP ports for an empty port on which to listen for connections. The `-p<n>` option forces `vrtaServer` to listen on TCP port `<n>` for connections. `<n>` can be a decimal or hexadecimal (`0x` prefix) number.

13.2.3 -silent (silent install or uninstall)

Installation and un-installation as a Windows service normally generates a confirmation message. The `-silent` option stops the confirmation message being displayed. See `-install` and `-uninstall`.

13.2.4 -standalone (run in standalone mode)

The `-standalone` option causes `vrtaServer` to run as a standalone Windows application rather than as a service. If you want to manually run `vrtaServer` rather than installing it as a service then use the `-standalone` option.

13.2.5 -start (start the vrtaServer service)

If `vrtaServer` is installed as service but has not been started then the `-start` option will cause the service to be started. You do not normally have to start the service yourself – a VECU or vrtaMonitor will start the service if it needs to.

13.2.6 -stop (stop the vrtaServer service)

If `vrtaServer` is installed as service and has been started then the `-stop` option will cause the service to be stopped.

13.2.7 -uninstall (un-install as a service)

The `-uninstall` option causes `vrtaServer` to uninstall itself as a Windows service. If the `-silent` option is not also specified a confirmation message will be displayed.

13.3 vrtaMonitor

`vrtaMonitor` supports two sets of command line options. The *global* options affect the overall operation of `vrtaMonitor`. It does not matter where the global options appear in the command line.

The *sequential* options are processed in the order they appear on the command line.

13.3.1 Command Files

Command line options can be passed to `vrtaMonitor` in a command file. If `vrtaMonitor` encounters a statement of the form `@<file>` on its command line it will

start processing command line options from the file <file>. Each option in <file> must be on a separate line.

13.4 vrtaMonitor Global Options

The following options affect the overall operation of vrtaMonitor.

13.4.1 -f‹filename› (close when ‹filename› appears)

The –f<filename> options tells vrtaMonitor to run until the file <filename> appears and then to terminate.

13.4.2 -k (terminate with specific error level)

The –k option causes vrtaMonitor to terminate with a specific error level when certain events occur.

13.4.3 -log=‹file› (write to a log file)

The –log=<file> options causes vrtaMonitor to log activity to the file <file>.

13.4.4 -scripter=‹name› (select a scripting engine)

The –scripter=<name> option selects the scripting engine called rtaScript<name>.dll.

13.4.5 -t‹n› (close after ‹n› seconds)

The –t<n> options tells vrtaMonitor to run for <n> seconds and then terminate.

13.5 vrtaMonitor Sequential Options

The following options are processed in the order they appear on the command line.

13.5.1 Virtual ECU Executable Name (auto-load)

If vrtaMonitor encounters the name of a Virtual ECU executable on its command line it attempts to load the named Virtual ECU. The Virtual ECU executable may be on the local PC or a remote PC depending on whether or not the –host option has been used. The –d, –r, –n and –g options affect how the Virtual ECU is auto-loaded.

13.5.2 -alias=‹name› (connect to VECU)

The –alias=<name> option tells vrtaMonitor to try and connect to an existing (loaded) Virtual ECU that has the alias <name>. By default vrtaMonitor assumes the Virtual ECU is on the local PC. If it is not the –host option should be used.

13.5.3 -d (load but not start a Virtual ECU)

The –d options causes the next Virtual ECU auto-loaded to be loaded but not started.

13.5.4 -g (load with a GUI)

The –g options causes the next Virtual ECU auto-loaded to run with an embedded GUI (i.e. run in GUI mode).

13.5.5 -host=‹hostname› (select a remote PC)

The –host=<hostname> option selects the remote PC for the –alias and auto-load options. <hostname> is the host name of the remote PC.

13.5.6 -mon=‹dev›.‹event› (monitor event)

The –mon=<dev>.<event> options tells vrtaMonitor to monitor the event called <event> from the device called <dev> in the Virtual ECU to which vrtaMonitor has most recently attached (–alias) or auto-loaded.

13.5.7 –n (load without a GUI)

The –n options causes the next Virtual ECU auto-loaded to run without an embedded GUI (i.e. run in silent mode). This is the default when auto-loading a Virtual ECU.

13.5.8 -p‹n› (select TCP port)

The –p<n> option tells vrtaMonitor that vrtaServer is listening on port <n>. By default vrtaMonitor looks for vrtaServer on a set of pre-defined TCP port numbers. <n> may be a decimal or hexadecimal (0x prefix) number.

13.5.9 -quit (terminate)

The –quit option causes vrtaMonitor to terminate.

13.5.10 –r (load and start a VECU)

The –r options causes the next Virtual ECU auto-loaded to be loaded and started. This is the default when auto-loading a Virtual ECU.

13.5.11 -script=‹file›

The –script=<file> option causes vrtaMonitor to run the script in the file <file>. Please contact ETAS if you need information on monitor scripts.

13.5.12 -send=‹dev›.‹act› (send action)

The –send=<dev>.<act> options tells vrtaMonitor to send the data-less action called <act> to the device called <dev> in the Virtual ECU to which vrtaMonitor has most recently attached (–alias) or auto-loaded.

13.5.13 -send=‹dev›.‹act›(‹str›) (send action)

The –send=<dev>.<act>(<str>) options tells vrtaMonitor to send the action called <act> to the device called <dev> in the Virtual ECU to which vrtaMonitor

has most recently attached (`-alias`) or auto-loaded. The string `<str>` is sent as action data.

13.5.14 -start (start a VECU)

The `-start` option tells `vrtaMonitor` to send a Start action to the Application Manager of the next Virtual ECU auto-loaded.

13.5.15 -wait=<n> (wait)

The `-wait=<n>` option causes `vrtaMonitor` to wait for approximately `<n>` milliseconds before processing the next command line option.

14 Glossary

TERM	DEFINITION
ACTION	AN ACTION IS A COMMAND SENT TO A VIRTUAL DEVICE.
APPLICATIONMANAGER (AM)	THE VIRTUAL DEVICE THAT CONTROLS ASPECTS OF THE OS THREAD.
OS THREAD	THIS IS THE THREAD THAT RUNS THE VIRTUAL ECU APPLICATION CODE – INCLUDING AUTOSAR TASKS. THE OS THREAD IS CREATED WHEN THE VRTASTART() API IS CALLED. THE OS THREAD'S ENTRY POINT IS THE FUNCTION CALLED OS_MAIN().
AUTOSAR	AUTOSAR is a partnership that is seeking to establish an open standard for automotive E/E architecture. See http://www.autosar.org
AUTOSTART MODE	IN AUTOSTART MODE THE OS THREAD STARTS IMMEDIATELY AFTER THE VRTASTART() API HAS BEEN CALLED AND THE VIRTUAL ECU TERMINATES AUTOMATICALLY AFTER THE OS THREAD RETURNS FROM OS_MAIN().
DEVICE COMMUNICATION LINK	A VIRTUAL ECU HAS A DEVICE COMMUNICATION LINK THAT CAN BE USED BY EXTERNAL APPLICATIONS TO MONITOR AND MANAGE THE VIRTUAL ECU. THIS DEVICE COMMUNICATION LINK USES TCP/IP. THE VRTASERVER APPLICATION KEEPS TRACK OF THE PORT NUMBERS USED BY VIRTUAL ECUS ON ITS LOCAL PC.
ECU	ELECTRONIC CONTROL UNIT
EVENT	AN EVENT IS A SIGNAL GENERATED (RAISED) BY A VIRTUAL DEVICE TO INFORM INTERESTED PARTIES THAT SOMETHING HAS HAPPENED. EVENTS MAY OR MAY NOT CONTAIN DATA.
GUI MODE	IN GUI MODE THE VIRTUAL ECU DISPLAYS ITS OWN GUI.
INTERRUPT CONTROL UNIT (ICU)	THE VIRTUAL DEVICE THAT CONTROLS ASPECTS OF THE APPLICATION'S VIRTUAL INTERRUPTS.
SILENT MODE	IN SILENT MODE THE VIRTUAL ECU DOES NOT DISPLAY ITS OWN GUI.

SLAVE MODE	IN SLAVE MODE THE OS THREAD IS NOT STARTED IMMEDIATELY AFTER THE VRTASTART() API HAS BEEN CALLED. INSTEAD THE VIRTUAL ECU WAITS UNTIL A START ACTION IS SENT TO THE DEVICE MANAGER BEFORE STARTING THE OS THREAD. IN *SLAVE* MODE THE VIRTUAL ECU DOES NOT TERMINATE IMMEDIATELY AFTER THE OS THREAD RETURNS FROM OS_MAIN(). INSTEAD THE VIRTUAL ECU WAITS UNTIL A TERMINATE ACTION IS SENT TO THE DEVICE MANAGER.
VIRTUAL DEVICES	VIRTUAL DEVICES ARE SOFTWARE COMPONENTS WITHIN VIRTUAL ECUS THAT SIMULATE HARDWARE DEVICES. VIRTUAL DEVICES INCLUDE: CLOCKS, COUNTERS, SENSORS, ACTUATORS AND CAN CONTROLLERS.
VIRTUAL ECU (VECU)	A VIRTUAL ECU IS COMPOSED OF AN APPLICATION, VIRTUAL DEVICES AND POSSIBLY AN RTA-OS KERNEL. A VIRTUAL ECU IS THE *VRTA* ANALOG OF AN APPLICATION RUNNING ON A REAL ECU.
vrtaMonitor	THE VRTAMONITOR(.EXE) PROGRAM IS AN APPLICATION SUPPLIED WITH *VRTA* THAT ALLOWS VIRTUAL ECUS RUNNING ON THE SAME OR A REMOTE PC TO BE MONITORED AND MANAGED.
VRTA	VIRTUAL RTA-OS : A SHORT FORM USED FOR THE *VRTA* PRODUCTS FOR PC AND LINUX.
vrtaServer	THE VRTASERVER PROGRAM IS AN APPLICATION SUPPLIED WITH *VRTA* THAT MANAGES ACCESS TO VIRTUAL ECU.

Index

_

_Parent event 77

A

action callback function 62, 63, 74, 130
Actions 20, 60, 84, 129, 247
Actuators 42
alias 81, 240, 243
AM 144
Application Manager 20, 144
AsyncGetState 74
AUTOSAR 15, 247
autostart mode 111, 136, 241, 247

C

C++
  Exporting functions to C 47
C++ name mangling 100
CAPCOM
  see Compare Devices 40
Clear 142
clock tick interval 151
Clocks 40
command file 87, 242
command line 147, 148, 240
command-line option 146
command-line parameter 146
Compare Devices 40
constraint 109
–core 240
Counters 40
Ctrl+A 86
Ctrl+Alt+R 85
Ctrl+Alt+S 84
Ctrl+L 83
Ctrl+R 85
Ctrl+S 84
current IPL 141, 142, 144
Custom Devices 43

D

-d 243
Data format Strings 106
Device Communication Interface 21
Device Communication Link 247
Device Manager 20, 138
DeviceActions 140
DeviceEvents 140
DeviceInfo 141
DeviceList 140
DeviceManager 21
Devices
  Initialization 35
  Logger 46
devices.cpp 111
DM 138

E

Embedded GUI 22
EnabledVecs 144
event handler 113, 114, 117
EventRegister 139
events 20, 60, 85, 129, 247

F

-f 243

G

-g 244
GetDeviceActions 140
GetDeviceEvents 140
GetDeviceInfo 140
GetInfo 145
GetIPL 143
GetMatch() 190
GetMax() 168, 173, 177, 183, 186
GetPending 143
GetValue 196
GetValue() 194
GetValues 195, 196
GUI mode 241, 247

H

Hardware drivers 97
high-performance timer 127
HookEvents 139

-host 244

I

ICU 21, 119, 141
IncrementMatch() 191
Info 147
Initialization 98
InitializeDevices 35
InitializeDevices() 110, 136
-install 241
Interfacing C code with C++ Devices 46
Interrupt Control Unit 21, 97, 99, 141
interrupt handler 119, 141
Interrupt number 141
interrupt priorities 22
interrupt priority level 22, 141
interrupt source 97, 99
interrupt vector 22, 119, 141
interrupts 99, 141
Interval 153, 154
IO 43
IPL 22, 141, 144
ISRs 99

K

-k 243

L

ListAll 139
-log 243

M

main() 119
Mask 142
masked 22, 142
masks 142
Match 192
Max() 156, 162
Maximum 160, 166, 170, 171, 176, 179, 180, 185, 188, 189
migration 96
Min() 156, 162
Minimum 160, 166
-mon 244
monitoring an event 86

N

–n 244
Non-volatile Data 57
–noserver 241

O

OnAction 74
Option 147
OptionText 147
OS thread 30, 111, 247
OS_MAIN() 111, 136
Oscillator 40

P

-p 242, 244
ParamText 147
Pause 145
Paused 146
Pending 143
pending flag 141, 142
pending interrupts 143
Position 172, 173
printf() 30
priorities 97
priority 141
–priority 240

Q

-quit 244

R

–r 244
Raise 142
RaiseEvent() 71, 75
RaiseInterrupt() 72, 75
ReadOption 146
ReadParam 146
real hardware 96
real-time behavior 28
remote PC 24, 80
Report 161, 167
Reset 133, 145, 147
Restart 145
Restarted 146
RTVECUErr_Conn 101
RTVECUErr_Dev 101
RTVECUErr_ID 101

RTVECUErr_NONE....101
RTVECUErr_VAL....101
Running....155

S

Scale....154
scaling factor....151, 152
-script....244
-scripter....243
scripting....87
scripting engine....243
-send....244
Sensors....42
Set....160, 161, 166, 167
SetInterval()....152
SetIPL....143
SetMatch()....190
SetMax()....158, 164, 169, 174, 178, 183, 187
SetMin()....157, 163
SetScale()....152
SetVal()158, 164, 169, 172, 175, 179, 181, 184, 187
SetValue()....193
SetValues....194
SetVector()....191
ShutdownHook()....30
-silent....241, 242
Silent mode....241, 247
–slave....241
Slave mode....112, 136, 241, 248
SpawnThread()....78
StackFaultHook()....30
-standalone....242
-start....242, 245
Start....143, 144, 154, 160, 166
Start()....153, 159, 165
Started....146
StartOS()....111
State....148
state callback function....62, 64, 131
-stop....242
Stop....144, 154, 160, 167
Stop()....153, 159, 165
system-tray application....81

T

-t 243
target hardware....96
tasks....97
TCP/IP....139
Terminate....145
Terminated....147
TestOption....146
Threads....54, 77

U

-uninstall....242
Unmask....142
unmasked....22
unmasks....143

V

Value...170, 176, 179, 181, 182, 185, 188, 195, 196
Value().157, 163, 168, 171, 174, 178, 180, 183, 186
Values....195, 196
Vector....192
VECU....248
VECU Interaction Threads....54
Virtual Device....60, 128, 248
Virtual ECU....240, 248
vResetType....131
vrta....26
vrtaActEvlD....100
vrtaAction....60, 102, 130, 134
vrtaActionCallback....130
vrtaActionID....100
vrtaActuator....42, 176, 177
vrtaActuatorDimmableLight....43, 176, 182
vrtaActuatorLight....43, 176, 180
vrtaActuatorMultiColorLight....43, 176, 185
VRTAAutoTick....39
vrtaBoolean....101
vrtaByte....101
vrtaClock....40, 151, 155, 161
vrtaCompare....41, 189
Raising Interrupts....41
with Actuators....43
with Sensors....42
vrtaCore.h....100, 138
vrtaDataLen....102
vrtaDataLenMax....102
vrtaDevice....40, 60, 71, 72, 73, 74, 75, 149
vrtaDevice.h....40, 60, 149
vrtaDevID....100
vrtaDevResetInfo....131

VRTADirectTick....39
vrtaDownCounter....161
vrtaEmbed....102
vrtaEnterUninterruptibleSection()....28, 112
vrtaErrType....101
vrtaEvent....60, 104, 127, 132
vrtaEventCallback....114
vrtaEventID....100
vrtaEventRegister()....113
vrtaEventUnregister()....114
VRTAFireISR....39
vrtaGetState()....64, 74, 115
vrtaHookEvent()....117
vrtaInitialize()....118, 119
vrtaInProcessAPI....38
vrtaIntPriority....100
vrtaIO....43, 192, 193
vrtaIsAppFinished....48
VRTAIsAppFinished....39
vrtaIsAppFinished()....112, 120, 135
vrtaIsAppThread()....121
vrtaIsIdle....48
vrtaIsIdle()....122
vrtaISRID....100
vrtaLeaveUninterruptibleSection()..28, 123
VRTALoad....39
vrtaLoadVM()....118, 124
vrtaLoggerDevice.h....46
vrtamillisecond....101
vrtaMonitor....26, 80, 83, 242, 248
vrtaNoneUserThread()....125
vrtaOptStringlistPtr....106
vrtaOSGetIPL()....125
vrtaOSSetIPL()....126
vrtaRaiseEvent()....64, 126
vrtaReadHPTime()....127
vrtaRegisterVirtualDevice()..61, 63, 64, 128
vrtaReset()....132
vrtaSampleDevices.h....40, 149
vrtaSendAction()....63, 74, 134
vrtaSensor....42, 167
vrtaSensorMultiway....173
vrtaSensorMultiwaySwitch....42, 167, 173
vrtaSensorToggle....171
vrtaSensorToggleSwitch....42, 167, 171
vrtaServer....24, 79, 241, 248
VRTASleep....39
vrtaSpawnThread()....134
VRTAStart....39
vrtaStart()....61, 118, 135, 137
vrtaStateCallback....132
vrtaStdDevices.h....138
VRTAStop....39
vrtaStringlistPtr....106
vrtaTerminate()....136
vrtaTextPtr....106
vrtaTimestamp....101
vrtaTypes.h....100
vrtaUpCounter....155
vrtaVectorTable....119
vrtaVM.h....100
vrtaYield()....122

W

-wait....245
Windows service....241, 242

X

XML file....96

15 Contact Information

Technical Support

For details of your local sales office as well as your local technical support team and product hotlines, take a look at the website: www.etas.com/hotlines

ETAS Headquarters

ETAS GmbH
Borsigstraße 24
70469 Stuttgart
Germany

Phone: +49 711 3423-0
Fax: +49 711 3423-2106
Internet: www.etas.com